



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

— **TELECOM** ESCUELA
TÉCNICA **VLC** SUPERIOR
DE INGENIERÍA DE
TELECOMUNICACIÓN

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

Escuela Técnica Superior de Ingeniería de
Telecomunicación

Desarrollo de un sintetizador de sonido embebido basado
en síntesis wavetable e inteligencia artificial

Trabajo Fin de Grado

Grado en Ingeniería de Tecnologías y Servicios de
Telecomunicación

AUTOR/A: Saez Vega, Alejandro

Tutor/a: López Monfort, José Javier

CURSO ACADÉMICO: 2025/2026

Resumen

Este Trabajo Fin de Grado presenta el diseño, construcción y validación de un sintetizador de sonido digital autónomo que combina la síntesis wavetable con un modelo de aprendizaje profundo. El usuario explora de forma continua un espacio de timbres aprendido con el dedo sobre una pantalla táctil, mientras toca las notas desde un teclado MIDI convencional.

El trabajo se divide en dos fases. En la primera, realizada en el ordenador, se entrena un autoencoder variacional con espacio latente bidimensional sobre un conjunto público de 4358 formas de onda de un ciclo, y con su decodificador se genera una rejilla de 256 tablas que se exporta al sistema embebido. En la segunda, el instrumento no ejecuta ninguna red: un ESP32-S3 interpola entre las tablas y una placa Daisy Seed, basada en un Cortex-M7, reproduce el resultado en tiempo real con transiciones sin discontinuidades, envolvente ADSR y filtro gobernados desde un panel analógico.

El prototipo se integra en una carcasa impresa en 3D con alimentación única. La cadena de datos se verifica muestra a muestra contra una referencia calculada en el ordenador y el aparato se caracteriza eléctricamente. Se recogen también las limitaciones encontradas y las líneas de continuación.

Resum

Aquest Treball Final de Grau presenta el disseny, la construcció i la validació d'un sintetitzador de so digital autònom que combina la síntesi wavetable amb un model d'aprenentatge profund. L'usuari explora de manera contínua un espai de timbres après amb el dit sobre una pantalla tàtil, mentre toca les notes des d'un teclat MIDI convencional.

El treball es divideix en dues fases. En la primera, feta a l'ordinador, s'entrena un autoencoder variacional amb espai latent bidimensional sobre un conjunt públic de 4358 formes d'ona d'un cicle, i amb el seu descodificador es genera una graella de 256 taules que s'exporta al sistema encastrat. En la segona, l'instrument no executa cap xarxa: un ESP32-S3 interpola entre les taules de la graella i una placa Daisy Seed, basada en un Cortex-M7, reproduïx el resultat en temps real amb transicions sense discontinuïtats, envoltant ADSR i filtre governats des d'un panell analògic.

El prototip s'integra en una carcassa impresa en 3D amb alimentació única. La cadena de dades es verifica mostra a mostra contra una referència calculada a l'ordinador i l'aparell es caracteritza elèctricament. S'arrepleguen també les limitacions trobades i les línies de continuació.

Abstract

This Bachelor's Thesis presents the design, construction and validation of a stand-alone digital sound synthesizer that combines wavetable synthesis with a deep learning model. The user explores a learned timbre space continuously with a finger on a touch screen, while playing notes from a conventional MIDI keyboard.

The work is divided into two phases. In the first, carried out on a computer, a variational autoencoder with a two-dimensional latent space is trained on a public set of 4358 single-cycle waveforms, and its decoder is used to bake a grid of 256 tables that is exported to the embedded system. In the second, the instrument runs no neural network at all: an ESP32-S3 interpolates between the tables

of the grid, and a Daisy Seed board, built around a Cortex-M7, plays the result in real time with click-free table transitions, an ADSR envelope and a filter driven from an analogue control panel.

The prototype is housed in a 3D-printed enclosure with a single power supply. The data chain is verified sample by sample against a reference computed on the PC, and the device is characterised electrically. The limitations found and the lines of future work are also reported.

RESUMEN EJECUTIVO

La memoria del TFG del GTIST debe desarrollar en el texto los siguientes conceptos, debidamente justificados y discutidos, centrados en el ámbito de la IT

CONCEPT (ABET)	CONCEPTO (traducción)	¿Cumple? (S/N)	¿Dónde? (páginas)
1.1. Problem statement and opportunity	1.1. Planteamiento del problema y oportunidad	S	1, 2, 11
1.2. Constraints (standards, codes, needs, requirements & specifications)	1.2. Toma en consideración de los condicionantes (normas técnicas y regulación, necesidades, requisitos y especificaciones)	S	3, 13, 17
1.3. Setting of goals	1.3. Establecimiento de objetivos	S	2
2. FORMULATE:	2. FORMULAR:		
2.1. Creative solution generation (analysis)	2.1. Generación de soluciones creativas (análisis)	S	14
2.2. Evaluation of multiple solutions and decision-making (synthesis)	2.2. Evaluación de múltiples soluciones y toma de decisiones (síntesis)	S	14, 26, 41, 48
3. SOLVE:	3. RESOLVER:		
3.1. Fulfilment of goals	3.1. Evaluación del cumplimiento de objetivos	S	63
3.2. Overall impact and significance (contributions and practical recommendations)	3.2. Evaluación del impacto global y alcance (contribuciones y recomendaciones prácticas)	S	63, 64, 73

*A Juan, Soledad, Eva y Claudia por ser pilares
fundamentales en mi vida.*

*A Jorge y Paulina, por hacer de la carrera algo que
siempre recordaré con cariño.*

*Y a todos los que, de una forma u otra, me inspiraron a
unir la ingeniería y el arte.*

Índice

I Memoria

1	Introducción	1
1.1	Contexto y planteamiento del problema	1
1.2	Motivación y oportunidad	2
1.3	Objetivos	2
1.4	Alcance y condicionantes	3
1.5	Estructura de la memoria	4
2	Marco teórico y estado del arte	5
2.1	Fundamentos de síntesis de sonido	5
2.2	Síntesis por tabla de ondas	7
2.2.1	Aliasing y limitación de banda	8
2.3	Autoencoders y espacio latente	8
2.3.1	El autoencoder variacional	9
2.4	Estado del arte	10
2.4.1	Modelos que generan la señal de audio	10
2.4.2	Modelos que generan tablas de onda	10
2.4.3	Posicionamiento de este trabajo	11
3	Diseño del sistema	13
3.1	Requisitos y condicionantes de partida	13
3.2	Alternativas consideradas	14
3.2.1	Ejecutar la red frente a precalcular una rejilla	14
3.2.2	Dónde reside la rejilla	14
3.2.3	Dónde se resuelve la transición entre tablas	15
3.3	Arquitectura resultante	15
3.4	Fase de preparación y fase de ejecución	16
3.5	Parámetros de diseño	17
3.6	Enlaces de comunicación	18
4	Modelo neuronal	21
4.1	El conjunto de datos de partida	21
4.2	Preprocesado	21
4.3	Arquitectura del modelo	23
4.4	Función de pérdida y elección de β	24
4.5	Entrenamiento	25

4.6	El espacio latente resultante	25
4.7	Fidelidad de la reconstrucción	26
4.8	Horneado de la rejilla	28
4.9	Validación previa al embarque	29
5	Firmware de control: ESP32-S3 y CYD	31
5.1	Lectura y calibración del táctil	31
5.1.1	De las cuatro esquinas a las dos ventanas	32
5.2	De la coordenada a la celda	33
5.3	Interpolación bilineal sobre la rejilla	33
5.4	El canal de vuelta y el dibujo de la onda	35
5.5	Envío de la tabla al procesador de audio	36
5.6	Estado del subsistema	37
6	Firmware de audio: Daisy Seed	39
6.1	El recorrido de la tabla	40
6.2	La llegada de la tabla	41
6.3	La transición entre tablas	41
6.4	Nota y dinámica	43
6.5	Envolvente y filtro desde el panel	44
6.6	El display	45
6.7	Arranque, instrumentación y recursos	46
6.8	Estado del subsistema	46
7	Hardware y alimentación	47
7.1	Componentes y equipos	47
7.2	La alimentación	48
7.3	Diseño del circuito y trazado de la placa	50
7.4	El panel de control	52
7.5	La carcasa	53
8	Pruebas y resultados	55
8.1	Validación de la cadena de datos	55
8.2	Continuidad y resistencia	56
8.3	Tensiones de alimentación y rizado	57
8.4	Consumo	58
8.5	La salida de audio	59
8.6	Incidencias detectadas	59
8.7	Balance de la verificación	61
9	Conclusiones y trabajo futuro	63
9.1	Síntesis y aportación del trabajo	63
9.2	Cumplimiento de los objetivos	63
9.3	Aprendizajes del proyecto	64
9.4	Trabajo futuro	64
	Bibliografía	67

II Anexos

A Presupuesto de materiales 71

B Relación con los Objetivos de Desarrollo Sostenible 73

Índice de figuras

2.1	Envolvente ADSR. Los dos primeros tramos duran un valor fijo. El tercer tramo depende de cuánto tiempo se mantenga pulsada la tecla. La liberación empieza al soltarla.	6
2.2	Respuesta en magnitud de un filtro paso bajo, uno paso alto y uno paso banda de segundo orden, todos con la frecuencia de corte en 1 kHz y factor de calidad $Q = 0,707$	7
2.3	Lectura de una tabla de ondas. A la izquierda, la tabla almacenada y la posición actual del acumulador de fase φ . A la derecha, el detalle de la interpolación lineal entre las dos muestras vecinas.	8
3.1	Arquitectura del sistema. Las flechas continuas son los enlaces digitales entre placas, rotuladas con el medio y el tamaño de la trama; las discontinuas, los periféricos del procesador de audio.	15
3.2	Las dos fases de operación. La red neuronal solo interviene en la fase de preparación; durante la ejecución el instrumento se limita a consultar la rejilla, mezclar y reproducir.	17
4.1	Invarianza de la alineación al punto de arranque. La misma onda desplazada 0, 137, 411 y 700 muestras produce cuatro curvas distintas; tras llevar el fundamental a fase cero, las cuatro quedan superpuestas.	23
4.2	Evolución del error de reconstrucción y de la divergencia de Kullback-Leibler a lo largo de las 300 épocas. La línea de puntos del panel izquierdo marca el momento en que la validación deja de mejorar; la del derecho, el final de la rampa de β	25
4.3	Espacio latente aprendido. Cada punto es la media de la distribución que el codificador asigna a una onda. El rectángulo a trazos delimita los percentiles 2 y 98 de cada eje.	26
4.4	Cuatro ondas del conjunto de entrenamiento frente a su reconstrucción, con el error cuadrático medio de cada una. La pérdida de detalle se concentra en los flancos verticales.	27
4.5	Contenido de la rejilla, dibujando uno de cada tres nodos en cada eje para que las ondas resulten legibles. El submuestreo permite ver la variedad de formas repartida por el plano, no la semejanza entre celdas contiguas.	29
5.1	Las dos ventanas del táctil sobre el plano de cuentas crudas del convertidor. La de captura decide qué toques valen y es la que se mapea al rango de la coordenada; la visible solo sirve para acotar el área de dibujo. La pulsación fantasma medida cae dentro de la visible y fuera de la de captura.	33

5.2	Interpolación bilineal para la coordenada (40000, 12000). A la izquierda, la celda alcanzada, los índices de las cuatro vecinas en la rejilla y el peso que recibe cada una. A la derecha, esas cuatro formas de onda y la que resulta de mezclarlas. . . .	34
5.3	Reducción de 1024 a 256 puntos para dibujar, sobre el nodo de la rejilla en el que más se separan los dos métodos. Promediar cada grupo de cuatro muestras y quedarse con una de cada cuatro dan prácticamente lo mismo en el grueso del ciclo, y difieren en los flancos.	35
6.1	Estructura del firmware de audio y los tres ritmos a los que trabaja. Las flechas continuas son la cadena de audio, muestra a muestra; las discontinuas, la información que cruza de un ritmo a otro.	40
6.2	La transición entre dos ondas de celdas contiguas de la rejilla, las mismas que alcanza la coordenada del apartado 5.3. A la izquierda, los pesos a lo largo de los 20 ms. En el centro, las dos tablas y su mezcla a mitad de camino. A la derecha, el nivel de pico durante la transición con mezcla lineal y con mezcla de potencia constante.	42
6.3	A la izquierda, un giro rápido del mando de corte tal y como lo entrega el conversor bloque a bloque, y el mismo giro tras el filtro de un polo. A la derecha, el recorrido del mando llevado a hercios con el mapeo exponencial y con uno lineal.	44
7.1	Esquema de la alimentación. El condensador de reserva C4 queda aguas arriba del interruptor y la rama del procesador de audio es la única con elemento en serie. . .	48
7.2	Trazado de la placa. Los dos módulos van sobre zócalos, la alimentación ocupa el extremo libre de los raíles y todo lo que el usuario toca sube al panel por el mazo. . .	51
7.3	Alzado del panel frontal. El trazo discontinuo es el contorno de las dos placas, que quedan por detrás; el continuo, lo que se ve desde fuera. Cotas en milímetros. . .	52
7.4	La carcasa, renderizada desde el modelo paramétrico. A la izquierda el conjunto montado; a la derecha el despiece, con el panel separado de la bandeja y las dos piezas de la trasera.	54
7.5	El instrumento terminado. A la izquierda, el panel de control con los seis mandos, el selector de tipo de filtro y los huecos de las dos pantallas. A la derecha, en funcionamiento: el dedo fija el punto del espacio latente, la pantalla táctil dibuja la forma de onda que se genera en ese punto y el display de servicio muestra el parámetro que se está editando.	54

Índice de tablas

3.1	Parámetros de diseño fijados antes de la implementación.	18
3.2	Enlaces de comunicación entre placas.	18
4.1	Configuración del autoencoder variacional y del entrenamiento.	24
7.1	Componentes del instrumento.	48
8.1	Validación de la cadena de datos frente a una referencia calculada en el ordenador.	56
8.2	Continuidad y resistencia, con el instrumento sin alimentar. El valor neto descuenta los $0,7 \Omega$ de las puntas. <i>OL</i> indica fuera de escala, es decir, del orden de los megaohmios.	56
8.3	Tensiones continuas referidas a masa, en reposo y en el peor caso. El guion indica un nodo que no se repitió en la segunda pasada.	57
8.4	Consumo del instrumento.	58
8.5	Salida de audio en el jack sin carga. Los valores en dBV se refieren a un voltio eficaz.	59
A.1	Presupuesto de materiales del instrumento.	71
B.1	Grado de relación del trabajo con los Objetivos de Desarrollo Sostenible.	73

Glosario

ADC *Analog-to-Digital Converter*. Conversor analógico-digital.

ADSR *Attack, Decay, Sustain, Release*. Ataque, decaimiento, sostenimiento y liberación.

AKWF *Adventure Kid Waveforms*. Corpus público de formas de onda de un ciclo.

BPF *Band-Pass Filter*. Filtro paso banda.

CRC *Cyclic Redundancy Check*. Comprobación de redundancia cíclica.

CYD *Cheap Yellow Display*. Nombre habitual de la placa ESP32-2432S028R, que integra un ESP32 y una pantalla táctil.

DAC *Digital-to-Analog Converter*. Conversor digital-analógico.

DAW *Digital Audio Workstation*. Estación de trabajo de audio digital.

DMA *Direct Memory Access*. Acceso directo a memoria.

DSP *Digital Signal Processing*. Procesado digital de señal.

FFT *Fast Fourier Transform*. Transformada rápida de Fourier.

FPU *Floating-Point Unit*. Unidad de coma flotante.

GPIO *General Purpose Input/Output*. Entrada/salida de propósito general.

GPU *Graphics Processing Unit*. Unidad de procesamiento gráfico.

HPF *High-Pass Filter*. Filtro paso alto.

I²C *Inter-Integrated Circuit*. Bus serie síncrono de dos hilos.

I²S *Inter-IC Sound*. Bus serie para transmisión de audio digital.

LPF *Low-Pass Filter*. Filtro paso bajo.

LSB *Least Significant Bit*. Bit menos significativo.

MCU *Microcontroller Unit*. Microcontrolador.

MIDI *Musical Instrument Digital Interface*. Protocolo de comunicación entre instrumentos musicales electrónicos.

MSE *Mean Squared Error*. Error cuadrático medio.

OLED *Organic Light Emitting Diode*. Diodo orgánico emisor de luz, empleado en pantallas.

PSRAM *Pseudo-Static Random Access Memory*. Memoria de acceso aleatorio pseudoestática.

RMS *Root Mean Square*. Valor eficaz o media cuadrática.

SPI *Serial Peripheral Interface*. Bus serie síncrono maestro-esclavo.

SVF *State Variable Filter*. Filtro de variables de estado.

TFT *Thin-Film Transistor*. Transistor de película fina, tecnología de las pantallas de matriz activa.

UART *Universal Asynchronous Receiver-Transmitter*. Transmisor-receptor asíncrono universal.

USB *Universal Serial Bus*. Bus serie universal.

VAE *Variational Autoencoder*. Autoencoder variacional.

Parte I

Memoria

Capítulo 1

Introducción

Todo el material de este trabajo está publicado en un repositorio de acceso libre. Incluye los guiones de preparación del conjunto de datos y de entrenamiento del modelo, el firmware de las tres placas, los ficheros parámetros de la carcasa y las fuentes de este documento.

<https://github.com/xalsave/Sintetizador-de-Espacio-Latente>



1.1 Contexto y planteamiento del problema

La síntesis wavetable es una de las técnicas más extendidas en los instrumentos musicales electrónicos actuales. Su principio es sencillo, en lugar de calcular la forma de onda muestra a muestra mediante una expresión matemática, se almacena un ciclo completo de la onda en una tabla y se recorre esa tabla a la velocidad a la que se quiere reproducir la nota. El coste computacional es bajo y el control sobre el timbre es directo, ya que basta con sustituir el contenido de la tabla para obtener un sonido distinto.

El problema aparece al intentar pasar de un timbre a otro de forma continua. Un banco de wavetables es un conjunto discreto, contiene las ondas que contiene y nada más. Promediar dos de ellas sin ningún criterio previo no produce un timbre intermedio convincente, sino cancelaciones entre armónicos que suenan a filtro de peine, porque las dos ondas no comparten una referencia de fase común. Tampoco existe una noción evidente de cercanía entre ondas. Dado un banco de varios miles de formas de onda no hay una ordenación natural que sitúe juntas las que suenan parecido.

Los autoencoders ofrecen una vía para resolver las dos cosas a la vez. Un autoencoder es una red neuronal formada por dos mitades: un codificador, que comprime cada onda a un número reducido de valores; y un decodificador, que reconstruye la onda a partir de esos valores. El conjunto de valores intermedios recibe el nombre de espacio latente. Si ese espacio se restringe a dos dimensiones, cada onda de un conjunto queda situada en un punto de un plano, y las ondas de sonido parecido tienden a agruparse. La variante variacional del autoencoder añade una condición adicional sobre la organización de ese plano. Lo obliga a ser continuo, sin huecos ni saltos bruscos. La consecuencia práctica es que el decodificador puede generar una forma de onda con sentido para cualquier punto del plano, incluidos los que no corresponden a ninguna onda del conjunto original. Ahí es donde nacen los timbres intermedios que un banco discreto no puede proporcionar.

Un plano continuo de dos dimensiones es, además, exactamente lo que un dedo puede controlar sobre una pantalla táctil. Esa coincidencia entre la dimensión del espacio aprendido y la del gesto de control es la que da sentido al instrumento que se plantea en este trabajo.

1.2 Motivación y oportunidad

La idea de generar wavetables mediante redes neuronales no es nueva. Existen trabajos previos que aplican autoencoders a la generación e interpolación de formas de onda de un ciclo, así como sistemas de síntesis neuronal en tiempo real capaces de funcionar sobre hardware de propósito general. Todos ellos, sin embargo, comparten un rasgo común: se materializan como aplicaciones de escritorio o como complementos para estaciones de trabajo de audio digital. El sonido se genera en un ordenador.

La oportunidad que aborda este trabajo es distinta. Se trata de llevar ese mismo concepto a un instrumento físico autónomo basado en microcontroladores de bajo coste que generen el audio en su propio hardware y respondan a un gesto táctil y a un teclado MIDI sin necesidad de un ordenador conectado. La aportación no está, por tanto, en la interpolación neuronal en sí, sino en su integración embebida completa en tiempo real. En resolver el reparto de tareas entre procesadores, la comunicación entre ellos, la latencia, la estabilidad del audio y la construcción del prototipo.

Esa integración impone una restricción que condiciona todo el diseño y que conviene enunciar desde el principio: la red neuronal no puede ejecutarse a frecuencia de audio. Generar 48.000 muestras por segundo mediante inferencia neuronal queda fuera del alcance de un microcontrolador de estas características. Ahora bien, la red no necesita trabajar a esa velocidad. Solo tiene que producir una forma de onda nueva cuando el usuario mueve el dedo, es decir, unas pocas decenas de veces por segundo. La reproducción de esa onda a 48 kHz es una operación aritmética elemental. Separar la generación de la tabla, lenta y esporádica, de su reproducción, rápida y continua, es lo que hace viable el sistema completo.

Desde el punto de vista académico, el trabajo integra contenidos de procesamiento digital de señal, aprendizaje automático, sistemas embebidos, protocolos de comunicación entre dispositivos y diseño electrónico. Es una gran oportunidad para aplicar transversalmente todos los conocimientos y competencias adquiridos en el grado.

1.3 Objetivos

El objetivo general es diseñar, construir y validar un sintetizador digital embebido y autónomo que permita explorar de forma continua un espacio tímbrico aprendido por un autoencoder, empleando una pantalla táctil como superficie de control y una entrada MIDI estándar para la ejecución musical.

De este objetivo general se derivan los siguientes objetivos específicos:

- Construir un conjunto de datos homogéneo a partir de un conjunto de datos público de formas de onda de un ciclo, resolviendo el remuestreo, la normalización de amplitud y la alineación de fase necesarias para que la interpolación posterior no introduzca artefactos.

- Entrenar un autoencoder variacional que comprima cada forma de onda a una coordenada bidimensional y verificar que el espacio latente resultante es continuo y está bien aprovechado.
- Precalcular, a partir del decodificador entrenado, una rejilla de formas de onda que pueda embarcarse en el sistema y exportarla en un formato adecuado para un microcontrolador.
- Implementar un motor de síntesis wavetable en tiempo real capaz de sustituir la tabla en reproducción sin discontinuidades audibles.
- Resolver la comunicación entre los tres microcontroladores que componen el sistema, definiendo los protocolos y verificando que la información llega íntegra.
- Incorporar el control de nota y dinámica mediante MIDI, así como un panel de control analógico para los parámetros de envolvente y filtro.
- Validar cada etapa de forma objetiva, contrastando el comportamiento del sistema embebido con una implementación de referencia en el ordenador siempre que sea posible.
- Integrar el conjunto en un prototipo físico alimentado desde una fuente única, con su propia electrónica de soporte.

1.4 Alcance y condicionantes

El sistema se construye sobre tres microcontroladores con funciones separadas. Una pantalla táctil con controlador ESP32 integrado se encarga de la interfaz de usuario. Un ESP32-S3 aloja la rejilla de formas de onda y calcula la interpolación. Una placa Daisy Seed, basada en un procesador ARM Cortex-M7, genera el audio gracias a su DAC de 24 bits. El teclado que proporciona las notas se conecta por MIDI DIN y se considera un elemento externo e intercambiable, de forma que el entregable del trabajo es un módulo de sonido y no un instrumento con teclado integrado.

El alcance queda acotado por varios condicionantes que conviene dejar explícitos:

- **La red neuronal no se ejecuta en el instrumento.** El decodificador se emplea fuera de línea, en el ordenador, para generar por adelantado la rejilla de formas de onda que se embarca en el sistema. Durante la ejecución solo hay consulta de tabla e interpolación. Se trata de una estrategia de despliegue, no de una renuncia: es la consecuencia directa de la restricción enunciada en el apartado anterior.
- **El espacio latente es bidimensional.** Esta elección viene impuesta por el control táctil, pero supone un cuello de botella considerable para representar varios miles de formas de onda distintas. Su efecto sobre la fidelidad de la reconstrucción se mide y se discute en el capítulo sobre el modelo neuronal.
- **El instrumento es monofónico.** La polifonía no forma parte de los objetivos y se plantea únicamente como línea de trabajo futuro.
- **El hardware es de bajo coste y de disponibilidad comercial.** No se ha diseñado ni fabricado ningún circuito integrado específico, y el montaje final se resuelve sobre placa de tiras de cobre en lugar de sobre un circuito impreso fabricado, por restricciones de plazo.

- **La amplificación y el altavoz integrados quedan fuera del alcance.** La salida del instrumento es una señal de línea por conector de 3,5 mm.

1.5 Estructura de la memoria

El presente documento se organiza en nueve capítulos.

El **capítulo 1** sitúa el problema, expone la motivación del trabajo, fija los objetivos y delimita el alcance.

El **capítulo 2** recoge los fundamentos teóricos necesarios para seguir el resto del documento y revisa los trabajos previos relacionados con sistemas de síntesis neuronal.

El **capítulo 3** describe el diseño del sistema completo: el reparto de funciones entre los tres microcontroladores, los protocolos de comunicación entre ellos y la separación entre la fase de preparación fuera de línea y la de ejecución en tiempo real.

El **capítulo 4** aborda la parte de aprendizaje automático, desde el preprocesado del conjunto de formas de onda hasta el entrenamiento del autoencoder y la generación de la rejilla que se embarca en el sistema.

Los **capítulos 5 y 6** describen el desarrollo del firmware. El primero se ocupa de la cadena de entrada, formada por la pantalla táctil y el ESP32-S3, y el segundo del motor de audio implementado sobre la Daisy Seed.

El **capítulo 7** trata la parte física del prototipo: la alimentación, el panel de control analógico y el montaje definitivo.

El **capítulo 8** reúne las pruebas realizadas y los resultados obtenidos, tanto las medidas eléctricas como las verificaciones numéricas de cada etapa de la cadena de procesado.

El **capítulo 9** recoge las conclusiones y plantea las líneas de trabajo futuro.

Capítulo 2

Marco teórico y estado del arte

Este capítulo reúne los conceptos necesarios para seguir el resto del documento. Se parte de los fundamentos de la síntesis de sonido, se detalla la técnica concreta sobre la que se construye el instrumento y se introducen los autoencoders y el espacio latente. El capítulo termina con una revisión de los trabajos previos que aplican redes neuronales a la generación de formas de onda, que es donde se sitúa la aportación de este trabajo.

2.1 Fundamentos de síntesis de sonido

Un sonido periódico queda descrito por tres magnitudes: la frecuencia fundamental, que determina la altura de la nota; la amplitud, que determina su intensidad; y el contenido armónico, que determina el timbre. Dos instrumentos que tocan la misma nota con la misma intensidad se distinguen precisamente por la distribución de energía entre los armónicos de esa fundamental. Estos armónicos dependen de características físicas del instrumento, como la forma de la caja de resonancia o la rigidez de las cuerdas. En la sintetización de sonido, en cambio, el timbre se genera mediante algoritmos que producen la señal de audio directamente, sin necesidad de un instrumento físico.

La síntesis de sonido agrupa las técnicas que permiten generar señales de audio por medios electrónicos. Las más habituales son las siguientes [1]:

- **Síntesis aditiva.** Construye el sonido sumando osciladores senoidales de distinta frecuencia y amplitud. Ofrece un control muy preciso del espectro, a costa de un coste computacional elevado cuando el número de frecuencias a sumar es alto.
- **Síntesis sustractiva.** Parte de una señal rica en armónicos y emplea filtros para eliminar las frecuencias no deseadas. Es la base de los sintetizadores analógicos clásicos y sigue vigente en los digitales.
- **Síntesis por modulación de frecuencia.** Modula la frecuencia de un oscilador con la salida de otro, generando espectros complejos con muy pocos operadores.
- **Síntesis por tabla de ondas.** Almacena en memoria uno o varios ciclos de forma de onda y los reproduce leyendo la tabla a la velocidad adecuada. Es la técnica sobre la que se construye este trabajo y se detalla en el apartado siguiente.

Con independencia de la técnica empleada, un sonido musical necesita evolucionar en el tiempo. Una nota que se mantuviese con amplitud constante desde que se pulsa la tecla hasta que se suelta resultaría artificial. Esa evolución se controla mediante una envolvente, y la más extendida es la ADSR, que divide la vida de la nota en cuatro tramos:

- **Ataque (Attack).** El tiempo que tarda la señal en alcanzar su nivel máximo.
- **Decaimiento (Decay).** El tiempo que tarda en descender el nivel desde el pico máximo hasta el nivel de sostenimiento.
- **Sostenimiento (Sustain).** El nivel que se mantiene mientras la tecla siga pulsada.
- **Liberación (Release).** El tiempo que tarda en devolver la señal a cero cuando se suelta la tecla.

La Figura 2.1 muestra la forma que adopta la envolvente y cómo se reparten esos cuatro tramos a lo largo de la duración de la nota.

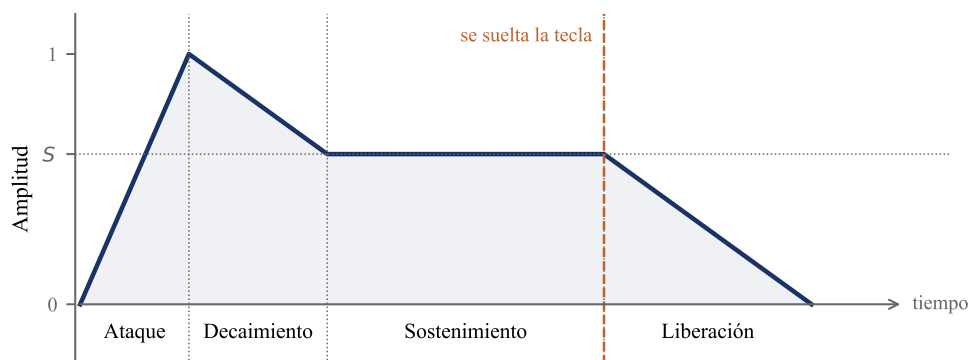


Figura 2.1: Envolvente ADSR. Los dos primeros tramos duran un valor fijo. El tercer tramo depende de cuánto tiempo se mantenga pulsada la tecla. La liberación empieza al soltarla.

El otro elemento habitual en la cadena de un sintetizador es el filtro. Un LPF atenúa las frecuencias por encima de una frecuencia de corte, un HPF hace lo contrario, atenúa las frecuencias por debajo de una frecuencia de corte; y un BPF deja pasar únicamente una banda de frecuencia intermedia. El parámetro de resonancia (Q) realza la respuesta en torno a la frecuencia de corte, lo que da al filtro buena parte de su carácter sonoro.

La Figura 2.2 compara la respuesta en magnitud de los tres tipos sobre una misma frecuencia de corte.

El orden clásico de la cadena en un sintetizador sustractivo es oscilador, filtro y amplificador controlado por la envolvente (ADSR), y es el que se ha mantenido en este trabajo.

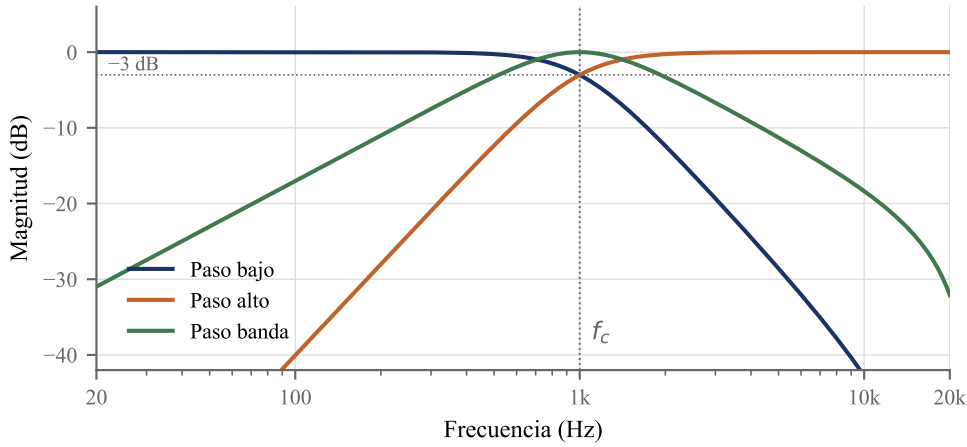


Figura 2.2: Respuesta en magnitud de un filtro paso bajo, uno paso alto y uno paso banda de segundo orden, todos con la frecuencia de corte en 1 kHz y factor de calidad $Q = 0,707$.

2.2 Síntesis por tabla de ondas

La síntesis wavetable se apoya en una observación sencilla. Si una señal es periódica, basta con almacenar un único ciclo y repetirlo. En lugar de evaluar una función trigonométrica para cada muestra, se guarda ese ciclo en una tabla de N posiciones y se recorre cíclicamente a la velocidad que corresponda a la nota deseada [2].

El mecanismo que gobierna esa lectura es el acumulador de fase. Se trata de una variable que avanza una cantidad fija en cada muestra y que vuelve al principio al llegar al final de la tabla. Siendo f_0 la frecuencia de la nota, f_s la frecuencia de muestreo y N el tamaño de la tabla, el incremento de fase por muestra es el de la Ecuación (2.1).

$$\Delta\varphi = \frac{f_0 \cdot N}{f_s} \quad (2.1)$$

El acumulador avanza según la Ecuación (2.2), donde el módulo implementa el retorno al inicio de la tabla al completarse cada ciclo.

$$\varphi[n+1] = (\varphi[n] + \Delta\varphi) \bmod N \quad (2.2)$$

Como $\Delta\varphi$ no es en general un número entero, el acumulador apunta casi siempre a una posición intermedia entre dos muestras de la tabla. La solución habitual es interpolar linealmente entre la muestra anterior y la siguiente, tal como recoge la Ecuación (2.3), donde $i = \lfloor \varphi \rfloor$ y $\alpha = \varphi - i$ es la parte fraccionaria.

$$s[n] = (1 - \alpha) \cdot T[i] + \alpha \cdot T[(i+1) \bmod N] \quad (2.3)$$

El coste de generar una muestra se reduce así a una suma, dos multiplicaciones y un acceso a memoria, lo que hace la técnica especialmente adecuada para microcontroladores. Bristow-Johnson señala además que, para tonos musicales cuasiperiódicos, este método resulta equivalente a la síntesis aditiva pero exigiendo mucho menos cálculo en tiempo real [2], que es justo el compromiso

que interesa en un sistema embebido. A cambio, el timbre queda fijado por el contenido de la tabla, de modo que cambiar de sonido implica cambiar de tabla.

La Figura 2.3 ilustra el proceso completo. El acumulador señala una posición fraccionaria dentro de la tabla y el valor de salida se obtiene pesando las dos muestras que lo rodean.

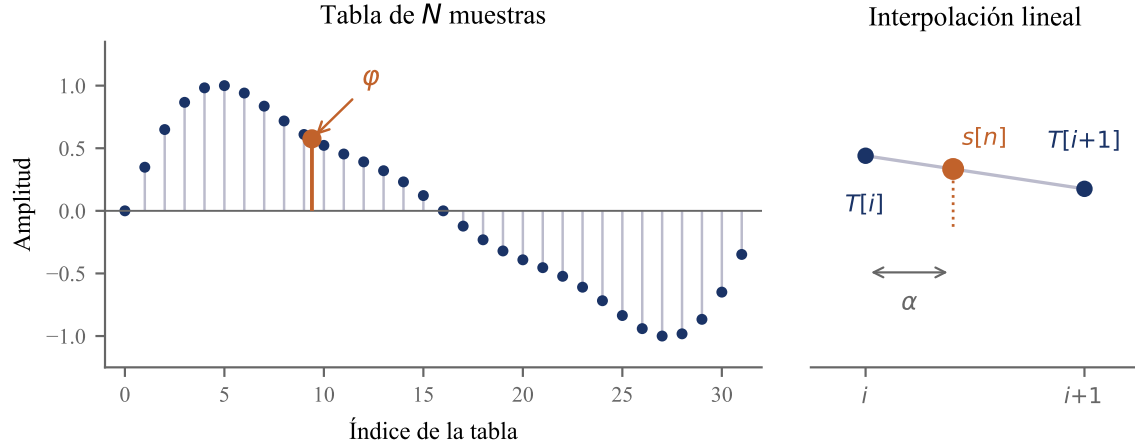


Figura 2.3: Lectura de una tabla de ondas. A la izquierda, la tabla almacenada y la posición actual del acumulador de fase φ . A la derecha, el detalle de la interpolación lineal entre las dos muestras vecinas.

2.2.1 Aliasing y limitación de banda

Una tabla de N muestras puede contener hasta $N/2$ armónicos. Al reproducirla a frecuencias altas, los armónicos superiores acaban rebasando la frecuencia de Nyquist y se reflejan hacia el interior del espectro como componentes de frecuencia no armónica, audibles como un timbre metálico y desafinado. Este fenómeno se conoce como *aliasing* y es el principal problema de la reproducción de tablas a frecuencias elevadas.

La solución habitual consiste en almacenar varias versiones de cada tabla, filtradas para distintos rangos de nota, y seleccionar en cada momento la que corresponda al tono que se está tocando [3]. Conviene señalar que este problema es independiente del tamaño de la tabla y del proceso de remuestreo empleado para construirla. Aumentar N no elimina el *aliasing*, solo cambia cuántos armónicos hay disponibles para reflejarse. Por las razones que se exponen en el capítulo 3, la limitación de banda queda fuera del alcance de este trabajo.

2.3 Autoencoders y espacio latente

Un autoencoder es una red neuronal entrenada para intentar reconstruir su propia entrada. Está formado por dos bloques. El codificador, que transforma la entrada en una representación de dimensión mucho menor; y el decodificador, que intenta reconstruir la entrada original a partir de esa representación reducida. La representación intermedia recibe el nombre de código o vector latente. Los autoencoders son capaces de organizar estos vectores latentes en base a las características de los datos de entrada, de modo que entradas similares acaban situadas cerca unas de otras en un espacio. Es lo denominado el espacio latente.

El interés de esta arquitectura no está en la reconstrucción en sí, que sería trivial si no se impusiera ninguna restricción, sino en el cuello de botella. Al obligar a que toda la información pase por un número reducido de valores, la red se ve forzada a quedarse con los rasgos que de verdad distinguen unas entradas de otras. Aplicado a un conjunto de formas de onda, el codificador acaba situando cerca las que comparten características espectrales, sin que se le haya indicado en ningún momento qué es un armónico o qué es un timbre.

2.3.1 El autoencoder variacional

Un autoencoder convencional organiza el espacio latente sin ninguna garantía sobre su estructura. Nada impide que queden zonas vacías en este espacio. Un punto tomado de una de esas zonas puede decodificarse en algo sin sentido. Para el uso que se pretende aquí, en el que el usuario recorre el plano con el dedo y pasa continuamente por posiciones que no corresponden a ninguna onda del conjunto de entrenamiento, esa falta de predictibilidad es un problema.

El autoencoder variacional o VAE [4] resuelve esa limitación cambiando la naturaleza del vector latente. En lugar de producir un vector de valores concretos, el codificador produce los parámetros de una distribución de probabilidad, es decir, un vector de medias μ y otro de varianzas σ^2 . Así pues, el vector latente que recibe el decodificador se obtiene muestreando esa distribución. El problema de este muestreo es que, al ser una operación aleatoria, no es derivable y bloquea la propagación del gradiente durante el entrenamiento. Para solucionarlo, Kingma y Welling proponen el truco de reparametrización (Ecuación (2.4)). El vector latente se expresa como una función determinista de la media y la desviación típica, utilizando una muestra de ruido externa. De este modo, la aleatoriedad queda aislada en un término independiente de la red y el gradiente puede propagarse sin impedimentos.

$$\mathbf{z} = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (2.4)$$

Para evitar que el espacio latente quede lleno de huecos vacíos, a la función de pérdida se le añade un segundo término: la divergencia de Kullback-Leibler. Esta métrica compara la distribución del codificador con una normal estándar y actúa como un imán, obligando a todas las nubes de probabilidad a agruparse y solaparse ordenadamente en torno al centro. Así, la pérdida total equilibra la fidelidad de la reconstrucción con este requisito de regularidad, tal como muestra la Ecuación (2.5).

$$\mathcal{L} = \underbrace{\text{MSE}(\mathbf{x}, \hat{\mathbf{x}})}_{\text{reconstrucción}} + \beta \cdot \underbrace{D_{KL}(q(\mathbf{z}|\mathbf{x}) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I}))}_{\text{regularización}} \quad (2.5)$$

Para un latente de d dimensiones, la divergencia admite la forma cerrada de la Ecuación (2.6), que es la que se implementa en la práctica.

$$D_{KL} = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2) \quad (2.6)$$

El factor β que aparece en la Ecuación (2.5) pondera la importancia relativa de ambos términos y da nombre a la variante conocida como β -VAE [5]. Higgins y sus coautores lo introducen como un hiperparámetro ajustable que equilibra la capacidad del canal latente frente a la precisión de

la reconstrucción, y con valores por encima de la unidad obtienen representaciones *factorizadas*, en las que cada dimensión del espacio recoge un único factor de variación de los datos y puede moverse sin alterar el resto. Sobre imágenes de caras, por ejemplo, una dimensión gobierna el giro de la cabeza y otra la anchura de la sonrisa.

Conviene no confundir esa propiedad con la continuidad, que es la que interesa aquí y que aporta el término de divergencia en cualquier autoencoder variacional. Lo que β regula es cuánto pesa la regularización frente a la fidelidad de la reconstrucción. Valores altos dan un espacio muy regular a costa de reconstrucciones pobres, porque el modelo prioriza acercar todas las distribuciones a la normal estándar antes que reproducir cada entrada con detalle; valores bajos hacen lo contrario. El compromiso adoptado en este trabajo se justifica en el capítulo 4.

Un fallo habitual en estos modelos es el conocido como colapso posterior. Ocurre cuando la red ignora por completo el espacio latente y el término de divergencia se anula. Este fallo fue observado por Bowman y sus coautores [6] al aplicar VAEs a texto. Esto pasa porque el decodificador es lo suficientemente potente como para reconstruir los datos por su cuenta, sin necesitar ayuda del espacio latente. Para solucionarlo, en este trabajo se ha aplicado su misma técnica: introducir el peso de la divergencia de forma progresiva. Así, durante las primeras épocas el modelo se centra solo en aprender a reconstruir, y el peso se va subiendo poco a poco hasta su valor definitivo, tal como se explica en el capítulo 4.

2.4 Estado del arte

La aplicación de redes neuronales a la síntesis de audio ha producido en los últimos años varias líneas de trabajo. Interesa aquí distinguir dos enfoques distintos según qué es lo que la red genera.

2.4.1 Modelos que generan la señal de audio

Un primer grupo de trabajos emplea la red para generar directamente la forma de onda de salida, muestra a muestra. El sistema DDSP [7] combina elementos clásicos de procesamiento de señal, como osciladores armónicos y filtros, con redes neuronales que controlan sus parámetros, de modo que el conocimiento sobre cómo se genera el sonido queda incorporado a la arquitectura en lugar de tener que aprenderse desde cero. RAVE [8] sigue una vía diferente. Se trata de un autoencoder variacional entrenado sobre audio en bruto, con un procedimiento de entrenamiento en dos fases, primero de aprendizaje de la representación y después de ajuste adversario. Sus autores, Caillon y Esling, añaden un análisis posterior del espacio latente que permite decidir el compromiso entre fidelidad de reconstrucción y compacidad de la representación. Es un problema que aparece de nuevo en este trabajo al fijar la dimensión del latente. Su interés para este trabajo es que demuestra que la síntesis neuronal puede alcanzar velocidades de tiempo real, aunque sobre hardware sensiblemente más potente que un microcontrolador.

2.4.2 Modelos que generan tablas de onda

El segundo grupo, más cercano a lo que se plantea aquí, no genera audio sino tablas. La red produce una forma de onda de un ciclo que después se reproduce mediante un oscilador convencional, con lo que la carga computacional no depende de la frecuencia de muestreo.

El trabajo de Hantrakul y Yang [9] es el precedente más directo. Reutilizan el modelo NSynth WaveNet ya entrenado como autoencoder de tablas para generar formas de onda de 512 muestras e interpolar entre ellas en el espacio latente. El resultado se distribuye como *plugin* VST/AU para DAW.

Dos observaciones tuyas resultan especialmente importantes aquí. La primera es de coste. Los propios autores miden unos 4 segundos para codificar y unos 7 para decodificar una única forma de onda sobre una máquina con GPU, lo que descarta por completo ejecutar la red durante la reproducción. Su solución fue generar por adelantado 300 tablas interpolando entre combinaciones de seno, sierra y triangular, y reproducirlas después con un oscilador convencional. Es exactamente la misma estrategia que se adopta en este trabajo. Conviene subrayar que la limitación aparece ya sobre una GPU de sobremesa. Al trabajar sobre microcontroladores de bajo coste, la limitación es aún más severa y obliga a generar las tablas de forma anticipada.

La segunda observación tiene que ver con la estructura del espacio latente. Hantrakul y Yang comprueban que interpolar a lo largo del segmento que une dos vectores del espacio produce mezclas utilizables de las dos ondas de partida, pero que apartarse de ese segmento genera ruido. Dicho de otro modo, en su modelo no todo el espacio latente es válido, solo lo son las trayectorias entre puntos conocidos. Para un instrumento en el que el usuario recorre libremente un plano con el dedo esa limitación es un problema. Esta es una de las razones que justifican emplear en este proyecto un autoencoder variacional en lugar de uno convencional.

Kreković [10] plantea un oscilador basado en un autoencoder y estudia el efecto del conjunto de entrenamiento y de los hiperparámetros sobre la variedad de las formas de onda generadas. Su conclusión fue que emplear las variables latentes directamente como parámetros de entrada produce una transición suave entre las ondas generadas cuando los cambios de esos parámetros son pequeños.

Más recientemente, Wavespace [11] propone un generador de tablas basado en un autoencoder variacional cuyo espacio latente se divide en subespacios asociados a distintos estilos de forma de onda. El generador se condiciona además con descriptores tímbricos y morfológicos, de modo que el usuario puede manipular cada subespacio de forma independiente en lugar de moverse por un plano indiferenciado. Los autores lo prototipan también como *plugin* de oscilador para DAW, y señalan que la restricción de velocidad de cálculo sigue siendo el principal obstáculo cuando se pretende decodificar una forma de onda para cada bloque de audio.

2.4.3 Posicionamiento de este trabajo

De la revisión anterior se desprende que la generación de tablas de onda mediante autoencoders y su interpolación en el espacio latente es un problema ya abordado, y resuelto con solvencia, en varios trabajos previos. La aportación de este trabajo no está, por tanto, en el modelo.

Lo que ninguno de esos trabajos aborda es la materialización del concepto como instrumento físico independiente. Todos ellos se presentan como *plugins*, aplicaciones de escritorio o prototipos de investigación que se ejecutan sobre un ordenador. La aportación diferencial que se persigue aquí es la integración embebida completa. Eso supone repartir el sistema entre microcontroladores de bajo coste, resolver la comunicación entre ellos, mantener la estabilidad del audio en tiempo real y construir un dispositivo autónomo con control táctil y entrada MIDI que no necesite un ordenador para funcionar. El reto no es de aprendizaje automático, sino de ingeniería de sistemas embebidos.

Capítulo 3

Diseño del sistema

Este capítulo explica cómo se reparte el sistema entre las placas que lo componen, qué alternativas se valoraron antes de llegar a ese reparto y qué parámetros quedaron fijados antes de empezar a ejecutar. La implementación concreta de cada bloque se trata en los capítulos 5, 6 y 7, y las medidas que verifican el diseño, en el capítulo 8. Aquí interesa el porqué de la estructura.

3.1 Requisitos y condicionantes de partida

El instrumento tiene que atender dos tareas que operan en escalas de tiempo muy distintas. Una es producir una forma de onda nueva cuando el usuario mueve el dedo por la pantalla y la otra es entregar 48.000 muestras por segundo de forma ininterrumpida, sin que una espera de comunicación o un cálculo largo dejen el búfer de audio sin llenar. La primera admite unos milisegundos de retraso; la segunda no admite ninguno o se tendrán *glitches* audibles.

De ahí sale la restricción que gobierna todo el diseño, ya enunciada en el capítulo 1. La inferencia de una red neuronal no puede ejecutarse a frecuencia de audio sobre un microcontrolador, pero tampoco necesita hacerlo. Un dedo sobre una pantalla no genera información nueva a 48 kHz. Basta con que el sistema táctil se lea cada 50 ms, es decir, a 20 Hz. Además, se debe programar para que solo se envíe una coordenada cuando el punto se ha desplazado lo suficiente y evitar tráfico innecesario. Separar la generación de la tabla, que ocurre a frecuencia de control; de su reproducción, que ocurre a frecuencia de audio; es lo que hace viable el conjunto sobre hardware de bajo coste.

Sobre esa base se fijaron los requisitos que el diseño debía cumplir:

- **Continuidad del control.** El usuario recorre el plano latente de forma continua, así que el cambio de una tabla a la siguiente no puede producir discontinuidades audibles.
- **Ejecución musical convencional.** La nota y la dinámica entran por MIDI estándar, y la envolvente y el filtro se controlan desde mandos físicos, no desde la pantalla.
- **Autonomía.** El instrumento debe funcionar sin depender de un ordenador. Debe conformar un módulo de sonido autónomo, con teclado intercambiable y sin necesidad de software adicional.

- **Integridad de la información.** La tabla que sale de una placa tiene que llegar íntegra a la siguiente, y una trama dañada no puede traducirse en ruido a la salida.
- **Hardware comercial de bajo coste.** No se diseña ni se fabrica ningún circuito integrado específico, y el reparto de tareas tiene que caber en placas de desarrollo disponibles en el mercado.

A estos requisitos se suman los condicionantes normativos y de especificación que impone el material elegido. La entrada de notas sigue la especificación MIDI 1.0, y los rangos de tensión, la tolerancia de los pines y el techo de la frecuencia de corte salen de las hojas de características de las placas y de la documentación de la biblioteca de audio. Cada uno se cita en el capítulo en que se aplica.

3.2 Alternativas consideradas

Las tres decisiones que dan forma al sistema se tomaron antes de empezar a construirlo, contrastando cada una con la opción que se descartaba.

3.2.1 Ejecutar la red frente a precalcular una rejilla

La primera alternativa era ejecutar el decodificador dentro del propio instrumento, cuantizado a 8 bits y desplegado sobre el ESP32-S3 de modo que cada coordenada táctil produjera una forma de onda recién generada. La segunda era muestrear el plano latente por adelantado en el ordenador, obtener una rejilla de formas de onda ya decodificadas y embarcar esa rejilla, dejando al instrumento la consulta y la mezcla.

Se adoptó la segunda como camino principal. Aunque la red solo tenga que trabajar a frecuencia de control, ejecutar el decodificador en cada toque introduce un tiempo de cálculo variable que complica el modelo. Sobre un procesador que además tiene que atender la comunicación con las otras dos placas, esto se traduce en posibles microinterrupciones en el audio. La rejilla precalculada convierte ese cálculo en un acceso a memoria y una interpolación, de coste conocido y constante. El trabajo más cercano hace lo mismo por el mismo motivo, y allí la limitación aparece incluso sobre una máquina con GPU dedicada [9].

El precio de esa decisión es que la resolución del espacio queda congelada en el momento de generar la rejilla, y que esa rejilla ocupa memoria. El decodificador embarcado se mantiene como posible ampliación y se retomará en el capítulo 9.

3.2.2 Dónde reside la rejilla

Fijada la rejilla, quedaba decidir en qué placa vive y, en consecuencia, qué viaja por el bus. La primera opción era guardarla en la Daisy Seed, que dispone de memoria externa de sobra, y hacer que la CYD le enviara únicamente la coordenada táctil, seis bytes por toque. La segunda opción era guardarla en el ESP32-S3, hacer allí la interpolación y enviar a la Daisy la forma de onda completa, algo más de dos kilobytes por toque.

Se eligió la segunda. La primera es más sencilla y mueve mucha menos información pero carga al procesador de audio con una tarea que no le corresponde y, sobre todo, cierra la puerta a la ampliación del apartado anterior. Manteniendo la rejilla y la interpolación en el ESP32-S3, sustituirla más adelante por el decodificador ejecutándose en tiempo real no obliga a tocar el código de la Daisy Seed, ya que seguiría recibiendo exactamente lo mismo. El reparto resultante deja a cada placa una responsabilidad única, esto es lo que permitió después validar cada etapa por separado.

3.2.3 Dónde se resuelve la transición entre tablas

Sustituir la tabla en reproducción mientras suena una nota produce un salto en la señal, audible como un chasquido. La transición puede resolverse de dos formas. Una es esperar a que el acumulador de fase complete el ciclo y cambiar la tabla justo en ese punto. La otra es mezclar la tabla antigua y la nueva durante unos milisegundos, con pesos complementarios. En ambos casos se necesitaría doble búfer: uno para la forma de onda que se está reproduciendo y otra para la nueva.

Se optó por la mezcla, porque funciona con independencia de dónde esté la fase en el momento del cambio y porque encaja con el uso previsto en el que el dedo se desplaza de forma continua y las tablas consecutivas se parecen mucho entre sí. Este cálculo se hará en la Daisy ya que es la única placa que conoce la posición instantánea del acumulador de fase, debido a que es quien recorre la tabla muestra a muestra. El ESP32-S3 no dispone de esa información y llegaría siempre tarde o pronto al punto de corte, así que la transición debe de hacerla el procesador de audio y no en el de control.

3.3 Arquitectura resultante

El sistema queda repartido entre tres microcontroladores con funciones separadas, tal como muestra la Figura 3.1.

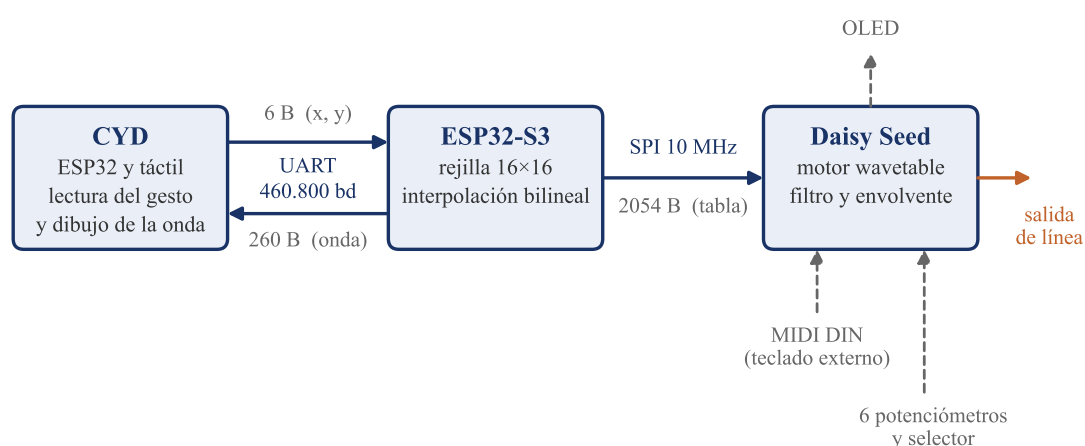


Figura 3.1: Arquitectura del sistema. Las flechas continuas son los enlaces digitales entre placas, rotuladas con el medio y el tamaño de la trama; las discontinuas, los periféricos del procesador de audio.

La CYD es la interfaz de usuario. Se trata de una placa que integra un ESP32 y una pantalla táctil resistiva de 2,8 pulgadas. Lee la posición del dedo, la normaliza a un rango de 16 bits por eje y la envía por UART. En sentido contrario recibe la forma de onda que está sonando y la dibuja en pantalla. Así el usuario puede ver el timbre que está seleccionando.

El ESP32-S3 es el procesador de control. Aloja la rejilla de formas de onda en su memoria de programa, calcula la interpolación entre las cuatro celdas vecinas a la coordenada recibida y entrega el resultado al procesador de audio por SPI. Es también quien devuelve a la pantalla la onda para que pueda representarla. Se eligió la variante con 16 MB de memoria de programa y 8 MB de PSRAM, con margen sobrado para la rejilla y para una eventual ampliación.

La Daisy Seed es el procesador de audio. Reproduce la tabla que recibe mediante un acumulador de fase, resuelve la transición entre tablas, aplica el filtro y la envolvente ADSR y entrega la señal por su convertidor digital-analógico. Recibe además la nota y la velocidad de esta por MIDI y lee los seis potenciómetros y el selector del panel de control. Además, refleja en una pantalla OLED el mando que se está manipulando para que el usuario sea conocedor del valor que está aplicando. No conoce la rejilla ni interpola, solo reproduce lo que le llega.

El teclado que proporciona las notas queda fuera del conjunto. Se conecta exclusivamente por MIDI DIN, cuyo aislamiento por optoacoplador evita que las dos masas se unan. Esta separación es deliberada, y su consecuencia es que el entregable de este trabajo es un módulo de sonido con teclado intercambiable y no un instrumento con teclado propio. Esta es la práctica habitual en la instrumentación electrónica modular.

3.4 Fase de preparación y fase de ejecución

El sistema opera en dos fases separadas en el tiempo y en el lugar, recogidas en la Figura 3.2. La primera ocurre una sola vez en el ordenador, y es la única en la que interviene la red neuronal. El conjunto de formas de onda se estandariza, se entrena el autoencoder variacional, se muestrea el plano latente en una rejilla regular decodificando cada nodo. El resultado se exporta como una cabecera en C que se compila junto al firmware del ESP32-S3. La segunda ocurre en el instrumento cuando se reciben coordenadas con cada toque. Esta no ejecuta ninguna red, solo consulta la rejilla, interpola y envía la forma de onda resultante al procesador de audio.

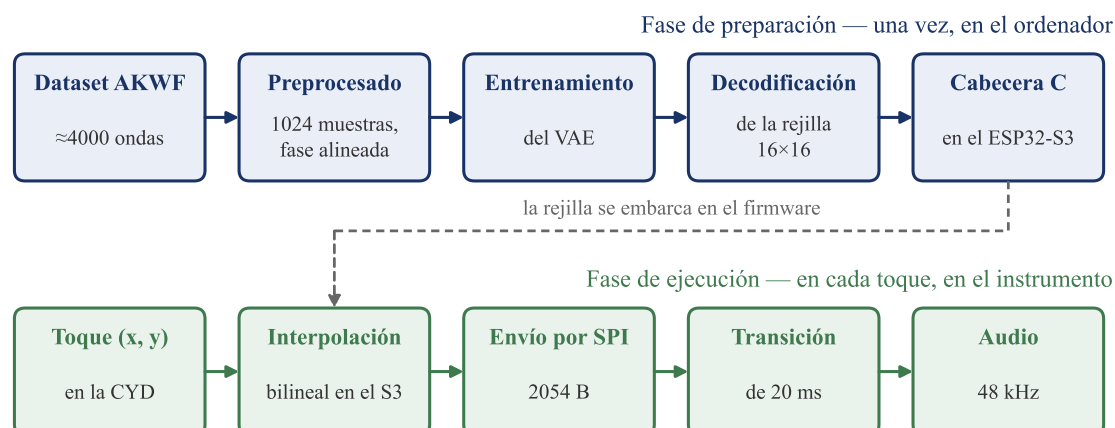


Figura 3.2: Las dos fases de operación. La red neuronal solo interviene en la fase de preparación; durante la ejecución el instrumento se limita a consultar la rejilla, mezclar y reproducir.

Conviene aclarar qué contiene la rejilla, porque es el punto que sostiene toda la estrategia. Los 256 nodos no son una selección de las ondas del conjunto de datos. Una vez entrenado el modelo, el codificador reparte las ondas originales como puntos dispersos por un plano continuo. El decodificador sabe generar una onda para cualquier posición de ese plano, incluidas las que no corresponden a ninguna onda real. Generar la rejilla es muestrear esa superficie continua en cruces regulares, de modo que la mayoría de los nodos son formas de onda nuevas. Lo aprendido a partir del conjunto de datos reside en los pesos del decodificador, y la rejilla es solo una captura de esa superficie a una resolución elegida. Lo que fija su tamaño, por tanto, es la finura con la que se navega el espacio y la memoria que se consume, no la variedad de timbres disponibles.

La segunda cuestión es por qué basta con una interpolación lineal durante la ejecución, siendo la operación más simple posible. Hay dos motivos. El primero es que todas las formas de onda comparten una referencia de fase común. Esta fue fijada en el preprocesado con lo que no hay desalineamientos gruesos que puedan cancelarse al promediar. El segundo es que dos celdas contiguas de la rejilla no son dos ondas cualesquiera, sino dos puntos muy próximos de una misma función continua y suave, el decodificador, de modo que sus salidas se parecen mucho. Interpoliar linealmente entre dos formas de onda casi idénticas deja un error pequeño, que además disminuye al aumentar la densidad de la rejilla. El trabajo complejo de organizar el espacio latente ya lo hizo el decodificador al generar los nodos. En tiempo de ejecución, la interpolación lineal solo se encarga de transicionar de forma fluida entre puntos que ya están ordenados.

3.5 Parámetros de diseño

La Tabla 3.1 recoge los valores que se fijaron al principio y que no se modificaron después, porque condicionan a la vez el entrenamiento, el formato de las tramas y el motor de audio.

El tamaño de 1024 muestras merece justificación porque las ondas del conjunto de datos tienen 600. Una onda de 600 muestras contiene como mucho 300 armónicos, que es el límite de Nyquist del propio ciclo. Remuestrear a 1024, la menor potencia de dos por encima de las 600 muestras

Tabla 3.1: Parámetros de diseño fijados antes de la implementación.

Parámetro	Valor
Muestras por forma de onda	1024
Formato de muestra	entero de 16 bits con signo (Q15)
Dimensión del espacio latente	2
Tamaño de la rejilla	16×16 (256 formas de onda)
Memoria ocupada por la rejilla	512 kB
Frecuencia de muestreo de audio	48 kHz
Tamaño del bloque de audio	48 muestras
Duración de la transición entre tablas	20 ms
Periodo de lectura del táctil	50 ms

originales, recoge entonces todo el contenido armónico disponible sin desperdicio. Que sea potencia de dos aporta además una ventaja práctica en la reproducción, ya que el retorno al inicio de la tabla se resuelve con una máscara de bits en lugar de con una operación de módulo.

Queda por justificar una ausencia. Como se indicó en el capítulo 2, la limitación de banda de las tablas no forma parte de este trabajo. El *aliasing* no lo introduce el remuestreo del conjunto de datos, sino la reproducción de notas agudas en las cuales los armónicos altos de la tabla superan la frecuencia de Nyquist. Corregirlo exige almacenar varias versiones filtradas de cada forma de onda y elegir la adecuada según la nota lo que multiplica por ese número de versiones el volumen de datos. Esto queda como ampliación futura, y el instrumento se entrega con las tablas originales sin filtrar.

3.6 Enlaces de comunicación

El sistema tiene tres enlaces, resumidos en la Tabla 3.2. Los dos primeros comparten el mismo par de hilos en sentidos opuestos, aprovechando que el UART es bidireccional.

Tabla 3.2: Enlaces de comunicación entre placas.

Enlace	Medio	Contenido	Trama	Tiempo
CYD → ESP32-S3	UART 460.800 bd	coordenada táctil	6 B	130 μ s
ESP32-S3 → CYD	UART 460.800 bd	onda para dibujar	260 B	5,6 ms
ESP32-S3 → Daisy	SPI 10 MHz	forma de onda	2054 B	1,6 ms

La elección del medio para el enlace principal viene dada por el volumen. Enviar 1024 muestras de 16 bits por UART, incluso a 921.600 baudios, llevaría unos 22 ms. Este es tiempo suficiente para que el gesto del dedo se note retrasado. El mismo envío por SPI a 10 MHz ocupa menos de 2 ms lo que lo convierte en la opción viable para la forma de onda. Para la coordenada táctil en cambio el UART sobra, porque son seis bytes y el tiempo de transmisión es despreciable frente al periodo de lectura del táctil.

Las tres tramas comparten criterio de construcción. Todas empiezan por una cabecera de sincronización conocida que permite al receptor reengancharse si pierde el hilo. Las de mayor tamaño

llevan además un número de secuencia, útil para correlacionar lo enviado con lo recibido durante la validación. La comprobación de integridad se dimensiona según lo que hay en juego. Una suma de comprobación por XOR de los bytes de datos basta en los enlaces cortos, mientras que la forma de onda viaja protegida por un CRC de 16 bits. El criterio ante un fallo es el mismo en los tres casos, descartar la trama y conservar el último dato válido. En el enlace de audio eso significa que el instrumento sigue sonando con la tabla anterior en lugar de reproducir muestras corrompidas. Esto es justamente el comportamiento que exigía el requisito de integridad del apartado 3.1.

El canal de vuelta hacia la pantalla merece una nota. Transporta la onda reducida a 256 muestras de 8 bits, no la tabla completa, porque su destino es una representación gráfica de 320 píxeles de ancho. Enviar los 2048 bytes originales a esa velocidad ocuparía unos 45 ms por onda, imposible de seguir con el dedo en movimiento, frente a los algo más de cinco milisegundos que cuesta la trama reducida.

Capítulo 4

Modelo neuronal

Este capítulo recorre la fase de preparación descrita en el apartado 3.4, la única del sistema en la que interviene una red neuronal. Se parte de un conjunto público de formas de onda de un ciclo, se homogeneiza, se entrena con él un autoencoder variacional con espacio bidimensional y se muestrea el plano aprendido en una rejilla que se exporta como cabecera en C. Esto se hizo en el ordenador y una sola vez. De esta manera, el instrumento recibe únicamente el resultado del muestreo de la red.

4.1 El conjunto de datos de partida

El material de entrenamiento procede de AKWF [12], una colección pública de formas de onda de un ciclo pensada originalmente para cargar en sintetizadores wavetable. Contiene 4358 ondas repartidas en 65 carpetas, todas con el mismo formato: monofónicas y de 600 muestras por ciclo.

Esa homogeneidad de formato es lo que hace la colección utilizable sin apenas trabajo de limpieza. Hay carpetas de formas geométricas clásicas, de emulaciones de instrumentos acústicos, de sonidos de videoconsola y de material ruidoso sin tono definido. Un autoencoder solo puede organizar un plano de timbres si el material de partida tiene timbres distintos que organizar, así que tener esta variedad de sonidos hace que este dataset sea utilizable.

Conviene aclarar desde el principio qué son esas 65 carpetas. Son etiquetas de organización que puso el autor de la colección, no una clasificación tímbrica rigurosa. Se emplean más adelante para colorear las figuras del espacio latente, porque ayudan a leerlas, pero en ningún momento entran en el entrenamiento. El modelo no ve etiquetas de ningún tipo, solo las ondas puras.

4.2 Preprocesado

Antes de entrenar hay que dejar las 4358 ondas en un formato común, tarea de la que se ocupa el programa de preprocesado `2_build_dataset.py`. Aquí se aplican cuatro operaciones a cada onda.

Remuestreo a 1024 muestras. Se emplea `scipy.signal.resample`, que interpola en el dominio de la frecuencia mediante la FFT en lugar de hacerlo en el tiempo. Una interpolación espectral

respetar la periodicidad de la señal y devuelve un ciclo que sigue cerrando sobre sí mismo, mientras que un método temporal introduciría una discontinuidad en el punto de empalme. El porqué de 1024 y no otro tamaño ya se justificó en el apartado 3.5.

Eliminación de la componente continua. Se resta la media de la onda. Una tabla con desplazamiento vertical produce un salto en la salida del conversor al cambiar de timbre y, sumada a otra tabla durante una transición, desplaza el resultado sin aportar nada audible.

Alineación de fase. Es el paso más importante. Las ondas de AKWF vienen guardadas de tal manera que empiezan aproximadamente en el cruce por cero ascendente del ciclo, pero esa referencia no es fiable. Una onda con mucho contenido armónico cruza el cero muchas veces, así que el punto de arranque elegido es en buena medida arbitrario y varía de una onda a otra. Dos ondas con el mismo espectro y distinto punto de referencia son, para la red, dos entradas diferentes. Al promediarlas se producirían cancelaciones entre armónicos en lugar de un timbre intermedio.

La solución adoptada consiste en rotar cada ciclo hasta dejar su armónico fundamental en fase cero. La rotación se hace en el dominio de la frecuencia multiplicando el armónico k por $e^{-jk\varphi}$, siendo φ la fase del fundamental. Eso equivale a un desplazamiento circular de la onda que además admite valores fraccionarios de muestra. Como se aplica a todos los armónicos de forma coherente, el ciclo sigue cerrando y no aparece ningún chasquido.

Tras alinear, las ondas no arrancan todas en el mismo valor de amplitud. No es un fallo del procedimiento. Lo que queda alineado es el fundamental, mientras que la muestra cero de cada onda es la suma de ese fundamental y de todos los armónicos superiores, cuyo reparto es distinto en cada caso. Dicho de otro modo, lo que sobrevive a la alineación es precisamente lo que distingue un timbre de otro.

La propiedad que de verdad se persigue es la invarianza. La Figura 4.1 la comprueba tomando una misma onda y arrancándola en cuatro puntos distintos del ciclo. Antes de alinear son cuatro curvas claramente diferentes; después, las cuatro colapsan en una sola, con una diferencia máxima entre ellas del orden de 10^{-16} , que es cero a efectos del ordenador. Con esa garantía, el punto de arranque de la onda original deja de existir como variable, y la interpolación lineal entre tablas realizada en los embebidos, justificada en el apartado 3.4, opera sobre ondas que comparten referencia.

Normalización de amplitud. Cada onda se divide por su valor absoluto máximo, de modo que todas ocupan el rango $[-1, 1]$. Así el modelo no gasta capacidad en representar diferencias de nivel, que en un instrumento las fija la dinámica de la nota y no la tabla.

Queda un caso que el procedimiento no puede tratar igual. Las ondas de ruido o percusivas no tienen un fundamental identificable. Por ello, rotar el ciclo tomando como referencia la fase de un armónico despreciable introduciría un giro aleatorio. El script detecta esa situación comparando la magnitud del fundamental con la del armónico más fuerte. Si la relación cae por debajo de 10^{-3} , es decir, no tiene una fundamental claramente diferenciada, deja la onda sin rotar en lugar de descartarla. Afecta a 100 ondas, un 2,3 % del total. Corresponde sobre todo a las ondas con armónico fundamental débil o ausente (la carpeta overtone y varias de las numeradas), más algunas de material estéreo. Mantenerlo en el conjunto aporta variedad tímbrica en los bordes del espacio sin comprometer la alineación del resto.

El resultado es una matriz de 4358×1024 valores en coma flotante, que es lo que consume el entrenamiento.

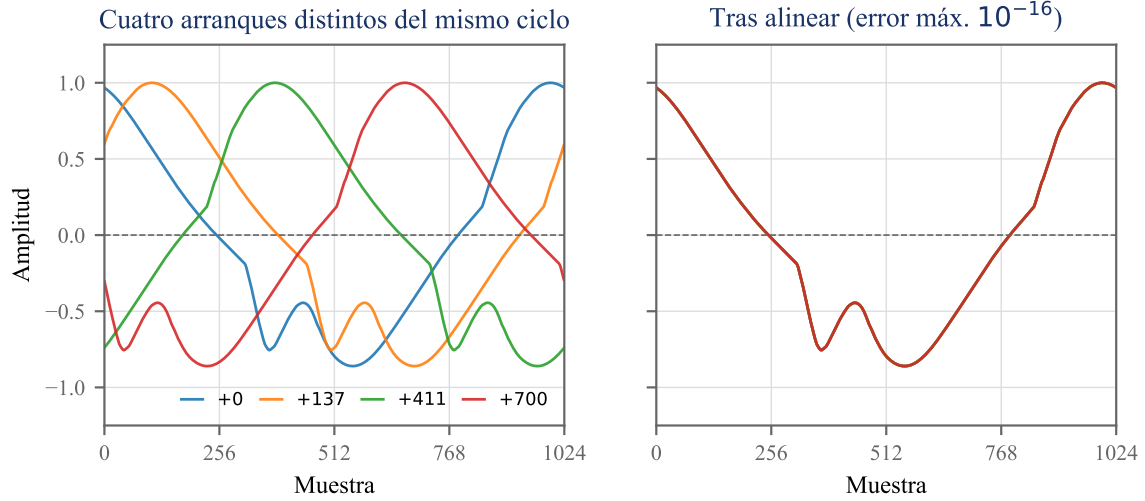


Figura 4.1: Invarianza de la alineación al punto de arranque. La misma onda desplazada 0, 137, 411 y 700 muestras produce cuatro curvas distintas; tras llevar el fundamental a fase cero, las cuatro quedan superpuestas.

4.3 Arquitectura del modelo

El modelo es un autoencoder variacional de capas totalmente conectadas. El codificador reduce las 1024 muestras a 512 y después a 128. De ahí salen dos vectores de dos componentes, la media y el logaritmo de la varianza de la distribución latente. El decodificador es el espejo exacto del codificador, de 2 a 128, a 512 y a 1024. Las capas intermedias usan la función de activación lineal rectificada y la salida usa tangente hiperbólica, que acota el resultado en $[-1, 1]$ y recoge así la normalización de amplitud del apartado anterior. En total suma 1.182.724 parámetros.

Cabe preguntarse por qué capas totalmente conectadas y no convolucionales, que son la elección habitual cuando la entrada es una señal. La razón está en el preprocesado. Una red convolucional aporta invarianza a traslación, útil cuando el mismo patrón puede aparecer en cualquier posición de la entrada, pero la alineación de fase ya eliminó esa variabilidad y todas las ondas llegan a la red con la misma referencia y la misma longitud. La invarianza que aportaría la convolución ya está resuelta antes de entrar, así que solo añadiría complejidad.

La Tabla 4.1 recoge la configuración completa del modelo y del entrenamiento.

Tabla 4.1: Configuración del autoencoder variacional y del entrenamiento.

Parámetro	Valor
Dimensión de entrada y salida	1024
Capas ocultas del codificador	512, 128
Dimensión del espacio latente	2
Activación intermedia	lineal rectificada
Activación de salida	tangente hiperbólica
Parámetros entrenables	1.182.724
Optimizador	Adam, tasa de aprendizaje 10^{-3}
Tamaño de lote	64
Épocas	300
División entrenamiento/validación	3923 / 435 (90/10)
Peso de la divergencia, β	0,001
Rampa de β	lineal, 50 épocas
Semilla	42

4.4 Función de pérdida y elección de β

La función que se minimiza es la que se planteó en el capítulo 2, el MSE entre la onda de entrada y su reconstrucción más la divergencia de Kullback-Leibler ponderada por un factor β . El primer término corresponde a la fidelidad de reconstrucción y el segundo hace que las distribuciones que produce el codificador se parezcan a una normal estándar, lo que mantiene el plano latente ocupado de forma continua.

El valor adoptado es $\beta = 0,001$, y conviene precisar en qué dirección va esa elección. Higgins y sus coautores introducen el factor proponiendo valores *mayores* que la unidad, porque su objetivo es obtener representaciones desenredadas, en las que cada dimensión del latente controle un factor de variación independiente e interpretable [5]. Aquí se ha adoptado el compromiso contrario, con un valor tres órdenes de magnitud por debajo de la unidad.

Un latente de dos dimensiones destinado a un dedo sobre una pantalla no necesita ejes que controlen de forma independiente un parámetro concreto del sonido. El usuario no manipula parámetros con nombre, sino que explora una superficie y se queda donde le gusta lo que oye. Lo que sí necesita es reconstruir con la mayor fidelidad posible y no dejar huecos entre zonas ocupadas. Penalizar con fuerza la divergencia habría acercado todas las distribuciones a la normal estándar. De esta manera, todas las reconstrucciones se habrían parecido a la onda media, que es el resultado que arruina un instrumento pensado para recorrer timbres. La regularización sigue haciendo falta, y por eso el término está presente pero con el peso justo para ordenar el plano sin aplanarlo.

Al entrenamiento se le añadió además una precaución que no estaba en el plan inicial. En lugar de aplicar β desde la primera época, se sube de cero a su valor definitivo mediante una rampa lineal a lo largo de las 50 primeras. Es la técnica que emplean Bowman y sus coautores para evitar el colapso posterior [6], ya descrita en el capítulo 2. Durante esas primeras épocas el modelo se concentra en aprender a reconstruir, y solo cuando ya tiene algo que perder entra la presión de la regularización.

4.5 Entrenamiento

El entrenamiento se ejecutó en local sobre una tarjeta gráfica NVIDIA RTX 3060. El modelo es pequeño para el hardware disponible, así que las 300 épocas no supusieron ninguna limitación y no hubo necesidad de recurrir a servicios de cómputo remoto. El conjunto se dividió en 3923 ondas para entrenar y 435 para validar, con semilla fija para que la partición sea reproducible.

La Figura 4.2 recoge la evolución de las dos magnitudes que interesan.

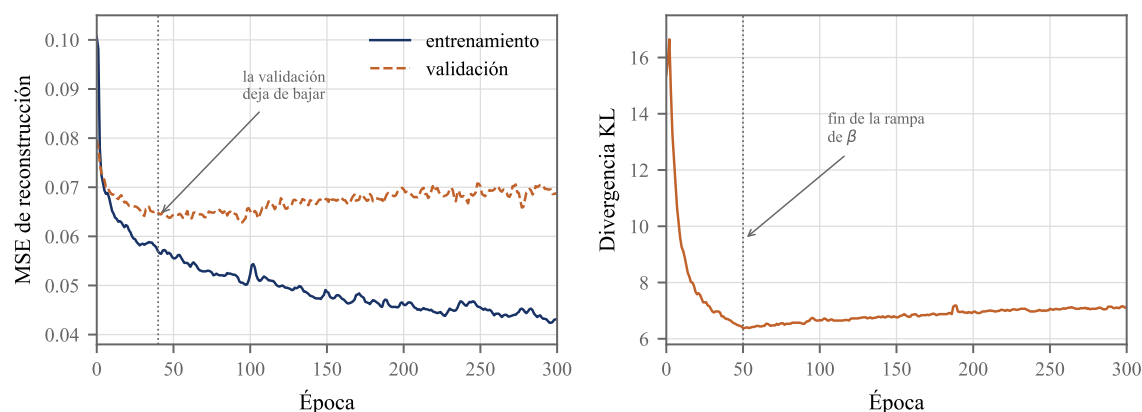


Figura 4.2: Evolución del error de reconstrucción y de la divergencia de Kullback-Leibler a lo largo de las 300 épocas. La línea de puntos del panel izquierdo marca el momento en que la validación deja de mejorar; la del derecho, el final de la rampa de β .

Los valores al terminar son un error de reconstrucción de 0,043 en entrenamiento y de 0,068 en validación, con la divergencia estabilizada en torno a 7. Esa divergencia es el primer indicio de que el modelo funciona como debe. Un valor claramente distinto de cero significa que el latente transporta información. Si el modelo hubiese optado por ignorarlo, la divergencia habría caído a cero y el decodificador produciría la misma onda para cualquier coordenada.

El panel izquierdo muestra también un comportamiento que conviene nombrar sin rodeos. El error de entrenamiento sigue bajando hasta el final, pero el de validación se aplanan en torno a 0,065 hacia la época 40 y a partir de ahí incluso repunta ligeramente. Es la forma habitual del sobreajuste, y significa que las más de 250 épocas restantes no aportaron capacidad de generalización alguna. La distancia entre ambas curvas es moderada y el modelo sigue siendo utilizable, pero el mismo resultado se habría conseguido en una fracción del tiempo deteniendo el entrenamiento hacia la época 80.

4.6 El espacio latente resultante

Terminado el entrenamiento, cada una de las 4358 ondas tiene asignada una coordenada en el plano. La Figura 4.3 las representa todas, coloreando las doce carpetas más numerosas y dejando el resto en gris.



Figura 4.3: Espacio latente aprendido. Cada punto es la media de la distribución que el codificador asigna a una onda. El rectángulo a trazos delimita los percentiles 2 y 98 de cada eje.

La nube ocupa aproximadamente de $-4,2$ a $7,0$ en el primer eje y de $-5,3$ a $6,9$ en el segundo, con la densidad concentrada en torno al origen y unos pocos puntos dispersos hacia los bordes. Es la forma que produce un autoencoder variacional típico. El término de regularización empuja la masa hacia el centro, y las ondas más atípicas son las únicas que compensa alejar pagando el coste que eso supone en la pérdida. Un latente colapsado tendría un aspecto muy distinto. Tendría todos los puntos apelotonados en una región mínima y una divergencia próxima a cero.

Hay un rasgo de la figura que a primera vista parece un mal resultado y no lo es. Las carpetas se solapan mucho entre sí, y ninguna forma un grupo bien separado del resto. Ahora bien, encajar 65 categorías en un plano de dos dimensiones de forma separable no es posible, y tampoco es lo que se pide al modelo. El codificador organiza las ondas por su forma, que es lo único que ve. Así pues, dos carpetas distintas con contenido espectral parecido acaban ocupando la misma zona. Para el uso previsto eso es justo lo deseable, porque lo que tiene que garantizar el plano es que puntos próximos suenen parecido, no que la carpeta de origen sea recuperable.

4.7 Fidelidad de la reconstrucción

Las cifras de pérdida dicen cuánto se equivoca el modelo de media, pero no en qué se equivoca. La Figura 4.4 compara cuatro ondas del conjunto con la salida del decodificador, ordenadas de menor a mayor error.

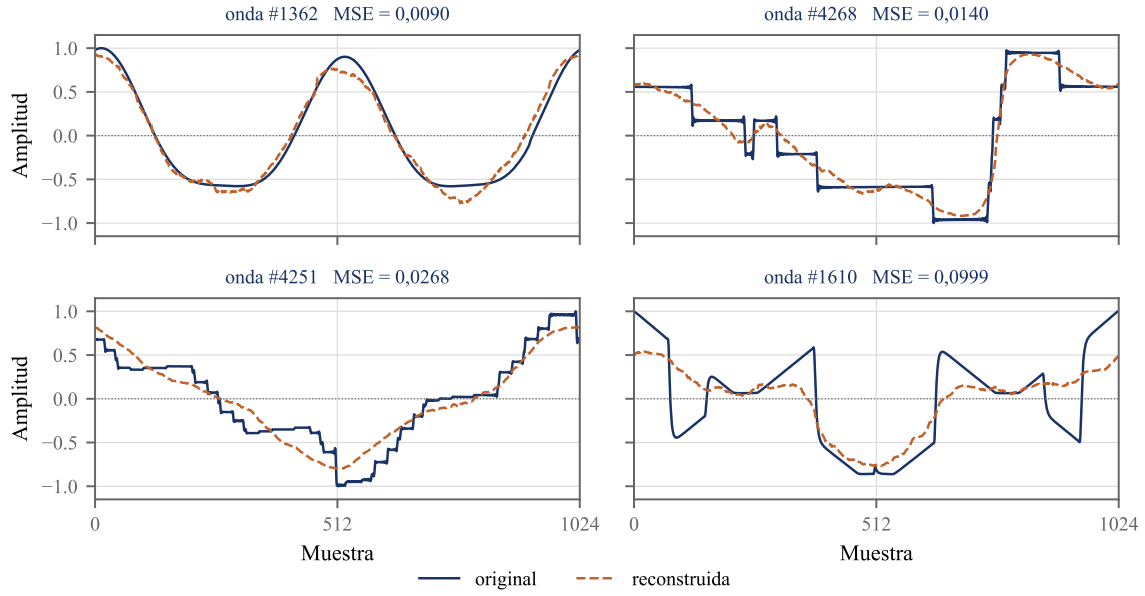


Figura 4.4: Cuatro ondas del conjunto de entrenamiento frente a su reconstrucción, con el error cuadrático medio de cada una. La pérdida de detalle se concentra en los flancos verticales.

El patrón es claro. Las ondas suaves, con pocos armónicos y sin discontinuidades, se recuperan casi exactas. Las que tienen escalones y flancos verticales pierden precisamente los flancos, que el decodificador sustituye por transiciones redondeadas. El error crece a medida que aumenta el número de esos escalones.

Este comportamiento es el límite intrínseco del cuello de botella de dos dimensiones y no un defecto del entrenamiento. Un flanco vertical exige mucho contenido de alta frecuencia y ese contenido no cabe en dos números. Obligado a elegir, el modelo aprende los rasgos perceptuales gruesos, la cantidad de brillo y el número de picos por ciclo, y descarta el detalle fino. Es la contrapartida directa de haber fijado el latente en dos dimensiones por exigencia del control táctil, condicionante que ya se enunció en el capítulo 1.

Con estos resultados sobre la mesa se planteó si convenía volver a entrenar antes de seguir. Se decidió que no, por cuatro motivos. El primero es que el objetivo del instrumento no es reproducir fielmente cada onda del conjunto, sino recorrer un plano de timbres con transiciones suaves. Para eso importa que el latente sea continuo y esté bien repartido, cosa que ya cumplía. El segundo es que subir β para suavizar aún más el plano habría acercado todo hacia la media, con lo que se gana regularidad y se pierde carácter. El tercero es que retirar del conjunto las carpetas de ondas más dentadas habría reducido la variedad tímbrica disponible sin resolver el suavizado de flancos, que depende de la dimensión del latente y no del material de entrada. El cuarto es que recortar el número de épocas ahorra tiempo de cálculo pero no cambia el modelo resultante. Se fijó, eso sí, un criterio para más adelante: si al escuchar el resultado apareciera algo objetivamente defectuoso, se reentrenaría entonces con ese dato y no antes.

4.8 Horneado de la rejilla

El último paso de la fase de preparación convierte el decodificador en una tabla. El script `4_bake_grid.py` codifica todo el conjunto para conocer la distribución real del plano, fija los límites de la zona que interesa muestrear, decodifica una onda en cada nodo de una rejilla regular y exporta el resultado.

La primera decisión es dónde poner los límites. Un cuadrado fijo y generoso, del tipo $[-6, 6]$ en ambos ejes, dejaría las esquinas de la rejilla en zonas del plano donde no cayó ninguna onda durante el entrenamiento. El decodificador sigue produciendo algo en esos puntos, pero es una extrapolación de la que no hay ninguna garantía. En su lugar se toman los percentiles 2 y 98 de cada eje, que recortan la zona efectivamente poblada y dejan fuera los pocos puntos extremos. Los valores resultantes son $[-2,239, 3,054]$ en el eje X y $[-3,010, 2,429]$ en el eje Y. Son los que dibuja el rectángulo de la Figura 4.3. El precio es perder los timbres más raros, que quedaban justamente en los bordes. A cambio, cualquier punto que toque el usuario cae en puntos con ondas cercanas.

La segunda es que la decodificación es determinista. El decodificador es una función fija de la coordenada, así que un mismo nodo produce siempre la misma onda; el muestreo aleatorio del autoencoder variacional solo interviene al codificar durante el entrenamiento. Lo que sí exige el modo de evaluación es el cálculo de los límites del párrafo anterior, que codifica el conjunto entero y toma la media de cada distribución en lugar de una muestra. Sin eso, los percentiles, y con ellos la rejilla, dependerían de la semilla y dejarían de ser reproducibles.

La tercera es el formato de salida. El decodificador entrega valores en $[-1, 1]$ y la cabecera los guarda como enteros de 16 bits con signo en formato Q15 tal como se fijó en el apartado 3.5. Así pues, las 256 tablas de 1024 muestras ocupan 512 kB de memoria de programa, frente al doble que habrían ocupado en coma flotante. La cabecera fija además la convención de índices que el firmware tiene que respetar, con la fila recorriendo el segundo eje y la columna el primero. Además, deja anotados los cuatro límites del plano latente como constancia de qué región representa la rejilla. Esos límites no llegan a usarse en el instrumento, porque al ESP32-S3 le basta con recorrer la rejilla por índices y la correspondencia con el plano latente queda implícita en el propio muestreo. Estos valores se guardan porque son lo que permite reconstruir de dónde salió cada tabla si se rehornea con otros valores.

La Figura 4.5 muestra el contenido de la rejilla horneada.

Al saltarse dos nodos de cada tres, la figura no sirve para juzgar si celdas vecinas se parecen entre sí, pero sí deja ver que el plano cubre formas muy distintas, desde pulsos casi rectangulares hasta ondas de varios picos con mucho contenido armónico. Que las celdas realmente contiguas se parezcan viene garantizado por otra vía, y es que el decodificador es una función continua y dos nodos adyacentes están muy próximos en el plano de entrada.

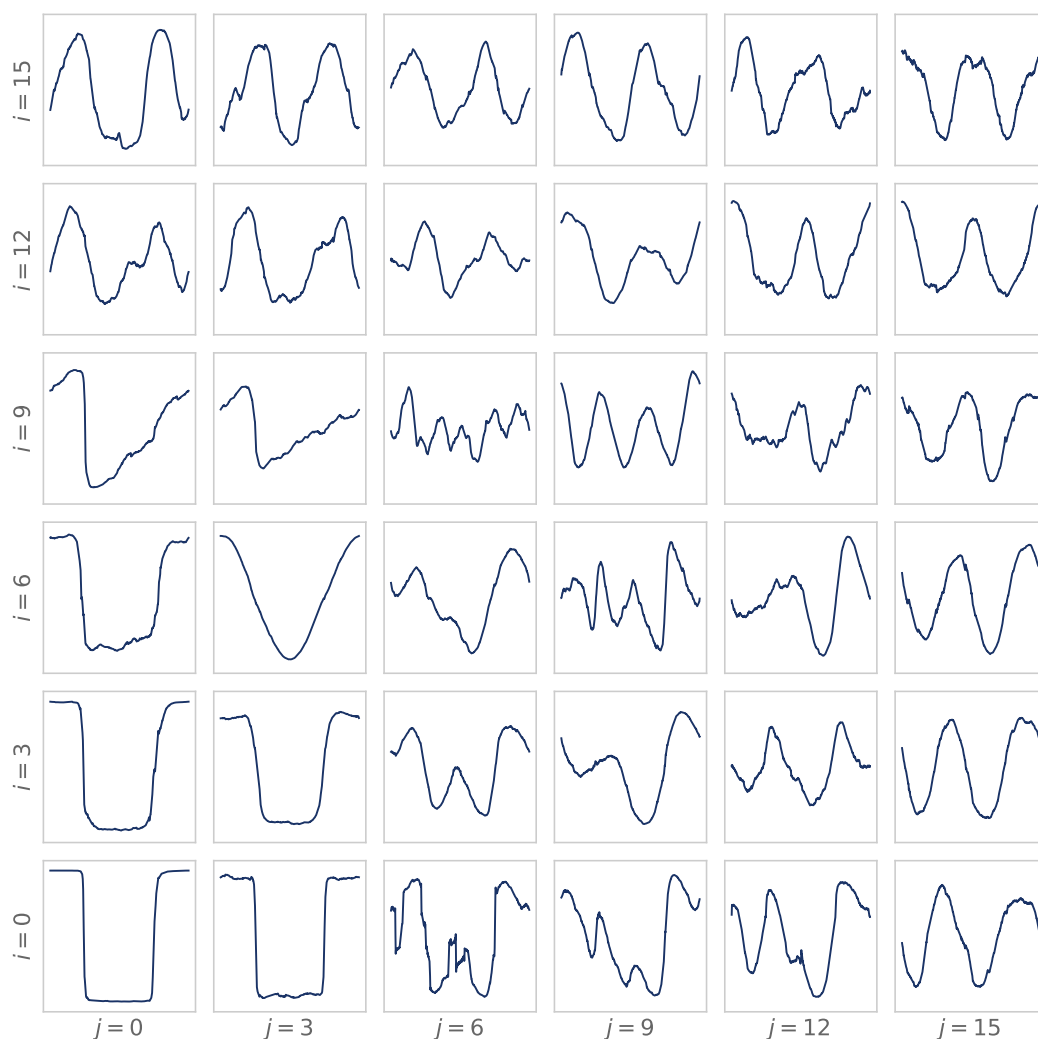


Figura 4.5: Contenido de la rejilla, dibujando uno de cada tres nodos en cada eje para que las ondas resulten legibles. El submuestreo permite ver la variedad de formas repartida por el plano, no la semejanza entre celdas contiguas.

4.9 Validación previa al embarque

Antes de llevar la rejilla al hardware convenía escucharla. Para eso se escribió `5_demo_morph.py` que reproduce en el ordenador exactamente lo que después hará el instrumento. Usando el ratón en lugar del dedo, haciendo interpolación bilineal entre las cuatro celdas vecinas y usando un acumulador de fase que cambia de tabla sin reiniciarse. Trabajar así permite separar los problemas del modelo de los del firmware, porque cualquier defecto que aparezca en esta demostración es necesariamente del modelo o de la rejilla.

La primera escucha dio un aviso. Al recorrer el plano, todas las posiciones sonaban bastante parecidas. Ese síntoma puede venir por dos causas muy distintas y conviene no confundirlas. Podía ser que la rejilla estuviera colapsada, con todo el plano decodificando prácticamente la misma onda pese a lo que indicaba la divergencia. O podía ser un efecto perceptual, porque el tono de prueba estaba fijado en 110 Hz y a esa altura los armónicos quedan tan juntos que timbres distintos tienden a fundirse al oído.

Para distinguirlas se añadieron tres pruebas de diagnóstico a la propia demostración. Un osciloscopio que dibuja la onda interpolada en tiempo real, así se puede ver si la forma cambia aunque el oído no lo note. Un arpegiador con el tono base subido a 220 Hz para escuchar un recorrido en un registro donde los armónicos se separan. Y un medidor de diversidad que calcula la distancia RMS entre la onda actual y la onda media de la rejilla, junto a una sonda que compara las cuatro esquinas al arrancar.

Las medidas descartaron el colapso. La distancia RMS entre las cuatro esquinas de la rejilla va de 0,63 a 1,04 según el par que se compare, y la diversidad respecto a la onda media recorre de 0,18 a 0,68 al barrer el plano. Sobre ondas normalizadas en $[-1, 1]$ son diferencias grandes, no matices. Al subir el tono, la variación tímbrica pasó a ser evidente también al oído, con lo que la causa quedó identificada como perceptual y atribuible al tono grave de la prueba inicial.

Con esa verificación, la fase de preparación queda cerrada. El resultado que pasa al resto del sistema es un único archivo, la cabecera con las 256 tablas y los límites del plano latente, que se compila junto al firmware del ESP32-S3 descrito en el capítulo siguiente.

Capítulo 5

Firmware de control: ESP32-S3 y CYD

Este capítulo recorre las dos placas que convierten un dedo apoyado sobre un cristal en una forma de onda lista para sonar. El reparto de tareas entre procesadores, la elección del medio de cada enlace y el tamaño de las tramas ya quedaron justificados en el capítulo 3, en los apartados 3.3 y 3.6. Aquí interesa el cómo, es decir, cómo se lee y se calibra el táctil, cómo se localiza la celda de la rejilla y se mezclan sus vecinas, cómo se construyen las tramas y cómo se reduce la onda para dibujarla. Todo lo que sigue corresponde a la fase de ejecución descrita en el apartado 3.4, en la que no interviene ninguna red neuronal.

5.1 Lectura y calibración del táctil

La CYD es una placa genérica sin manual de fabricante, cuya documentación es comunitaria [13], así que los pines se confirmaron uno a uno contra el esquemático que la acompaña. Integra una pantalla TFT de 240×320 puntos y, encima de ella, una lámina táctil resistiva. Esa lámina no entrega una posición, sino una tensión que cambia según dónde se apoye el dedo, y quien la traduce a número es un XPT2046, un conversor analógico-digital de 12 bits [14]. Doce bits dan 4096 valores, de modo que cada eje se lee como un entero entre 0 y 4095. Conviene retener que esos números no son píxeles ni milímetros, sino cuentas del conversor. De ahí sale todo lo que viene después, calibrar consiste precisamente en averiguar qué cuenta corresponde a cada borde.

La pantalla y el táctil son dos chips distintos y cada uno se comunica con el ESP32 por su propio bus SPI. La placa dispone de dos utilizables, llamados VSPI y HSPI, y aquí se usa uno para cada cosa. El táctil se queda en el VSPI, que es donde lo dejó cableado el fabricante, y la pantalla se lleva el HSPI. El reparto no es opcional porque la librería de la pantalla, TFT_eSPI [15], toma el VSPI por omisión y sin forzarla los dos chips acabarían disputándose el mismo bus. El HSPI sale además ganando, ya que los pines 12 a 15 son los que el ESP32 tiene reservados para él [16] y eso les da una conexión interna directa, más rápida que la ruta genérica. El táctil se lee con la librería de Stoffregen [17].

Esa configuración vive en el fichero de proyecto de PlatformIO y no en el `User_Setup.h` de la librería, que es donde normalmente se pone. El motivo es práctico. Ese fichero se instala dentro del directorio de dependencias descargadas, fuera del control de versiones, de modo que la configuración se perdería al reinstalar la librería o al clonar el proyecto en otro ordenador. Declarándola

como definiciones de compilación viaja con el repositorio. Hay además un detalle de la placa que conviene no reaprender, y es que la retroiluminación arranca apagada. La pantalla puede estar pintando perfectamente y verse negra, indistinguible de un cuelgue, así que se enciende a mano en la primera línea del arranque. Justo después se dibuja un patrón de barras de color con un marco, que permite validar el controlador, la orientación y los colores sin depender todavía del ESP32-S3.

5.1.1 De las cuatro esquinas a las dos ventanas

La primera calibración se hizo tocando las cuatro esquinas del cristal con un puntero, que al tener un área de contacto pequeña da lecturas más extremas y limpias que el dedo. De ahí salieron los límites 133 a 3947 en el eje horizontal y 193 a 3897 en el vertical.

Esos números dejaron de valer cuando la placa se montó girada 180 grados en el panel, decisión de montaje que se explica en el capítulo 7 y que busca alejar el conector USB de la pantalla OLED. Al girar la imagen hay que girar también el táctil, y aquí apareció un hallazgo que ahorró trabajo. La librería sí aplica la rotación a las lecturas crudas, restándolas de 4095, lo que evita invertir el mapeo a mano. Pero, por eso mismo, los cuatro límites de calibración quedan espejados y hay que rehacerlos cruzando mínimo con máximo, con lo que pasan a 148 a 3962 y 198 a 3902.

Falta descontar lo que el panel tapa. Con la placa ya montada, tocando las cuatro esquinas del hueco, el área que asoma resultó ir de 450 a 3730 en los dos ejes. Se intentó afinar esquina por esquina y se descartó porque las lecturas no cuadraban entre sí, señal de que el táctil está algo girado respecto al hueco y un mapeo rectangular no corrige un giro. Apareció además un problema que ese recorte no resolvía. El labio impreso que apoya sobre el cristal en todo el perímetro genera pulsaciones fantasma pegadas al borde, una de ellas medida en (3714, 643), es decir dentro del área que se había dado por buena. Filtrar por presión habría sido la salida evidente, pero la estimación que da el XPT2046 en esta placa es poco fiable y un umbral mal ajustado podía dejar el táctil mudo.

La solución fue dejar de usar un solo rectángulo y separar dos, recogidos en la Figura 5.1. La ventana *visible*, de 450 a 3730, describe lo que asoma por el hueco y solo se emplea para calcular el área de dibujo. La ventana de *captura*, de 600 a 3580, decide qué toques se dan por buenos y deja 150 cuentas de margen por cada lado, unos 2,5 mm. El fantasma medido queda fuera de ella con 134 cuentas de holgura.

El detalle que hace que esa holgura no cueste nada es cuál de las dos ventanas se mapea al rango de la coordenada. Se mapea la de captura, no la visible, de modo que el espacio latente se sigue alcanzando entero de extremo a extremo y lo único que se pierde son esos 2,5 mm de recorrido del dedo por lado. Mapear la visible habría dejado las esquinas del plano inalcanzables, que es justamente el defecto que la recalibración venía a quitar. Y el rechazo es total, no un recorte. La versión anterior sujetaba la lectura al rango con una función de saturación, así que un contacto del plástico se enviaba igual, clavado contra el borde del espacio latente. Ahora un toque fuera de la ventana de captura no genera envío ni repinta nada, y el instrumento se queda exactamente como estaba.

El táctil se atiende cada 50 ms, sin bloquear el bucle, y solo se envía coordenada cuando el punto se ha desplazado más de 300 unidades respecto al último enviado. Como una celda de la rejilla ocupa 4369 unidades del rango de 16 bits, ese umbral es menos de la décima parte de una celda, suficientemente fino para que el recorrido se perciba continuo y suficientemente grueso para no saturar el enlace con la vibración de un dedo quieto.

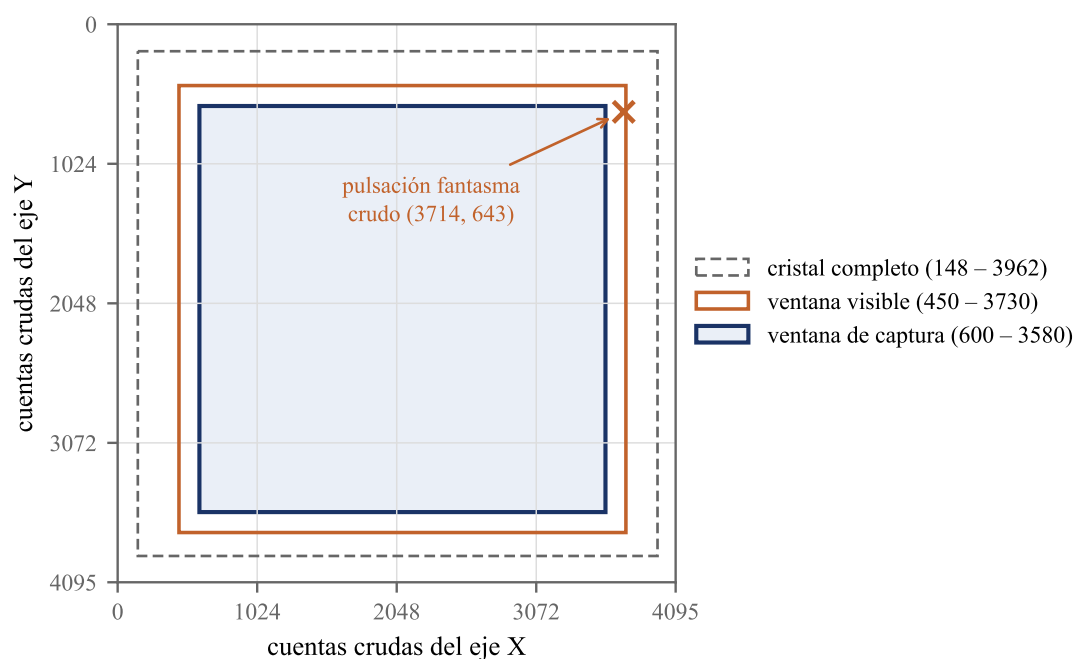


Figura 5.1: Las dos ventanas del táctil sobre el plano de cuentas crudas del convertidor. La de captura decide qué toques valen y es la que se mapea al rango de la coordenada; la visible solo sirve para acotar el área de dibujo. La pulsación fantasma medida cae dentro de la visible y fuera de la de captura.

5.2 De la coordenada a la celda

La CYD normaliza la lectura al rango de 16 bits antes de enviarla, y esa decisión tiene una consecuencia comprobada. Ni el giro de 180 grados ni la recalibración de banco obligaron a tocar una sola línea del firmware del ESP32-S3, porque este sigue recibiendo el mismo rango de siempre. Resolver el giro en el procesador de control habría roto la separación de responsabilidades del apartado 3.3 y habría dejado a la CYD enviando coordenadas que no se corresponden con lo que ella misma muestra.

Del lado del ESP32-S3, la trama de seis bytes descrita en el apartado 3.6 la decodifica una máquina de estados que busca el byte de cabecera, lee los cuatro de datos y valida la suma de comprobación. Ante cualquier fallo vuelve al estado inicial a buscar cabecera, de forma que un byte perdido descuadra una trama pero no el flujo. Conviene señalar que esa entrada también se puede hacer por USB. Aquí la coordenada se escribe a mano como texto mediante el puerto serie. Fue lo que permitió construir y validar la interpolación antes de que existiera el firmware de la CYD, y es también lo que hace posibles las medidas del capítulo 8, que necesitan una coordenada conocida y estable.

5.3 Interpolación bilineal sobre la rejilla

Localizar la celda es una multiplicación. La coordenada de 16 bits se escala por 15/65535, con lo que se obtiene la posición real dentro de la rejilla de 16×16 . La parte entera de esa posición da la celda y la parte fraccionaria da el peso hacia la vecina siguiente. Los dos ejes se tratan de forma independiente, así que un punto asimétrico no recibe ningún trato especial.

El borde se resuelve sin caso particular. Cuando la coordenada vale el máximo, la posición escalada da exactamente 15, la vecina de la derecha se sujeta sobre sí misma y el peso hacia ella queda a cero. La operación sigue siendo la misma combinación de cuatro tablas, solo que dos de ellas coinciden y aportan peso nulo, con lo que no se sale del array ni hace falta una rama distinta.

Hay que respetar la disposición matricial de índices que fija la cabecera generada en el apartado 4.8, con la fila recorriendo el eje vertical y la columna el horizontal. Es el detalle más fácil de equivocarse de todo el firmware y el que peor avisa cuando se equivoca, porque permutar los ejes no rompe nada visible. La interpolación sigue devolviendo formas de onda perfectamente válidas, solo que el espacio de timbres queda transpuesto bajo el dedo.

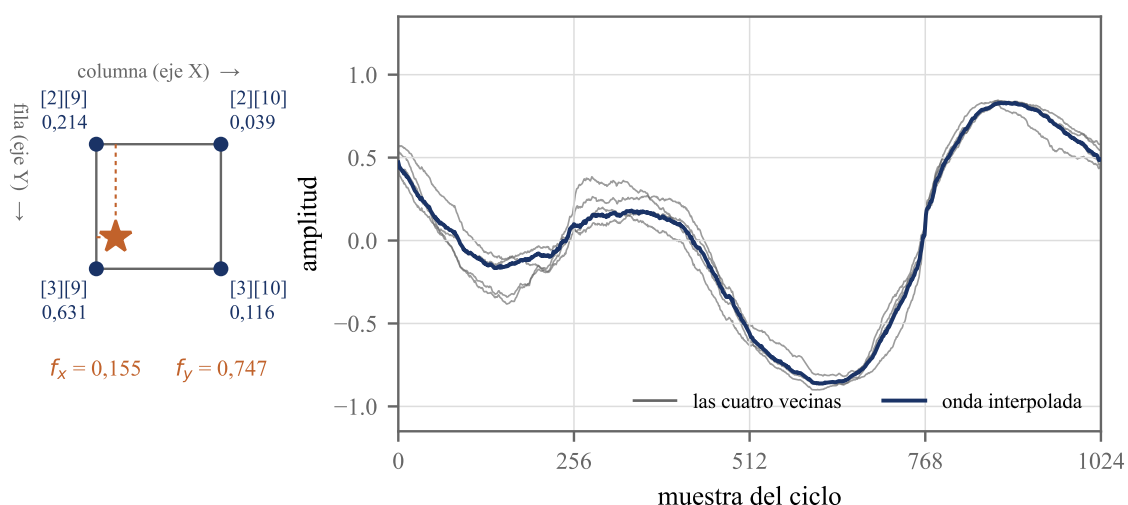


Figura 5.2: Interpolación bilineal para la coordenada (40000, 12000). A la izquierda, la celda alcanzada, los índices de las cuatro vecinas en la rejilla y el peso que recibe cada una. A la derecha, esas cuatro formas de onda y la que resulta de mezclarlas.

La Figura 5.2 muestra el resultado para la coordenada con la que después se validó el enlace. Cae en la celda que va de la columna 9 a la 10 y de la fila 2 a la 3, con pesos de 0,155 y 0,747, de modo que la vecina inferior izquierda se lleva 0,631 de la mezcla y la superior derecha apenas 0,039. Se aprecia que la onda resultante no es una de las cuatro ni un promedio simple, sino que sigue de cerca a la que más pesa.

Merece una aclaración cómo se hace la aritmética, porque el formato de almacenamiento y el de cálculo no coinciden. Las tablas se guardan como enteros de 16 bits en formato Q15, pero la mezcla se calcula en coma flotante de 32 bits y solo se redondea al pasar el resultado a Q15, una vez por muestra. El ESP32-S3 tiene unidad de coma flotante en hardware, así que las más de mil operaciones que esto cuesta son despreciables frente a los 1,6 ms que después ocupa la transferencia. Cabe recordar que el cálculo se ejecuta a frecuencia de control, unas decenas de veces por segundo como mucho, así que ese coste es irrelevante. La ventaja de hacerlo así es que cada muestra sufre un único redondeo en lugar de acumular el de cada paso intermedio. Tampoco hace falta saturar el resultado porque los cuatro pesos suman uno y están todos entre cero y uno. De este modo, la mezcla es una combinación convexa de cuatro valores que ya caben en Q15 y no puede salirse del rango.

La rejilla se compila como una constante de $16 \times 16 \times 1024$ enteros de 16 bits, es decir los 512 kB del apartado 3.5, y vive en memoria de programa. En el ESP32-S3 la caché mapea esa memoria

al espacio de direcciones, de modo que se lee como cualquier otro array, sin las macros de acceso que exigen otras arquitecturas. En memoria de datos quedan la tabla de trabajo y los dos búferes de trama, unos cuatro kilobytes. La PSRAM de 8 MB de la placa no llega a intervenir en el camino principal y queda como margen para la ampliación planteada en el apartado 3.2.

Para comprobar que esta implementación es correcta no basta con ver que produce algo. Se escribió una reimplementación independiente en numpy que recalcula la misma interpolación sobre el fichero de la rejilla y se compara muestra a muestra con el volcado que el ESP32-S3 imprime por USB. Es una comprobación de equivalencia frente a una referencia, no de plausibilidad. El resultado se recoge en el capítulo 8.

5.4 El canal de vuelta y el dibujo de la onda

Lo que el ESP32-S3 devuelve a la pantalla es dato de representación, nunca de síntesis. La rejilla y la interpolación siguen viviendo solo en el procesador de control, y la forma de onda real únicamente sale hacia el procesador de audio por SPI. El enlace con la CYD ya era bidireccional, así que abrir el sentido de vuelta no costó más que añadir el hilo y un emisor.

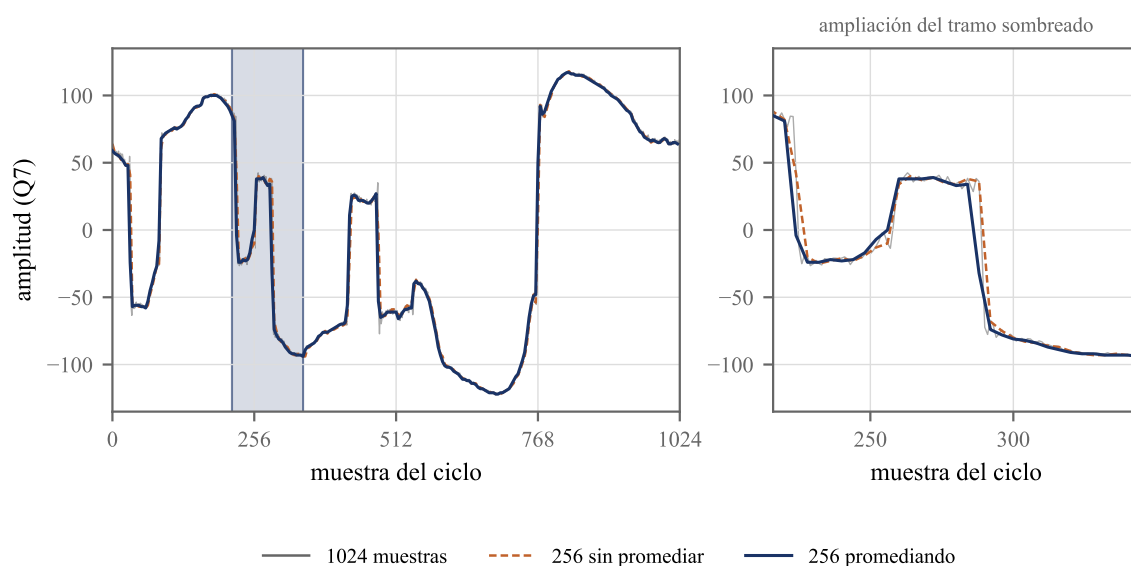


Figura 5.3: Reducción de 1024 a 256 puntos para dibujar, sobre el nodo de la rejilla en el que más se separan los dos métodos. Promediar cada grupo de cuatro muestras y quedarse con una de cada cuatro dan prácticamente lo mismo en el grueso del ciclo, y difieren en los flancos.

Reducir la onda a 256 puntos, cuyo porqué se explicó en el apartado 3.6, admite dos maneras. La inmediata es quedarse con una muestra de cada cuatro y descartar las otras tres. La que se usa aquí es promediar cada grupo de cuatro, que cuesta prácticamente lo mismo y no tira información por el camino. Cada muestra pasa además de 16 a 8 bits conservando la mitad alta. La Figura 5.3 enfrenta los dos métodos sobre el nodo de la rejilla en el que más se diferencian. En casi todo el ciclo coinciden, y la separación se concentra en los tramos de pendiente fuerte, donde alcanza 68 cuentas sobre una amplitud máxima de 122. Descartar muestras no haría irreconocible el dibujo, pero desplazaría los flancos, que es justamente donde se aprecia si una onda es cuadrada, de sierra o de pulso.

Dos tropiezos de esta parte tienen causa identificada y merece la pena dejarlos escritos. El primero fue que no llegaba ninguna trama buena, no que llegaran corruptas. El buffer de recepción del puerto serie en esta plataforma es de 256 bytes por omisión, justo por debajo de los 260 que ocupa la trama, y hay que agrandarlo antes de abrir el puerto. El segundo fue que las tramas se perdían solo mientras el dedo se movía, que es justo cuando llegan. La causa era una espera bloqueante en la lectura del táctil, que desatendía la recepción durante ese tiempo; se substituyó por una temporización no bloqueante.

El repintado se hace en dos pasadas, borrando entera la traza anterior con el color de fondo antes de dibujar la nueva. Repintar el área de golpe con un rectángulo relleno produce parpadeo en cada trama, y borrar e ir pintando de forma intercalada se come trozos de la línea nueva allí donde las dos se cruzan. El borrado arrastra también los puntos de la línea central de referencia, que se repintan sobre la marcha.

Queda el encaje con el panel. Como este tapa un borde del cristal, dibujar sobre la pantalla completa dejaría parte de la interfaz debajo del plástico, así que el área útil se acotó a los píxeles que salen de convertir la ventana visible de la Figura 5.1. Dentro de ella se reparten un título, una banda de estado con la coordenada, el número de secuencia y el contador de tramas descartadas, y el recuadro de la onda. El patrón de arranque cierra el asunto de forma barata, porque dibuja un marco blanco justo en el límite del área útil y basta con ver si los cuatro lados asoman por la ventana para saber si las constantes cuadran con el hueco.

5.5 Envío de la tabla al procesador de audio

El ESP32-S3 tiene cuatro controladores SPI, pero dos están permanentemente ocupados hablando con la memoria del propio chip y no se pueden usar [18]. Se emplea uno de los dos libres. Los pines tampoco se eligieron con libertad porque esta versión de la placa monta unas memorias que ocupan más patillas de lo habitual y otra estaba ya dedicada a recibir el UART de la CYD. Los GPIO 10 a 13 fueron los que quedaron.

En este enlace manda el ESP32-S3 y la Daisy escucha, a 10 MHz, en modo 0 y con el bit más significativo por delante, que son los ajustes que los dos extremos tienen que compartir para entenderse. La línea que anuncia el comienzo de cada transferencia se gobierna desde el programa en vez de dejarla al periférico, porque la función que envía el bloque de golpe no la mueve por su cuenta. Los 2054 bytes tardan 1,6 ms, y durante ese rato el programa no hace nada más, cosa que no estorba cuando se envía unas pocas decenas de veces por segundo.

El contenido de la trama es el del apartado 3.6. Las 1024 muestras se copian en bloque, sin recorrer el array muestra a muestra, porque los dos extremos del enlace almacenan los enteros de 16 bits en el mismo orden de bytes y no hay nada que reordenar. Sobre ese bloque se calcula el CRC de 16 bits que cierra la trama.

Ese CRC está escrito tres veces de forma idéntica, en el emisor, en el receptor y en el guion de validación. Duplicar código suele ser mala señal, pero aquí es deliberado, ya que tener la misma función en los tres sitios permite comparar el valor calculado en cada uno y distinguir un fallo de transporte de un fallo de cálculo. La validación cruzada del enlace tiene una particularidad de método que conviene contar. El primer intento se hizo deslizando el dedo por la CYD, y resultó inútil, porque con el dedo en movimiento no hay forma de saber con certeza qué trama recibida corresponde a qué coordenada enviada. Se repitió con una coordenada fija escrita por USB, que

es lo que da una referencia inequívoca. Las cifras están en el capítulo 8.

5.6 Estado del subsistema

Con esto queda cerrada la cadena que va del dedo a la trama de audio. La CYD lee el táctil, lo filtra contra la ventana de captura, lo normaliza y lo envía; el ESP32-S3 localiza la celda, mezcla las cuatro vecinas y entrega la tabla por SPI, además de devolver a la pantalla una versión simplificada de lo que está sonando. Las dos comprobaciones cuantitativas que sostienen este capítulo, la equivalencia de la interpolación frente a la referencia y la integridad del enlace, se recogen juntas en el capítulo 8 para poder leerlas al lado del resto de medidas del sistema montado. Lo que llega al otro extremo del bus son 2054 bytes cada vez que el dedo se mueve, y qué se hace con ellos es el asunto del capítulo siguiente.

Capítulo 6

Firmware de audio: Daisy Seed

El capítulo anterior terminaba con 2054 bytes saliendo por el bus cada vez que el dedo se mueve. Este cuenta qué se hace con ellos. El reparto de tareas entre placas, la elección del medio de cada enlace y el tamaño de las tramas están justificados en el capítulo 3, y el lado emisor del enlace, con su empaquetado y su CRC, en el apartado 5.5. Aquí interesa el cómo del lado del audio, es decir, cómo recorre la tabla el acumulador de fase, cómo se recibe la trama sin perturbar el sonido, cómo se resuelve la transición de un timbre al siguiente, cómo entran la nota y la dinámica y cómo se leen los mandos del panel.

La Daisy Seed es una plataforma de DSP embebido construida alrededor de un MCU ARM Cortex-M7 a 480 MHz, con hardware de audio de hasta 96 kHz y 24 bits [19]. El microcontrolador concreto es un STM32H750, que lleva unidad de coma flotante (FPU) de doble precisión e instrucciones de tratamiento de señal [20]. Esa FPU es la que permite trabajar en coma flotante en todo el camino de audio en lugar de en aritmética de punto fijo, y por tanto olvidarse de los desbordamientos intermedios. El firmware es una única unidad de compilación construida sobre las dos bibliotecas oficiales de la plataforma, `libDaisy` para el acceso al hardware [21] y `DaisySP` para los bloques de procesado [22].

Lo primero que conviene fijar es que el firmware trabaja a tres ritmos distintos, recogidos en la Figura 6.1, y que casi todas las decisiones de este capítulo salen de decidir qué va en cada uno. La cadena de audio se recorre una vez por muestra, a 48 kHz. La lectura de los mandos ocurre una vez por bloque de 48 muestras, es decir a 1 kHz. Y la atención al MIDI, a las tramas y al display vive en el bucle principal, sin cadencia garantizada. Las flechas discontinuas de la figura son las que cruzan de un ritmo a otro, y son justamente las que hay que resolver con cuidado.

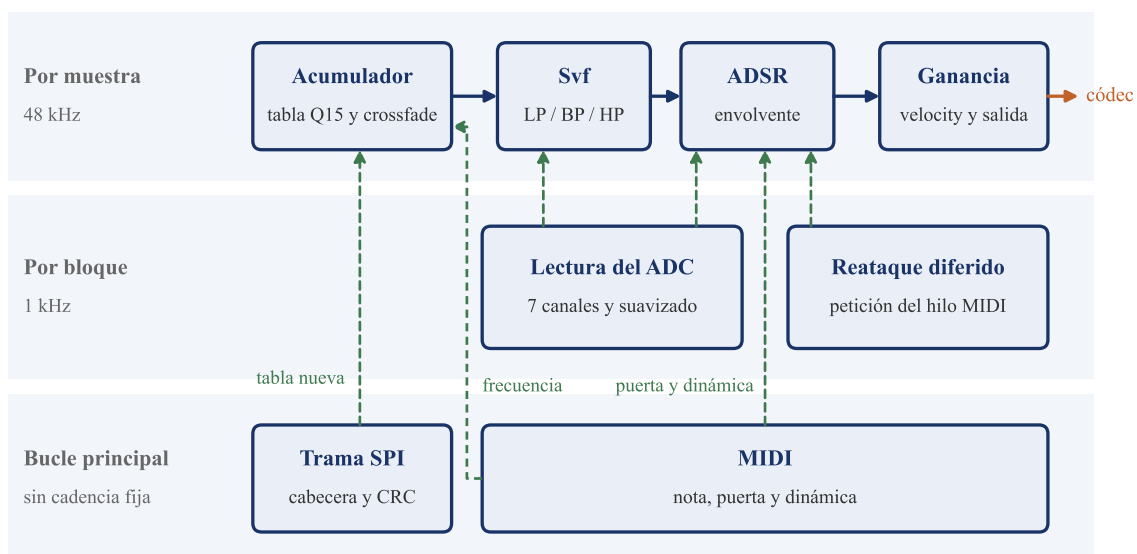


Figura 6.1: Estructura del firmware de audio y los tres ritmos a los que trabaja. Las flechas continuas son la cadena de audio, muestra a muestra; las discontinuas, la información que cruza de un ritmo a otro.

6.1 El recorrido de la tabla

El acumulador de fase y la interpolación entre muestras vecinas quedaron descritos como técnica en el capítulo 2, con sus ecuaciones (2.1) a (2.3). Lo que toca aquí es cómo se implementan.

La fase se guarda como un número en coma flotante de 32 bits que avanza $f_0 \cdot 1024 / f_s$ en cada muestra, con la frecuencia de muestreo que reporta la propia placa y no un valor supuesto. La parte entera de la fase da el índice de la muestra anterior y la parte fraccionaria, el peso hacia la siguiente. El hecho de que el tamaño de tabla sea potencia de dos, fijado en el apartado 3.5, es útil porque el índice siguiente se obtiene con una máscara de bits en lugar de con una operación de módulo, y el retorno al principio del ciclo se resuelve restando el tamaño de la tabla en vez de dividiendo.

Las tablas se almacenan en el mismo formato en que llegan, enteros de 16 bits en Q15, así que entre la recepción y la reproducción no hay ninguna conversión. El paso a coma flotante se hace al leer cada muestra, dividiendo por 32768. Podría plantearse hacer todo el camino en aritmética entera, pero no compensa. Con la FPU del Cortex-M7 una multiplicación en coma flotante no cuesta más que una entera y el margen dinámico deja de ser un problema en cuanto se encadenan el filtro, la envolvente y dos ganancias.

Tras la inicialización las dos tablas quedan a cero, de modo que la placa sería muda hasta que llegara la primera trama válida del procesador de control, aunque el MIDI estuviera funcionando perfectamente. Eso convierte el primer escalón de cualquier encendido por partes en un falso negativo garantizado. Para evitarlo se construye una tabla senoidal y se carga en el búfer activo antes de arrancar el audio. Con ella la Daisy es un instrumento autónomo desde el primer segundo, y lo que hace el enlace SPI es *sustituir* ese timbre en lugar de crearlo.

6.2 La llegada de la tabla

La Daisy actúa como esclavo del bus, sobre el primero de sus controladores SPI y con los pines que fija la propia placa [19]. El enlace se configura de recepción exclusiva y con el selector de chip gobernado por hardware, así que el firmware no tiene que vigilar ninguna línea por su cuenta.

La pieza que hace esto viable es el DMA. Los 2054 bytes de la trama tardan 1,6 ms en entrar, que a 48 kHz son casi dos bloques de audio. Atender el bus byte a byte desde el procesador dejaría el audio sin llenar durante todo ese tiempo. Con acceso directo a memoria la transferencia ocurre sin intervención del procesador, que sigue generando muestras mientras la trama entra. El búfer de recepción vive además en una región de memoria reservada para el DMA, que es obligatorio en este microcontrolador porque tiene memorias caché y un búfer normal podría entregar datos antiguos.

La rutina que se ejecuta al terminar la transferencia corre en contexto de interrupción y hace una sola cosa, levantar una bandera. La comprobación de la cabecera, el cálculo del CRC y la copia del contenido se hacen después, en el bucle principal. La razón es de coste. El CRC recorre 2048 bytes bit a bit, y hacerlo dentro de la interrupción retrasaría la del audio, que es la única que no puede esperar. Si la transferencia termina con error, en cambio, la propia rutina vuelve a armar la recepción de inmediato, porque en ese caso no hay nada que validar.

El tratamiento de la trama sigue el criterio del apartado 3.6. Se comprueba la cabecera de sincronización, se recalcula el CRC sobre el contenido y se compara con el recibido. Solo si las dos comprobaciones pasan se copia la tabla y se pide la transición. Conviene señalar el orden, porque la recepción se rearma inmediatamente después de validar y copiar, antes de escribir nada por el puerto serie. Cuando algo falla no se hace nada más, y el instrumento sigue sonando con la tabla anterior en lugar de reproducir muestras corrompidas. La comprobación de que el transporte es exacto, con su error medido en LSB, está en el capítulo 8.

6.3 La transición entre tablas

Que la transición se resuelva en el procesador de audio y no en el de control ya quedó justificado en el apartado 3.2 por ser este el único que conoce la posición del acumulador. Aquí interesa cómo se hace y por qué dura lo que dura.

El motor mantiene dos tablas, la activa y la siguiente. Al llegar una tabla nueva se copia sobre la siguiente y, solo cuando la copia ha terminado, se puede comenzar la transición poniendo su posición en 0. Ese orden es toda la sincronización del mecanismo, y merece explicarse porque no es evidente. El bucle principal escribe la tabla mientras el callback de audio lee, así que hay una carrera real entre los dos. Como la posición de la transición es lo único que el callback consulta para decidir si mira la tabla nueva, y se escribe después de la copia, el callback no puede llegar nunca a un búfer a medio escribir.

Durante la transición las dos tablas se leen *en la misma posición del acumulador* y se mezclan con pesos complementarios, que avanzan de cero a uno a lo largo de 20 ms. Al terminar, la tabla nueva pasa a ser la activa. Que ambas se lean en la misma posición tiene una consecuencia importante, y es que no hay discontinuidad de fase en el momento del cambio. Lo único que varía de golpe es la amplitud en ese punto concreto.

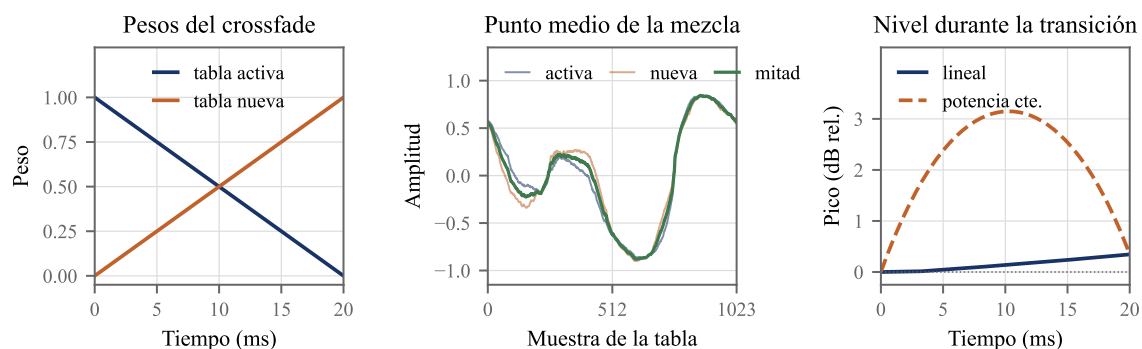


Figura 6.2: La transición entre dos ondas de celdas contiguas de la rejilla, las mismas que alcanza la coordenada del apartado 5.3. A la izquierda, los pesos a lo largo de los 20 ms. En el centro, las dos tablas y su mezcla a mitad de camino. A la derecha, el nivel de pico durante la transición con mezcla lineal y con mezcla de potencia constante.

De ahí sale la elección de mezcla lineal, que a primera vista parece la opción sencilla. La mezcla de potencia constante es útil para señales *descorrelacionadas*, donde la lineal deja un hueco de 3 dB en el punto medio porque las dos señales no se refuerzan. Aquí ocurre lo contrario. Todas las formas de onda comparten referencia de fase, fijada en el preprocesado del apartado 4.2 y además se leen en la misma posición del acumulador. De este modo, sus fundamentales y armónicos bajos suman en vez de cancelarse. La Figura 6.2 lo mide sobre las dos celdas contiguas del ejemplo del apartado 5.3, cuyo coeficiente de correlación es 0,971. Con mezcla lineal el pico de la señal se mantiene dentro de 0,34 dB durante toda la transición, mientras que la de potencia constante se va a 3,15 dB por encima. Es decir, en este régimen la opción teóricamente más elaborada sería la que introduce el problema.

La duración de 20 ms se fijó tras probar 5, 10 y 20. A 220 Hz un ciclo dura unos 4,5 ms, así que 20 ms son apenas cuatro ciclos y la transición es corta en cualquier caso. Ahora bien, al ajustar ese valor apareció un resultado que conviene contar tal cual, porque a primera vista parecía un fallo. Cambiando la duración entre 5 y 20 ms no se apreciaba ninguna diferencia demasiado notoria, ni siquiera poniéndola a cero. La comprobación que resolvió la duda fue subirla a 500 ms, un valor absurdo para el instrumento pero útil como diagnóstico. Con medio segundo se oye sin ninguna ambigüedad cómo un timbre nace despacio sobre el otro. El motor funciona pero la diferencia era prácticamente imperceptible entre las ondas elegidas. Al coger otras, con armónicos más distintos, la transición de 20 ms sí se nota y queda como un compromiso entre suavidad y respuesta.

Queda una limitación conocida que el crossfade no arregla dure lo que dure. Al saltar entre timbres muy distintos se percibe un cambio de volumen, y no es un artefacto de la transición sino una diferencia de valor eficaz. La normalización del apartado 4.2 iguala el *pico* de todas las ondas, no su energía, que es lo que el oído interpreta como volumen. Una cuadrada tiene valor RMS igual a su pico y una senoidal, un 70 % del suyo. Corregirlo exigiría normalizar por RMS en el conjunto de datos, no tocar la transición. Esto tiene su propia pega, las ondas con factor de cresta alto se recortarían. Se dejó como está, porque en el uso real el recorrido ocurre entre celdas vecinas, que como muestra la figura son casi idénticas.

6.4 Nota y dinámica

La interfaz MIDI de hardware es un lazo de corriente de 5 mA que transmite a 31 250 baudios, de forma asíncrona y con el bit menos significativo por delante. Además, exige que emisor y receptor estén separados por un optoacoplador para no unir sus masas [23]. Esa separación es la que justifica que el teclado sea un aparato aparte con su propia alimentación, como se explicó en el apartado 3.3. La señal entra por el USART1 de la placa, cuya entrada tolera 5 V [19], así que la salida del optoacoplador se conecta directamente sin adaptación de niveles. El circuito de entrada pertenece al capítulo 7.

Del lado del firmware, la recepción también se hace por DMA, con un búfer propio y sobre un periférico distinto del que atiende el bus SPI [21]. Es lo que permite que las dos entradas convivan sin estorbarse, y de hecho añadir el MIDI no obligó a tocar ni una línea del receptor de tramas. El instrumento responde a todos los canales, sin filtrar, que para este uso es más robusto que quedarse con uno.

La voz es monofónica con prioridad de última nota. Cada tecla pulsada entra en una pila y fija la frecuencia. Al soltarla se vuelve a la anterior que siga pulsada, cambiando el tono pero sin reatacar la envolvente, lo que produce un ligado natural. Si no queda ninguna, se suelta la puerta (MIDI gate) y la envolvente entra en su tramo de liberación. La dinámica se guarda en una pila paralela, porque al volver a una tecla pulsada hay que recuperar también con qué fuerza se tocó. Un mensaje de nota activa con dinámica cero se trata como si fuera de nota apagada, que es lo que exige la norma y además la forma más habitual de apagar una nota, porque permite encadenar mensajes sin repetir el byte de estado [23].

La norma numera las teclas con enteros y sitúa el do central en el 60 [23], de donde el *La* de 440 Hz es la nota 69. El paso a frecuencia es el del temperamento igual, en el que una octava son doce semitonos y a la vez un factor de dos, así que la razón entre dos notas contiguas es la raíz duodécima de dos. La frecuencia de la nota n queda entonces como la Ecuación (6.1).

$$f(n) = 440 \cdot 2^{(n-69)/12} \quad (6.1)$$

Esta conversión está escrita una sola vez y se usa tanto en el sonido como en el volcado con el que se valida, de modo que la tabla que comprueba el guion de verificación es exactamente la que suena. El método está aquí; las cifras, en el capítulo 8.

La dinámica se traduce a ganancia con una curva cuadrática en lugar de lineal. Con respuesta lineal casi todo el rango útil se concentra en la mitad alta del recorrido de la tecla. Al elevar al cuadrado, la sensación se reparte de forma más parecida a como se percibe la sonoridad. Esa ganancia se suaviza además con un filtro de un polo de unos 5 ms. Esto se hace debido a que el reataque de la envolvente es suave, no baja a cero. Así pues, al tocar una nota nueva mientras suena otra el salto de ganancia caería sobre una señal en marcha y se oiría como un chasquido.

Toda modificación de la envolvente se aplaza a una bandera y se ejecuta dentro del callback de audio. La regla que se sigue en todo el firmware es que quien llama al procesado de un bloque es también quien lo modifica, porque las funciones de DaisySP no están pensadas para que se las reconfigure desde otro hilo mientras están procesando.

Al final de la cadena hay una ganancia global fija de 0,8 porque compensar el nivel aguas abajo, en un previo, amplificaría señal y ruido por igual.

6.5 Envoltente y filtro desde el panel

Los siete canales del panel, cuatro para la envoltente, dos para el filtro y uno para el selector de tipo, se leen por el ADC de la placa, que libDaisy configura a 16 bits con sobremuestreo por 32 y entrega ya normalizado entre cero y uno [21]. Los potenciómetros cuelgan de la alimentación analógica de 3,3 V porque el conversor referencia su fondo de escala a esa misma tensión y así cualquier deriva afecta por igual a referencia y señal, y se cancela. El montaje está en el capítulo 7.

La lectura se hace una vez por bloque de audio, es decir a 1 kHz, y desde dentro del callback, por la misma razón de la sección anterior. Los siete canales pasan por un filtro de un polo con constante de tiempo de unos 30 ms, y eso es lo que elimina el ruido de escalera al barrer la frecuencia de corte. El conversor entrega un valor por bloque, así que un giro del mando no llega como una rampa sino como la escalera de la Figura 6.3, y sin suavizar cada peldaño se oye como un crujido.

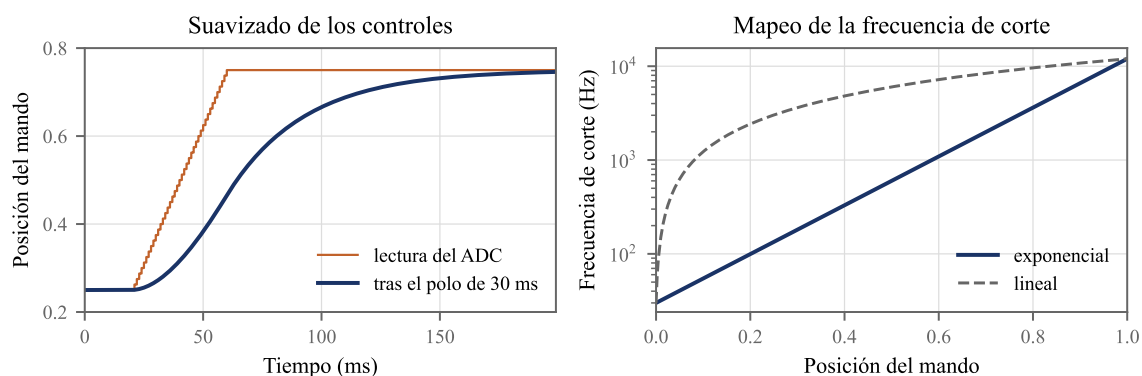


Figura 6.3: A la izquierda, un giro rápido del mando de corte tal y como lo entrega el conversor bloque a bloque, y el mismo giro tras el filtro de un polo. A la derecha, el recorrido del mando llevado a hercios con el mapeo exponencial y con uno lineal.

La frecuencia de corte se mapea en escala exponencial, no lineal, porque el oído percibe la altura de forma logarítmica, y el panel derecho de la figura compara las dos opciones. Los tres tiempos de la envoltente van en cambio en escala lineal, con la contrapartida de que el tramo de 1 a 100 ms, que es el que decide el carácter de un ataque percusivo, cae dentro del primer cinco por ciento del giro del mando. Los rangos son de 1 ms a 2 s para el ataque, de 5 ms a 2 s para el decaimiento, de 5 ms a 4 s para la liberación y de 30 Hz a 12 kHz para el corte. El sostenimiento es un nivel y no un tiempo, así que también se mapea lineal. El techo de 12 kHz no es arbitrario, ya que el filtro empleado exige que su frecuencia de corte quede por debajo de un tercio de la frecuencia de muestreo [22].

El filtro es un SVF de variables de estado con doble muestreo [22], colocado entre el oscilador y la envoltente, que es el orden clásico de la síntesis sustractiva descrito en el capítulo 2. La razón concreta de elegirlo frente a un filtro en escalera es que calcula sus cuatro salidas a la vez, de modo que ofrecer paso bajo, paso banda y paso alto por selector no cuesta ni un ciclo adicional de procesador. Basta con leer una salida u otra. Lo que sí necesita es compensación de ganancia. El pico del filtro en la frecuencia de corte vale aproximadamente la inversa de su factor de amortiguamiento, que tiende a cero conforme la resonancia sube [22], así que cerca del máximo el realce pasa de 20 dB y basta un armónico de la tabla cayendo cerca del corte para saturar la salida. La corrección es cuadrática en la resonancia y neutra cuando esta vale cero, de forma que con la resonancia al mínimo el nivel de salida sigue siendo exactamente el calibrado.

El selector de tres posiciones se lee como un canal más del conversor, con tres tensiones que fija un divisor resistivo descrito en el capítulo 7. No hace falta antirrebote aparte, porque la decisión se toma sobre el valor ya suavizado y los 30 ms del filtro de control son exactamente eso.

Aquí apareció un fallo de los que no avisan. El sentido del selector estaba invertido respecto a la serigrafía del panel, de forma que dos de las tres posiciones seleccionaban un filtro distinto del que anunciaban. Se corrigió el firmware y no la serigrafía, que no se recompila. No es un fallo que se detecte moviendo mandos a ver si algo suena, porque los tres filtros suenan. Lo que falla es que ninguno lo hace donde pone el panel. De ahí salió la regla de verificar los siete canales uno a uno después de cada grabación.

6.6 El display

El panel lleva una pantalla OLED de 128 por 64 puntos gobernada por un SSD1306 [24], conectada por I²C. La elección de pines no es libre y esconde una trampa, porque el mismo controlador de ese bus sale también por los pines que ocupa el USART1 del MIDI [19]. Tomar el mapeo por defecto habría cableado el display justo encima de la entrada de notas.

La presencia del display se comprueba en el arranque en lugar de darse por hecha. El controlador de libDaisy transmite de forma bloqueante y no devuelve error, así que una pantalla ausente o mal cableada no se manifestaría como un fallo sino como un arranque lentísimo y un bucle principal frenado en cada refresco. El sondeo envía el byte de control y la orden de apagar el display, y mira si hay reconocimiento en la dirección de esclavo del integrado [24]. Si no contesta en 10 ms, el instrumento arranca sin pantalla y sigue sonando igual.

El bus se configura a la más rápida de las tres velocidades que ofrece la biblioteca, que esta documenta como unos 886 kHz efectivos y no como el megahercio nominal [21]. La rutina de dibujo compara lo que tendría que pintar con lo que ya está pintado y sale sin tocar el bus si no ha cambiado nada visible, de modo que tocando el teclado sin mover mandos el display no consume ni un ciclo, y los 80 ms del temporizador son un límite superior al refresco y no una cadencia fija.

Un refresco sano manda el kilobyte entero de la memoria de imagen y cuesta del orden de diez milisegundos, pero el transporte de la biblioteca espera hasta un segundo por cada página, así que con el bus colgado un solo refresco congela el bucle principal durante segundos y con él la atención al MIDI y al SPI. Por eso la pantalla se apaga sola, y no se vuelve a tocar el bus en toda la sesión, cuando tres refrescos pasan de 100 milisegundos. Se prefiere perder la pantalla a perder el instrumento. Al sondeo de arranque le precede además una maniobra de desbloqueo, nueve pulsos de reloj y una condición de parada, que libera a un esclavo atascado a mitad de un byte.

Lo que muestra es el mando que se está tocando. Una línea de cabecera da su nombre y su valor en unidades reales, y debajo va una gráfica, la envolvente para los cuatro mandos del ADSR y la respuesta del filtro para el corte, la resonancia y el selector. La gráfica se dibuja con los mismos valores mapeados que usa el audio, así que la figura es la que de verdad suena. Con siete GPIO y una pantalla de este tamaño, repartirla en casillas diminutas la habría dejado ilegible justo cuando hace falta leerla, que es mientras se gira un mando. El foco lo roba el último control movido más de un 3 % de su recorrido, y cada canal lleva su propia referencia. Ese margen está puesto por encima del ruido que entrega el panel real, porque un canal cuyo cursor tiemble más que el umbral se queda con el foco de forma permanente; subirlo no cuesta resolución de ajuste, ya que solo decide a qué canal mira la pantalla y no cómo se aplica el valor.

Que esta pantalla cuelgue del procesador de audio y no de la CYD, que ya tiene una, lo decide el sentido del flujo de datos. La cadena es unidireccional y los mandos están en el extremo final, así que pintarlos en la CYD habría exigido remontar los dos enlaces al revés, incluido el SPI del apartado 5.5.

6.7 Arranque, instrumentación y recursos

Diagnosticar el primer arranque del montaje completo resultó imposible durante un buen rato por una carencia del propio firmware, que no imprimía nada al empezar. Un firmware colgado a mitad de la inicialización y uno vivo esperando entrada son indistinguibles por el puerto serie, porque los dos callan. La solución fue barata. Tres parpadeos del diodo al arrancar dan prueba de vida sin depender de un ordenador, una línea por fase de inicialización dice hasta dónde llegó el arranque, y un latido cada dos segundos lleva los contadores de cada subsistema. Ese latido cuenta los eventos MIDI de cualquier tipo, incluidos los que el firmware ignora, lo que separa dos preguntas que antes estaban mezcladas, a saber, si no llega ni un byte por el cable o si llega pero falla el tratamiento de las notas. El principio general es que un firmware que solo habla cuando todo va bien no se puede depurar.

En cuanto a recursos, el firmware ocupa el 82,7 % de los 128 kB de memoria de programa interna del microcontrolador, frente al 70,6 % que ocupaba antes de añadir el display, el 65,3 % al cerrar la cadena completa y el 50,8 % con solo el motor de tabla. La placa dispone además de 8 MB de memoria de programa externa y 64 MB de memoria dinámica [19] que este firmware no llega a usar, de modo que el límite no es de la placa sino de la región interna contra la que se enlaza.

6.8 Estado del subsistema

Con esto queda cerrado el camino que va de la trama a la señal analógica. El procesador de audio recibe la tabla por acceso directo a memoria, comprueba su integridad, la incorpora con una transición de 20 ms que no altera la fase, la recorre con un acumulador gobernado por la nota que llega por MIDI, la pasa por el filtro y la envolvente que gobiernan los mandos del panel y la entrega por el DAC de su códec, que recibe las muestras por un bus serie de audio I²S en formato de 24 bits alineado a la izquierda [19]. La salida es de nivel de línea con impedancia de 100 Ω .

Las tres placas quedan así descritas por completo. Lo que falta es el soporte físico que las sostiene y las alimenta, que es el asunto del capítulo siguiente, y las medidas que verifican todo el conjunto, que se recogen juntas en el capítulo 8.

Capítulo 7

Hardware y alimentación

Los tres capítulos anteriores describen qué hace cada placa. Este describe el soporte físico que las sostiene. De dónde sale la energía y cómo se reparte, sobre qué van montadas, cómo llegan hasta ellas los mandos que el usuario toca y qué las encierra. El reparto de tareas entre placas está en el capítulo 3, la lectura de los canales del panel en el apartado 6.5 y el tratamiento de las notas en el apartado 6.4.

7.1 Componentes y equipos

La Tabla 7.1 recoge los componentes del instrumento. No hay ninguna pieza fabricada a medida salvo las cuatro piezas de la carcasa, en línea con el condicionante de partida del apartado 3.1. El banco de trabajo se reduce a una impresora de filamento con cama de 220×220 mm, una fuente de laboratorio, un soldador y un multímetro. Los importes se recogen en el anexo de presupuesto.

El teclado que aporta las notas no figura en la tabla. Por la decisión del apartado 3.3 es un aparato aparte, con su propia alimentación, y el instrumento funciona con cualquier controlador MIDI.

Tabla 7.1: Componentes del instrumento.

Bloque	Componente	Cant.
Módulos	Daisy Seed 1.2	1
	CYD ESP32-2432S028R, pantalla táctil de 2,8 pulgadas	1
	ESP32-S3 N16R8	1
	Shield MIDI con optoacoplador 6N138	1
Alimentación	Alimentador externo de 230 V a 5 V / 3 A	1
	Conector de alimentación de 2,1 mm, de chasis	1
	Interruptor de encendido ON-OFF-ON	1
	Condensador electrolítico de 470 μ F, reserva del rail	1
	Condensador electrolítico de 100 μ F, filtro de audio	2
	Condensador cerámico de 100 nF	8
	Resistencia fusible de 3,3 Ω y 1 W	1
	Diodo Schottky 1N5817	1
Panel	Potenciómetro lineal de 10 k Ω	6
	Conmutador ON-OFF-ON, selector de filtro	1
	Resistencia de 10 k Ω , divisor del selector	2
	Pantalla OLED de 0,96 pulgadas, SSD1306	1
	Jack de audio de 3,5 mm mono, cuerpo de plástico	1
Soporte	Placa perforada de tiras, 100 $\times$ 160 mm, baquelita	1
	Tira de pines hembra de 2,54 mm, 20 contactos	2
	Tira de pines hembra de 2,54 mm, 22 contactos	2
	Conector JST de 1,25 mm, enlace con la CYD	2
	Cable de silicona de 24 AWG	—
	Filamento PLA y tornillería M3 con tuerca	—

7.2 La alimentación

Las cuatro cargas cuelgan en paralelo de un único rail de 5 V, que es la tensión de entrada de los tres módulos y del shield, y cada placa regula internamente sus propios 3,3 V. La Figura 7.1 recoge el esquema completo.

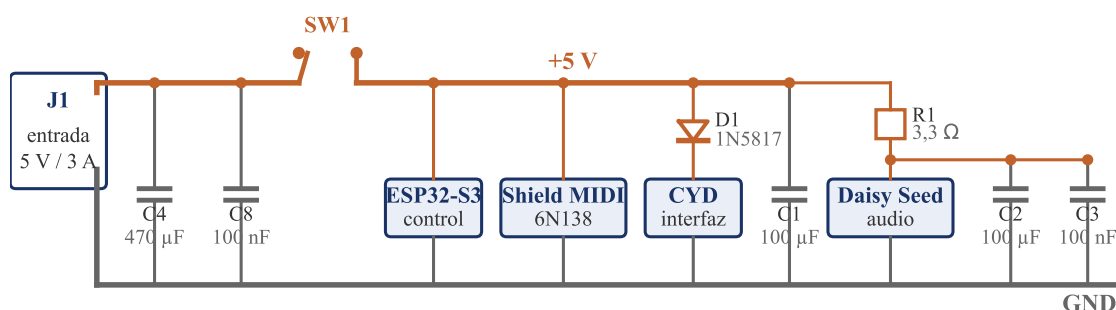


Figura 7.1: Esquema de la alimentación. El condensador de reserva C4 queda aguas arriba del interruptor y la rama del procesador de audio es la única con elemento en serie.

El consumo del conjunto se estimó en unos 600 mA, de modo que un alimentador de 3 A deja

margen de sobra. Va fuera de la carcasa, así que dentro no hay tensión de red en ningún punto. Alimentar todo desde un solo raíl tiene además un efecto que el UART, el SPI y el MIDI necesitan: las masas de las tres placas quedan unidas.

El interruptor y el condensador de reserva

El raíl no se alimenta directamente desde el conector de entrada. El electrolítico de 470 μF (C4) queda aguas arriba y el interruptor aguas abajo, de manera que al cerrar, con el condensador ya cargado, las placas ven un flanco rápido en lugar de la rampa lenta con la que sube el alimentador. La razón es que durante esa rampa las placas pasan un tiempo largo en la zona en la que su circuito de reinicio no sabe decidir, y arrancan de forma intermitente. Con el flanco rápido y el condensador aportando el pico de corriente que piden al arrancar el problema desaparece. Para que esto sea posible el raíl de 5 V deja de ser una tira continua y se corta en un punto, aguas arriba de todas las cargas, es el interruptor de encendido el que decide cuando se cierra el circuito y alimenta las cargas. El raíl de masa, en cambio, no se interrumpe nunca. Cortarlo obligaría a las corrientes de retorno a buscar camino por los cables de datos.

Los filtros

El raíl lo comparten la pantalla y dos microcontroladores conmutando. El ruido que generan puede acabar en la sección analógica del procesador de audio y salir por el jack. Los condensadores de la Figura 7.1 lo atacan aprovechando que la impedancia de un condensador es un circuito abierto en continua y tiende a un cortocircuito conforme sube la frecuencia. Este comportamiento se describe con la ecuación (7.1), donde j es la unidad imaginaria, ω la frecuencia angular y C la capacidad del condensador.

$$Z_C = \frac{1}{j\omega C}, \quad (7.1)$$

Puesto en paralelo con la alimentación, un condensador deriva a masa la componente de alta frecuencia sin tocar la continua que alimenta las placas. Por eso el bulk son dos en paralelo y no uno: el electrolítico de 470 μF aporta la capacidad que sostiene los transitorios lentos y el cerámico de 100 nF es el que es efectivo en la parte alta del espectro.

La rama del procesador de audio lleva además un elemento en serie. La resistencia R1 de 3,3 Ω junto con los 100 μF de C2 forman un paso bajo de primer orden cuya frecuencia de corte es la ecuación (7.2), que es la frecuencia a la que la señal se atenúa 3 dB.

$$f_c = \frac{1}{2\pi RC} \approx 482 \text{ Hz}, \quad (7.2)$$

De este modo, el ruido de conmutación llega atenuado a la entrada de alimentación de la placa. El cerámico C3 cubre la parte del espectro que el electrolítico ya no ve, y va soldado entre las patillas de entrada y de masa del módulo, que son adyacentes. Ese elemento en serie no puede ir en el raíl común porque su caída la pagarían también las demás cargas. Los parámetros del optoacoplador del shield están especificados con una alimentación de 4,5 V [25], que es la más baja a la que el fabricante garantiza algo, así que sobre un raíl de 5 V el margen es estrecho para poner nada

en serie. La resistencia elegida es del tipo fusible, es decir, está diseñada para abrirse ante una sobrecarga. Conviene saberlo porque si se abre, la placa de audio deja de recibir alimentación y el instrumento queda mudo.

El diodo de la rama de la pantalla táctil

La CYD se reprograma por su propio conector USB, así que en algún momento va a estar alimentada por las dos fuentes a la vez. El diodo Schottky D1 en su rama resuelve el conflicto. La placa toma la más alta de las dos fuentes, así la caída del Schottky sería suficiente para que la USB predomine. Se elige Schottky por su caída directa baja, del orden de tres décimas de voltio, que sobre un raíl de 5 V es asumible. Los otros dos módulos llevan ese diodo integrado y no lo necesitan.

Las masas

El fabricante del procesador de audio exige unir la masa analógica con la digital en toda aplicación [19], y esa unión se hace en un único punto y cerca de la placa. Conviene precisar qué significa eso, porque unidas en un punto las dos masas son el mismo nodo y a corriente continua están al mismo potencial. Lo que las distingue no es el nodo sino el camino: el cobre que separa cualquier punto de masa del punto de unión tiene resistencia, y por él circula la corriente de retorno de lo que haya aguas arriba. Medido sobre la placa, entre una y otra masa hay unos 1,7 mV.

De ahí salen las dos decisiones de trazado. El manguito del jack se lleva a la patilla de masa analógica y no al raíl general, para que el retorno de la salida de audio no comparta cobre con el de los dos microcontroladores. Los potenciómetros, en cambio, se referencian a la masa digital, que quedaba mucho más a mano en el trazado, porque 1,7 mV sobre un fondo de escala de 3,3 V son un 0,05 % y no mueven una lectura de mando. La patilla superior sigue colgando de la alimentación analógica, así que la cancelación que se explicó en el apartado 6.5 queda intacta.

7.3 Diseño del circuito y trazado de la placa

El montaje definitivo va sobre una placa perforada de tiras de cobre de 100×160 mm, que a paso de 2,54 mm son 39 tiras de 63 agujeros cada una. El circuito se dibujó en Fritzing sobre su vista de placa de tiras, que es a la vez esquema y plano de montaje. En una placa de este tipo la posición de cada componente forma parte del circuito. El resultado es la Figura 7.2. Sobre ese fichero se verificó el trazado decodificándolo hueco a hueco.

Los cortes de pista

Un módulo con dos filas de patillas, colocado perpendicular a las tiras, deja enfrentadas la patilla p y la $41 - p$ sobre la misma tira, y por tanto en cortocircuito. Colocarlos paralelo no es la solución, porque entonces las veinte patillas de una fila caen sobre la misma tira. Hay que cortar, por tanto, las tiras que pasan bajo las placas. Hay que llevar especial cuidado con cinco de esas uniones ya que son destructivas si se olvidan. La peor pone la alimentación analógica directamente contra la masa analógica, otra mete los 5 V de la entrada de alimentación en una GPIO y una tercera deja el cursor de un potenciómetro sobre la salida de audio.

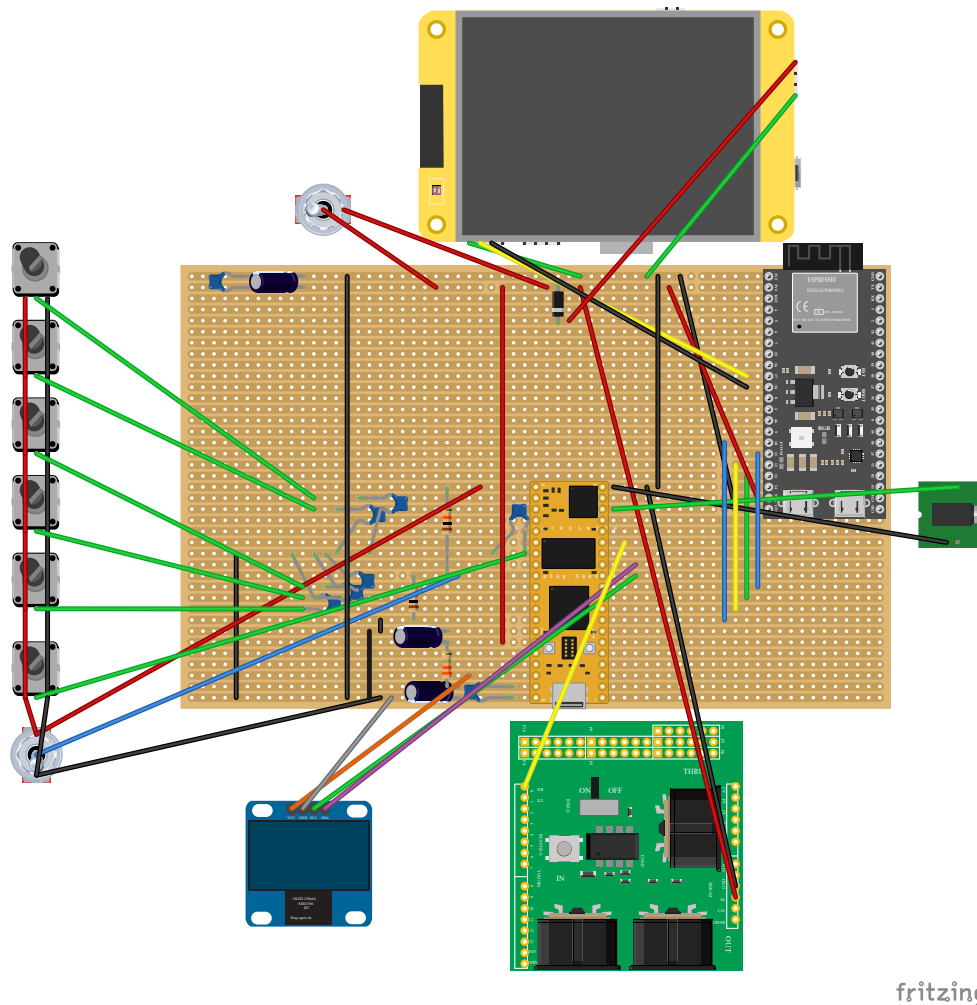


Figura 7.2: Trazado de la placa. Los dos módulos van sobre zócalos, la alimentación ocupa el extremo libre de los raíles y todo lo que el usuario toca sube al panel por el mazo.

Tras hacer los agujeros se comprueban con doble medida. Una a lo largo de la tira cruzando el agujero, que tiene que quedar en abierto, y otra contra las dos tiras vecinas, por si la broca ha raspado el borde. De hecho, gracias a la segunda medida se cazó una rebaba de cobre que había creado un cortocircuito.

Reparto de la placa

Para la disposición en la placa se tomaron unos acuerdos. La tira 1 es el raíl de masa y la tira 2 el de 5 V. El reparto a lo largo de los 63 agujeros se hizo por naturaleza de la señal. El enlace SPI corre a 10 MHz, por lo que es la señal más agresiva de la placa y sus flancos se acoplan con facilidad a lo que tenga cerca. Por este motivo su recorrido tiene que ser corto y no pasar cerca del jack ni de los cursores. Mientras que la salida de audio y la masa analógica se van al borde opuesto, donde además caen las patillas de los potenciómetros. Así se acaba creando una zona digital, una mixta y una analógica, y la sección de alimentación en el extremo libre de los raíles, que es donde la obliga a estar el corte del interruptor.

Conviene subrayar que esta placa no es «el circuito» sino un «concentrador». Los dos módulos sobre zócalos, la alimentación con su filtro y una fila de conexiones. Nada de lo que el usuario toca o mira va soldado a ella. Todo eso vive en el panel y baja por cables a la placa, como en un pedal de guitarra.

El trazado se calcó sobre la cara de cobre en lugar de sobre la de baquelita, y como la vista de la herramienta es la cara de componentes, la placa salió en espejo respecto al diseño. Con los cortes ya taladrados se optó por adaptarse a la placa y volver a trazar esta vez contando sobre la baquelita. Por este motivo, en la placa se pueden visualizar cortes de pista que no son necesarios y otros que sí lo son. La Figura 7.2 es la versión final. El cambio, además, obligó a reasignar tres cursores y el selector de filtro a otras patillas.

7.4 El panel de control

La Figura 7.3 es el alzado del panel, dibujado a escala a partir del mismo fichero paramétrico del que salió la pieza impresa.

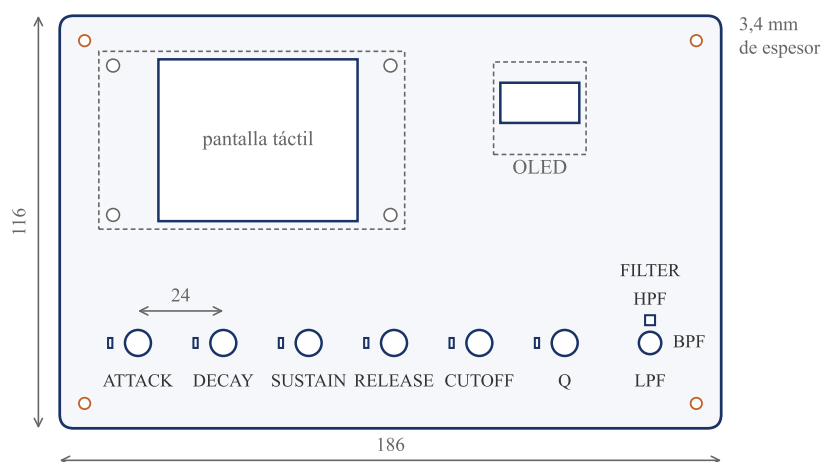


Figura 7.3: Alzado del panel frontal. El trazo discontinuo es el contorno de las dos placas, que quedan por detrás; el continuo, lo que se ve desde fuera. Cotas en milímetros.

Los seis potenciómetros son lineales de 10 k Ω y cuelgan de la alimentación analógica del procesador de audio, que es la disposición que el fabricante recoge como aplicación típica [19]. El valor hace que los seis sumen unos 2 mA de consumo sobre el raíl analógico. Mientras que un valor menor lo cargaría sin motivo, valores mayores harían la lectura más sensible al ruido captado por el cable. Cada cursor lleva un cerámico de 100 nF a la masa, soldado en el extremo de la placa para poder filtrar lo que ha recogido el hilo. Las dos resistencias de 10 k Ω del divisor del selector, cuya función se explicó en el apartado 6.5, van también en la placa, así que al panel sólo llega un hilo de cada potenciómetro y uno del selector. La alimentación 3V3 analógica y la masa van en daisy chain, desde el selector hasta el último potenciómetro, y de ahí a la placa.

Para el grosor del panel se tomó en cuenta la rosca del potenciómetro, que mide 7,0 mm, de la que hay que descontar arandela y tuerca. De esta manera, el máximo que podía medir el panel de grosor eran 4,1 mm pero se optó por 3,4 mm porque era más que suficiente. Se decidió también

añadir el hueco para la patilla antigiro de cada potenciómetro, que no es redonda sino una pletina rectangular de 2,8 por 1,2 mm, y que se corta pasante como el propio agujero de la rosca. Es lo que impide que el cuerpo del potenciómetro gire dentro del panel al mover el mando, con la tuerca limitándose a apretarlo contra la plancha.

Una parte complicada fueron las ventanas para las pantallas montadas en el panel. La ventana de la CYD lleva el labio izquierdo cuatro milímetros más adentro que el derecho para tapar un borde negro que asomaba. El labio, que apoya sobre el cristal en todo el perímetro, fue una fuente de pulsación fantasma como se comentó en el apartado 5.1. Además, la pantalla va girada media vuelta por un motivo físico, que su conector USB quede en el lado opuesto a la OLED. Esto hizo que se tuviera que rehacer parte de su firmware y recalibrarla entera, está descrito en el apartado 5.1. La OLED se resolvió en tres niveles. Uno para el asiento de la placa, otro para el hueco del cristal y un labio para acotar la parte visible de la pantalla con un canal para que su cable plano no quede pinzado.

Para las etiquetas, se decidió que fuesen grabadas con un milímetro de profundidad y que su tamaño se ajustase solo según el número de caracteres, para que las largas no invadan a sus vecinas.

Fuera del panel quedan el jack, los conectores MIDI, la entrada de alimentación y el interruptor de encendido, que se llevaron a las paredes de la caja.

7.5 La carcasa

La carcasa no es una pieza de forma arbitraria, es un conjunto de agujeros a cotas exactas y cada cota depende de una pieza física que hay que medir. Para solucionar esto, se modeló con una herramienta de diseño paramétrico por texto, en la que corregir una cota es cambiar un número, en lugar de con un modelador orgánico, donde habría que rehacer la geometría. El método de trabajo fue ir imprimiendo probetas que comprueban si las cotas son correctas poco a poco. La probeta debía laminarse con el mismo perfil que la pieza definitiva, ya que el diámetro real de un agujero impreso depende de factores de la impresión como el ancho de línea, el número de perímetros o de la velocidad de impresión. Por eso la probeta se implementó dentro del propio fichero de la pieza, seleccionable con un parámetro, de modo que comparte parámetros con ella. Las probetas ayudan a depurar el modelo y a calibrar la impresora reduciendo el tiempo de prueba y error y el desperdicio de filamento. La Figura 7.4 muestra la carcasa montada y despiece.

El cuerpo es una cuña de 44 mm de alto por delante y 62 por detrás, lo que deja la cara inclinada a algo menos de nueve grados. Tiene una bahía trasera plana que lleva la planta a 186 por 186,6 mm, y está partido en tres piezas, como muestra la Figura 7.4. Cada pieza se imprime en la orientación en la que su geometría queda vertical o apoyada para minimizar el uso de soportes y para ser más preciso con los contornos críticos. Por ejemplo, al imprimir la trasera tumbada sobre su cara exterior, los huecos de los conectores MIDI se convierten en agujeros verticales pasantes. Así desaparecen a la vez los voladizos y los puentes de impresión que habrían empeorado la precisión de la pieza.

Todas las uniones son por tornillo contra tuerca metálica embutida y ninguna con rosca en el plástico. El material del que está impreso, PLA, no aguenta bien las fuerzas de rosca y puede deshacerse con el tiempo. Además, para todos los agujeros se crearon parámetros de holgura editables. Estos valores de holgura vendrán dados por lo comentado anteriormente, probetas y configuraciones de la impresora. De hecho, los agujeros impresos salen entre 0,2 y 0,35 mm menores que la cota real

puesta en el modelo, así que, por ejemplo, los pasos de tornillo de 3 mm van dibujados entre 3,4 y 3,8.

La Figura 7.5 muestra el instrumento terminado, ya montado y en funcionamiento.

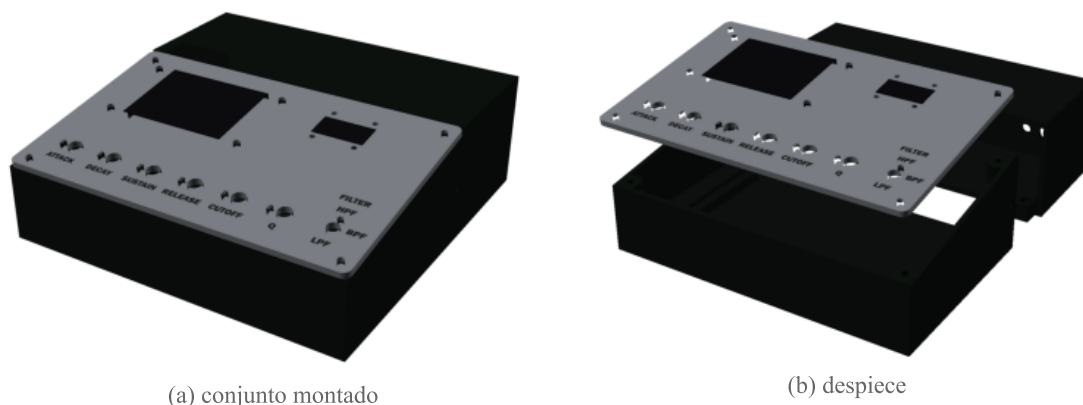


Figura 7.4: La carcasa, renderizada desde el modelo paramétrico. A la izquierda el conjunto montado; a la derecha el despiece, con el panel separado de la bandeja y las dos piezas de la trasera.



Figura 7.5: El instrumento terminado. A la izquierda, el panel de control con los seis mandos, el selector de tipo de filtro y los huecos de las dos pantallas. A la derecha, en funcionamiento: el dedo fija el punto del espacio latente, la pantalla táctil dibuja la forma de onda que se genera en ese punto y el display de servicio muestra el parámetro que se está editando.

Con la caja montada, el instrumento queda descrito por completo, del conjunto de datos al plástico. Queda pendiente la comprobación eléctrica sobre la placa ya soldada, que debe cerrar la cuestión de la tensión de entrada del procesador de audio, y queda comprobar que el conjunto hace lo que se dice que hace, que es el asunto del capítulo siguiente.

Capítulo 8

Pruebas y resultados

Este capítulo reúne las comprobaciones que verifican el instrumento terminado. Son de dos clases. Las primeras son validaciones de la cadena de datos, hechas con guiones que comparan lo que produce cada placa contra una referencia calculada aparte, y son las que los capítulos 5 y 6 dejaron apuntadas aquí. Las segundas son medidas eléctricas y de audio sobre el aparato montado, soldado y cerrado, y entre ellas está la que cierra la cuestión que el apartado 7.2 dejó abierta, la tensión de entrada del procesador de audio. El capítulo termina con las incidencias que la verificación destapó.

Conviene acotar antes lo que estas medidas pueden decir, porque todas salen de un único aparato, un multímetro Richmeters RM113D de valor eficaz verdadero y 6000 cuentas, con banda de medida en alterna de 40 Hz a 1 kHz. No hubo osciloscopio ni analizador de audio. De ahí salen tres límites. En el rango de 600 Ω el aparato resuelve 0,1 Ω , así que toda resistencia por debajo de un ohmio cae en su suelo y su lectura es un límite y no un valor. Las puntas cortocircuitadas dan 1,3 Ω a la primera y 0,7 Ω apretando con firmeza, diferencia que es presión de contacto y no propiedad del cable. Se adopta el valor más bajo como referencia y todas las resistencias se dan en bruto y en neto. El rizado de una fuente conmutada vive en centenares de kilohercios, muy por encima de la banda que el aparato integra, de modo que toda lectura de alterna sobre un raíl es una cota superior de la componente de baja frecuencia y no una medida del ruido total.

8.1 Validación de la cadena de datos

Las tres comprobaciones de esta sección tienen la misma forma. Un guion en el ordenador recalcula en doble precisión lo que debería salir, la placa vuelca por USB lo que de verdad ha calculado y se comparan término a término. No son pruebas de que el instrumento haga algo, sino de que hace exactamente lo que dice la referencia. El método de cada una está en los apartados 5.3, 5.5 y 6.4. Los resultados están en la Tabla 8.1.

La interpolación bilineal del ESP32-S3 se comparó en cuatro coordenadas elegidas por su geometría, una junto al origen, el centro exacto de la rejilla, una esquina y un punto asimétrico que obliga a mapear los dos ejes de forma independiente. En las cuatro el error máximo es de 1 LSB en Q15 sobre las 1024 muestras, y en la más desfavorable solo 2 de esas 1024 muestras se desvían, con un error medio de 0,002. Es el límite de redondeo que impone el paso de coma flotante a entero de 16

bits y no un defecto de la implementación, de modo que la aritmética del ESP32-S3 y la de numpy son equivalentes.

El transporte por SPI se validó con una coordenada fija escrita por USB, por la razón de método que expone el apartado 5.5. Para la coordenada (40000, 12000) el procesador de audio recibe las 1024 muestras con 0 LSB de error, es decir, la tabla que sale del interpolador y la que entra en el motor de síntesis son la misma bit a bit. El CRC de la trama cumple entonces el papel de detección para el que se puso y no ha tenido que descartar ninguna en esta prueba.

La conversión de nota a frecuencia se comprobó sobre las 128 notas MIDI. El firmware vuelca su propia tabla, calculada con la misma función que gobierna el oscilador, y el guion la contrasta contra la conversión en doble precisión y contra el incremento de fase deducido de la frecuencia de muestreo real de la placa. Ninguna de las 128 queda fuera de tolerancia. Las tolerancias se fijaron en 5 milihercios y 10 partes por millón. Esa tabla se validó sobre el firmware en mesa. Sobre el montaje definitivo la comprobación se repitió en vivo para el *la* central, que el volcado da en 440 Hz exactos con resolución de milihercios.

Tabla 8.1: Validación de la cadena de datos frente a una referencia calculada en el ordenador.

Etapas	Referencia	Error
Interpolación bilineal, 4 coordenadas	numpy sobre la rejilla	≤ 1 LSB
Transporte SPI de la tabla	la propia bilineal	0 LSB
Conversión nota a frecuencia, 128 notas	mtof en doble precisión	0 fuera de tolerancia

8.2 Continuidad y resistencia

Estas medidas se hicieron con el instrumento sin alimentar, que es la única condición en la que un óhmetro funciona. La Tabla 8.2 las recoge.

Tabla 8.2: Continuidad y resistencia, con el instrumento sin alimentar. El valor neto descuenta los $0,7 \Omega$ de las puntas. OL indica fuera de escala, es decir, del orden de los megaohmios.

Punto de medida	Bruto	Neto
Puntas cortocircuitadas (referencia)	1,3 / $0,7 \Omega$	—
Resistencia serie de la rama de audio	$3,6 \Omega$	$2,9 \Omega$
Raíl de 5 V contra masa	OL	—
Masa común entre las tres placas	$1,5 \Omega$	$0,8 \Omega$
Aislamiento de los cortes de tira	OL	—
Interruptor abierto	OL	—
Interruptor cerrado	$1,5 \Omega$	$0,8 \Omega$

La resistencia de la rama de audio es la única lectura contrastable contra un valor nominal, y da $2,9 \Omega$ netos frente a los $3,3 \Omega$ del componente. Está medida en circuito, con el resto del montaje en paralelo, y la incertidumbre de las puntas más la resolución del rango cubre la diferencia.

Lo importante de la tabla es lo que no aparece. El raíl de 5 V contra masa da fuera de escala, así que no hay cortocircuito, que era la comprobación que más preocupaba. Los cortes de tira del

apartado 7.3 aíslan todos, medidos a lo largo de la tira y contra las vecinas. La masa es común a las tres placas, condición de la que dependen los tres enlaces de datos del instrumento.

Entre los dos extremos del interruptor hay 34 mV con 0,42 A circulando, lo que sitúa la resistencia del interruptor y su cableado en unos 80 m Ω , un orden de magnitud por debajo de lo que el óhmetro resuelve. Por debajo de un ohmio la resistencia se deduce de la caída de tensión con corriente conocida y no se lee en el óhmetro.

8.3 Tensiones de alimentación y rizado

Todos los nodos de alimentación se midieron dos veces, primero con el instrumento encendido y en reposo y después en el peor caso que admite, con la pantalla táctil encendida, un dedo deslizándose sobre ella y una nota sostenida sonando. La Tabla 8.3 recoge ambas columnas.

Tabla 8.3: Tensiones continuas referidas a masa, en reposo y en el peor caso. El guion indica un nodo que no se repitió en la segunda pasada.

Nodo	Reposo	Peor caso
Raíl aguas arriba del interruptor, en vacío	4,928 V	—
Raíl aguas abajo, junto a la CYD	4,894 V	4,910 V
Shield MIDI	mismo nodo que el anterior	
Pin de 5 V del procesador de control	4,891 V	—
Entrada de la CYD tras el Schottky	4,557 V	4,576 V
Entrada del procesador de audio (pin 39)	4,354 V	4,475 V
Caída en la resistencia de 3,3 Ω	0,543 V	0,432 V
Raíl digital de 3,3 V (pin 38)	3,256 V	—
Raíl analógico de 3,3 V (pin 21)	3,232 V	—
Cursor de potenciómetro, tope superior	3,227 V	—
Cursor de potenciómetro, tope inferior	0,004 V	—

La segunda columna no es sistemáticamente menor que la primera, que es lo que cabría esperar al cargar el raíl, y eso no describe ninguna física razonable. La explicación está en la medida. Repitiendo el mismo punto sin tocar nada, la entrada del procesador de audio pasó de 4,354 a 4,445 V y la caída en la resistencia de 0,543 a 0,482 V, con la lectura visiblemente inestable. Lo que se mide no es un punto de trabajo estable sino una magnitud que varía con lo que el firmware esté haciendo. La única conclusión sólida que admiten estos números es la negativa, y es que el raíl no se hunde al cargarlo.

La lectura que de verdad importa cierra la cuestión que dejó pendiente el apartado 7.2. Sobre la placa de prototipado la entrada del procesador de audio rondaba los 3,4 V, aunque esa cifra nunca fue de fiar, porque la resistencia del camino fluctuaba al repetir la medida sin haber tocado nada. Esa inestabilidad era ya el indicio de que buena parte de la pérdida la ponían los contactos y no el filtro. Al soldar y mejorar los contactos esto se confirmó. Entre el raíl aguas abajo del interruptor, 4,894 V, y la entrada del procesador de audio, 4,354 V, hay 0,540 V, y la caída medida directamente sobre la resistencia del filtro es de 0,543 V. Las dos cifras coinciden dentro de tres milivoltios, de modo que ya no queda pérdida parásita en el camino y todo lo que se pierde se pierde en el componente que se puso para ello.

Eso no hace desaparecer el problema, solo lo explica. La hoja del procesador de audio especifica una tensión de entrada de 5 a 17 V [19] y el montaje le entrega 4,354 V en reposo, medio voltio por debajo del mínimo garantizado. El instrumento funciona en esas condiciones, con los raíles internos de 3,3 V en 3,256 y 3,232 V, pero funciona fuera de especificación, y conviene dejarlo escrito así y no como una tolerancia verificada. La corrección natural, que no cabía en la ventana de tiempo disponible, es rebajar la resistencia serie del filtro y compensar la pérdida de filtrado con capacidad, o alimentar esa rama antes del corte del raíl.

Los dos últimos pares de filas confirman lo que necesitaba el capítulo 6. El raíl analógico está a 3,232 V y el cursor de un potenciómetro llevado a sus topes da 3,227 y 0,004 V. El firmware ve el recorrido completo del conversor sin zonas muertas y con la referencia del conversor y el fondo de escala del divisor colgando de la misma tensión, que es la cancelación de deriva que buscaba el apartado 6.5.

El rizado se midió en alterna sobre el raíl de 5 V y la entrada del procesador de audio, y en ambos la lectura es de 0 mV. Eso no significa que no haya ruido, sino que el rizado dentro de la banda que el aparato integra está por debajo de su suelo de medida. La cota sigue siendo útil porque descarta el zumbido de red acoplado a la salida. Verificar el filtro de la rama de audio, cuyo codo se situó en 482 Hz en el apartado 7.2, exigiría un osciloscopio y queda sin hacer.

8.4 Consumo

El consumo del conjunto se recoge en la Tabla 8.4.

Tabla 8.4: Consumo del instrumento.

Medida	Valor
Conjunto, amperímetro en serie	0,42 A
Procesador de audio, deducido de la caída	130–165 mA
Pico de arranque	no capturable

La medida se realizó puenteando con el amperímetro el interruptor abierto, y el conjunto completo consume 0,42 A frente a los 600 mA que estimaba el apartado 7.2, así que el alimentador de 3 A queda con más margen del previsto.

El consumo del procesador de audio no se midió directamente sino que se dedujo, aprovechando que la resistencia serie de su rama hace de derivación. Con las caídas de la Tabla 8.3, de 0,432 a 0,543 V sobre 3,3 Ω nominales, salen de 130 a 165 mA. El rango no es incertidumbre de medida sino consumo real, porque la placa consume distinto según lo que el firmware esté haciendo en cada momento. La deducción usa además el valor nominal de la resistencia. Con los 2,9 Ω medidos las cifras subirían alrededor de un catorce por ciento, que es hasta dónde llega la precisión de este método.

El pico de corriente en el arranque, que es el que justifica el condensador de reserva, no se pudo medir. Es un transitorio de milisegundos y un multímetro promedia. Queda sin verificar, aunque el arranque sí se comprobó por su efecto, con diez encendidos de diez correctos.

8.5 La salida de audio

Las medidas de audio se tomaron en el jack, sin carga conectada, con el multímetro en alterna. La Tabla 8.5 las recoge.

Tabla 8.5: Salida de audio en el jack sin carga. Los valores en dBV se refieren a un voltio eficaz.

Magnitud	Valor	Equivalente
Nivel de salida, pulsación suave	85 mV eficaces	—
Nivel de salida, velocidad máxima	198,7 mV eficaces	−14 dBV
Componente continua en la salida	1 mV	—
Suelo de ruido, alterna	4,3 mV eficaces	−47,3 dBV
Suelo de ruido, continua	0,66 mV	—
Suelo del multímetro, puntas en corto	0 mV	—
Relación señal-ruido	33 dB	—

Las dos primeras filas son el mismo resultado con dos estímulos distintos, porque el nivel de salida depende de la velocidad de pulsación. Los 85 mV corresponden a una pulsación suave y no caracterizan al instrumento; la cifra que lo caracteriza es la de velocidad máxima, 198,7 mV eficaces, es decir −14 dBV. Conviene decir lo que esa cifra no es. El códec entrega 1 V eficaz a fondo de escala [19] y la ganancia global del firmware es de 0,8, así que una tabla normalizada a pico podría llegar a unos 0,8 V eficaces, y la medida queda unos 12 dB por debajo. Parte de esa diferencia la pone el propio aparato, que solo integra de 40 Hz a 1 kHz y deja fuera los armónicos de una onda rica, y parte el punto de trabajo, porque el nivel depende del sostenimiento, del corte y del tipo de filtro, y esos ajustes no se anotaron durante la medida. Es, por tanto, un nivel de línea de consumo en las condiciones en que se midió, del orden de lo que entrega un reproductor doméstico, así que el instrumento se conecta a una entrada de línea sin adaptación y sin riesgo de saturarla. La componente continua es de un milivoltio, despreciable, y confirma que la salida no arrastra ningún desequilibrio capaz de desplazar un altavoz de su posición de reposo.

El suelo de ruido merece un apunte por el modo en que se comprobó. Con el instrumento encendido y sin tocar ninguna nota, el jack entrega 4,3 mV eficaces en alterna y 0,66 mV en continua. Cortocircuitando las puntas y volviendo a leer se obtienen 0 mV, así que el multímetro no aporta nada al resultado y esos 4,3 mV salen del instrumento.

De ahí sale una relación señal-ruido de 33 dB. Del ruido solo se contabiliza la porción que cae en la banda del aparato, mientras que lo que se sospecha dominante, el acoplamiento de la conmutación de las tres placas y de la pantalla, vive muy por encima de ella. Lo honesto es decir que el instrumento tiene al menos 33 dB entre 40 Hz y 1 kHz, y que la cifra de ancho de banda completo es peor y no se ha medido. Quedan sin medir la distorsión armónica y la respuesta en frecuencia, que están fuera del alcance de un multímetro.

8.6 Incidencias detectadas

La verificación destapó tres incidencias. Dos son fallos del aparato y la tercera es una consecuencia de diseño que solo se manifiesta con el panel real montado. Ninguna de las tres ha podido cerrarse. Se recogen aquí juntas, con el mismo esquema de síntoma, comprobaciones y estado, en lugar de

repartirlas por las secciones de medida, para que quede en un solo sitio lo que el instrumento tiene pendiente.

Los bloqueos del procesador de audio

Tocando notas y moviendo mandos, el instrumento se congela: deja de responder al teclado y al panel y no revive apagando y volviendo a encender de inmediato, aunque sí tras dejarlo un rato largo sin tensión. Ese detalle, que no baste con el ciclo de encendido y haya que desconectarlo un tiempo, es lo único que distingue el fallo de un cuelgue corriente del firmware.

Con un multímetro se puede comprobar lo evidente, y eso es lo que se hizo. Con el instrumento colgado, la entrada del procesador de audio estaba a 4,5 V y su raíl digital a 3,255 V, la misma alimentación que en marcha, así que no se había quedado sin tensión; en esas mismas condiciones el arranque salió bien diez veces de diez. La placa no consumía más que funcionando y el encapsulado estaba apenas tibio, de modo que tampoco había nada conduciendo de más. Entre cada raíl de 3,3 V y su masa hay 1,6 k Ω con la placa sin alimentar, que sobre 3,3 V son dos miliamperios y no un cortocircuito, y la placa se llegó a sacar del zócalo para medirla por separado con el mismo resultado. La entrada de notas quedó descartada sin medir nada, porque el fabricante declara tolerante a 5 V la patilla de recepción del USART1 [19].

Lo único anómalo apareció al mirar el bus de la pantalla durante un bloqueo, con la línea de reloj a 0 V y la de datos a 0,8 V, es decir, el bus caído. Eso encaja con el mecanismo que describe el apartado 6.6, donde un refresco sobre un bus colgado congela el bucle principal durante segundos y con él la atención al MIDI y al SPI, y es la razón de que el firmware lleve dos medidas contra ello, el vigilante que apaga la pantalla tras tres refrescos anormalmente largos y la maniobra de desbloqueo del arranque. Ninguna de las dos ha bastado y el instrumento se sigue colgando.

Hay además dos observaciones que no encajan con esa explicación. En un bloqueo distinto las dos líneas del bus estaban a 3,246 V, limpias y en reposo, con la alimentación correcta, y aun así el firmware no arrancaba. Y en otra ocasión el raíl digital de 3,3 V se fue a 0 V con la tensión de entrada presente, es decir, con el regulador alimentado y sin salida, situación que el botón de reinicio no levanta porque actúa sobre el microcontrolador y no sobre el regulador, y de la que el instrumento salió solo tras quedarse un rato desconectado. Con un multímetro no se llega más lejos, ya que ver qué ocurre en el instante del cuelgue exigiría un osciloscopio. La causa no está encontrada y la incidencia queda abierta.

El display

La pantalla OLED del panel no llegó a funcionar de forma continuada en el montaje definitivo. El firmware la trata como opcional gracias al sondeo de arranque del apartado 6.6, así que el instrumento suena igual sin ella y lo que se pierde es la realimentación visual de los mandos.

Lo comprobable con el multímetro está comprobado y sale bien. Los cuatro hilos dan continuidad, incluida la masa. Los dos resistores de polarización están puestos y miden 9,85 k Ω desde cada línea del bus al raíl digital. Sin alimentar, cada línea da 8 k Ω contra la masa digital, lejos de un cortocircuito. Y la alimentación que llega al módulo son 3,3 V correctos. Con todo eso en orden, las dos líneas del bus se quedan en 0 V. Que puedan subir está visto, porque en las pocas ocasiones en que el bus apareció sano se midieron 3,246 V en ambas, de modo que algo las sujeta abajo la

mayor parte del tiempo.

Sustituir el módulo por uno nuevo dio exactamente el mismo comportamiento, y dos módulos distintos no fallan igual por azar, así que la avería no está en el display. La maniobra de desbloqueo del apartado 6.6 tampoco lo resuelve. Descartados el cable, los resistores y el propio módulo, lo que queda al otro lado del bus es el procesador de audio, pero confirmarlo exigía instrumentación de la que no se disponía. Agotada la ventana de acceso al hardware, la incidencia queda caracterizada y no reparada, y lo que se sabe es dónde no está el fallo, no dónde está.

El foco del display

El cursor del potenciómetro de sostenimiento tiembla en torno a un dos por ciento de su recorrido, bastante más de lo que se supuso al fijar el umbral con el que un mando roba el foco de la pantalla. El camino analógico lo explica sin dificultad, ya que se trata de un cursor de diez kiloohmios cuya señal recorre un mazo de hilos hasta la placa, y el filtrado del apartado 7.4 no basta para ese canal en concreto. Con un umbral por debajo de ese temblor el sostenimiento se queda con el foco de forma permanente y ningún otro mando llega a mostrarse.

El umbral del apartado 6.6 está fijado en el tres por ciento por esta razón, por encima del ruido que entrega el panel real, y subirlo no cuesta resolución de ajuste porque solo decide a qué canal mira la pantalla y no cómo se aplica el valor. Que ese margen baste no ha podido comprobarse sobre el aparato, porque la pantalla no llega a encenderse, así que la incidencia queda anotada y abierta junto con la anterior.

8.7 Balance de la verificación

Conviene terminar separando tres montones, porque el valor de un capítulo de medidas está en no mezclarlos.

Queda verificado que la cadena de datos es exacta de extremo a extremo, con la interpolación equivalente a su referencia dentro de 1 LSB, el transporte sin una sola muestra alterada y las 128 notas afinadas. Queda verificado que el instrumento no tiene cortocircuitos ni fallos de aislamiento, que la masa es común a las tres placas, que el raíl de 5 V no se hunde al cargarlo, que los raíles internos de 3,3 V son correctos, que los canales del panel recorren el fondo de escala completo, que el conjunto consume 0,42 A y que la salida entrega 198,7 mV eficaces de nivel de línea con un milivoltio de continua. La pérdida de tensión que el capítulo 7 dejó pendiente era, en efecto, resistencia de contacto de la placa de prototipado, y soldar la eliminó.

Queda medido solo como cota lo que la banda del multímetro no alcanza, esto es, el rizado de los raíles y la relación señal-ruido de 33 dB. Y queda deducido, que no medido, el consumo del procesador de audio, entre 130 y 165 mA.

Tampoco se midieron la carga del procesador de audio ni la latencia entre gesto y sonido. Para esta última solo hay una cota calculada, que suma el periodo de lectura del táctil, la transferencia y la transición y queda por debajo de los 75 ms en el peor caso.

Queda sin resolver lo que no cabía en la ventana disponible. El procesador de audio funciona medio voltio por debajo de la tensión de entrada que garantiza su fabricante, y esta es la única de las tres con la causa identificada y la corrección propuesta. El display no llega a funcionar, y lo

comprobado dice dónde no está el fallo, ni en el cable, ni en los resistores, ni en el módulo, pero no dónde está. Los bloqueos siguen apareciendo pese a las dos medidas que el firmware lleva contra ellos, y lo que se puede medir con un multímetro descarta las explicaciones sencillas sin dar con la buena. Y el umbral del foco no ha podido comprobarse en marcha, porque la pantalla no se enciende. Las tres quedan abiertas para una revisión posterior del instrumento, junto con las medidas que este capítulo no ha podido tomar, el pico de arranque, la distorsión y la respuesta en frecuencia. Todas ellas exigen instrumentación de la que no se dispuso, así que la primera partida de una continuación de este trabajo no es material sino un osciloscopio. El capítulo siguiente valora qué significa todo esto para los objetivos que se fijaron al principio.

Capítulo 9

Conclusiones y trabajo futuro

9.1 Síntesis y aportación del trabajo

El trabajo entrega una cadena completa y cerrada, que empieza en un conjunto público de formas de onda y termina en una señal de línea en un conector de 3,5 mm. Entre esos dos extremos hay un autoencoder variacional entrenado en el ordenador, una rejilla de 256 tablas horneada a partir de su decodificador, tres microcontroladores con una responsabilidad única cada uno, dos enlaces de datos verificados muestra a muestra y una carcasa que lo alberga todo. El reparto que hace posible ese encadenamiento se describe en el apartado 3.3. Lo que ese conjunto demuestra es que un espacio tímbrico aprendido se puede recorrer con el dedo sobre hardware de catálogo, con 83,82 euros de materiales según el Apéndice A, sin un ordenador delante y sin ningún circuito integrado hecho a medida.

Conviene dejar dicho con precisión qué papel juega aquí el aprendizaje automático, porque es donde más fácil resulta exagerar. En el instrumento no se ejecuta ninguna red neuronal. El codificador solo intervino durante el entrenamiento y el decodificador se ejecutó fuera de línea para hornear la rejilla, de modo que en marcha solo hay consulta de tabla e interpolación. La inteligencia está en el contenido de las tablas y no en el código que las lee, ya que el hecho de que celdas vecinas suenen emparentadas y de que cruzar el plano produzca una transición tímbrica coherente es producto del latente aprendido y no se obtiene a mano. Precalcular fue una estrategia de despliegue y no una renuncia, y el motivo está en el tiempo de cálculo y no en el espacio, tal como se razonó en el apartado 3.2. El tamaño de memoria, de hecho, no distinguía entre las dos opciones. La rejilla ocupa 512 kB (apartado 4.8) y el decodificador, con sus cerca de 592.000 parámetros, ocuparía del orden de 590 kB cuantizado a ocho bits. Lo que la rejilla compra es un coste de acceso constante y conocido. Lo que paga a cambio es una resolución del plano congelada en el momento de hornearla.

9.2 Cumplimiento de los objetivos

Los tres primeros objetivos específicos corresponden a la fase de preparación y quedan cumplidos. El conjunto de datos homogéneo se construyó resolviendo el remuestreo, la normalización de amplitud y la alineación de fase (apartado 4.2). El autoencoder variacional comprime cada onda a una

coordenada de dos dimensiones, y su plano resultó continuo y bien aprovechado (apartado 4.6), con la diversidad confirmada después sobre la propia rejilla (apartado 4.9). Y esa rejilla se horneó y se exportó en un formato que el firmware compila directamente (apartado 4.8).

Los tres siguientes son de firmware y también quedan cumplidos. El motor sustituye la tabla en reproducción sin discontinuidad audible, con una transición cuyo rizado de pico se midió en 0,34 dB (apartado 6.3). La comunicación entre las tres placas quedó definida y verificada, con la interpolación equivalente a su referencia dentro de 1 LSB y el transporte por SPI sin una sola muestra alterada (apartado 8.1). El control de nota y dinámica por MIDI funciona, con las 128 notas dentro de tolerancia y el la central en 440 Hz sobre el montaje definitivo, y el panel analógico recorre el fondo de escala completo en todos sus canales (apartado 8.3). Este último objetivo se cumple con una salvedad que no conviene disimular, y es que la realimentación visual de los mandos no llegó a funcionar sobre el aparato montado (apartado 8.6).

Quedan el objetivo de validar cada etapa contra una referencia calculada en el ordenador, que atraviesa todo el desarrollo y se recoge reunido en el capítulo 8, y el de integrar el conjunto en un prototipo físico alimentado desde una fuente única, que consume 0,42 A y entrega 198,7 mV eficaces de nivel de línea (apartados 8.4 y 8.5). También este último lleva salvedad, y es que el procesador de audio trabaja medio voltio por debajo de la tensión de entrada mínima que garantiza su fabricante, con la causa identificada y la corrección propuesta (apartado 8.7). Con esas dos salvedades sobre la mesa, el objetivo general se considera alcanzado. Hay un instrumento autónomo que permite explorar un espacio tímbrico aprendido con una pantalla táctil y una entrada MIDI estándar.

9.3 Aprendizajes del proyecto

Por encima de cualquier resultado concreto, el trabajo ha permitido aplicar sobre un mismo objeto contenidos que en el grado se estudian por separado. El aprendizaje automático entrena el modelo, el procesado de señal prepara las ondas y las reproduce, los sistemas embebidos reparten la carga entre tres procesadores, la electrónica analógica resuelve el panel y la alimentación, los protocolos de comunicación unen las placas y el diseño mecánico las encierra en algo que cabe en las manos. Que todas esas materias tuvieran que encajar en un único aparato, y no cada una en su práctica, ha sido lo más satisfactorio del proyecto, y es la razón por la que se eligió este tema y no otro más acotado.

9.4 Trabajo futuro

Las líneas de continuación salen todas de una limitación conocida, y conviene enunciarlas con ella delante. La primera es la que el capítulo 3 dejó abierta a propósito. El decodificador puede llegar a ejecutarse a bordo del ESP32-S3, y el reparto elegido lo permite sin tocar el firmware del procesador de audio, que seguiría recibiendo exactamente lo mismo (apartado 3.2). Antes de intentarlo conviene saber si merece la pena, y esa comprobación no necesita hardware. Como el decodificador no es lineal, interpolar entre dos de sus salidas no equivale a decodificar el punto latente intermedio, de modo que comparar ambas cosas en puntos intermedios daría la cifra de error de aproximación que hoy no existe, y con ella se sabría si una rejilla de 16×16 basta. La segunda línea es la limitación de banda de las tablas, que quedó fuera del alcance porque corre-

gira multiplica el volumen de datos embarcado por el número de versiones filtradas que haya que almacenar (apartado 3.5), y que se resolvería eligiendo la adecuada según el registro de la nota. La tercera es la fidelidad del cuello de botella de dos dimensiones, que redondea los flancos verticales (apartado 4.7) y cuya ampliación obligaría a replantear el control táctil, que es precisamente lo que fijó esas dos dimensiones.

Antes que ninguna de ellas está la revisión del propio aparato. Las tres incidencias que la verificación dejó abiertas siguen abiertas (apartado 8.6). Falta además levantar con osciloscopio y analizador lo que un multímetro no alcanza, esto es, la distorsión, la respuesta en frecuencia y el pico de corriente del arranque (apartado 8.7). La polifonía, descartada desde el principio, solo tiene sentido plantearla cuando todo lo anterior esté resuelto.

El prototipo es lo más perecedero de todo esto. Lo que permanece no está dentro de la caja, sino dentro de mi cabeza. La experiencia de haberlo hecho, de haberlo pensado y de haberlo escrito es lo que queda, y es lo que me permitirá abordar con más seguridad los próximos proyectos. Proyectos serán más ambiciosos, proyectos que espero que me dejen con la misma sensación de haber aprendido algo que no estaba en ningún libro ni en ninguna clase.

Bibliografía

- [1] Ray Wilson. *Make: Analog Synthesizers*. Sebastopol, CA: Maker Media, 2013. ISBN: 978-1449345228.
- [2] Robert Bristow-Johnson. “Wavetable Synthesis 101, A Fundamental Perspective”. En: *Proceedings of the 101st Audio Engineering Society Convention*. Paper 4400. Los Angeles, CA, 1996. URL: <https://aes2.org/publications/elibrary-page/?id=7379>.
- [3] Vesa Välimäki y Antti Huovilainen. “Antialiasing Oscillators in Subtractive Synthesis”. En: *IEEE Signal Processing Magazine* 24.2 (2007), págs. 116-125. DOI: 10.1109/MSP.2007.323276.
- [4] Diederik P. Kingma y Max Welling. *Auto-Encoding Variational Bayes*. 2013. arXiv: 1312.6114 [stat.ML]. URL: <https://arxiv.org/abs/1312.6114>.
- [5] Irina Higgins et al. “beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework”. En: *International Conference on Learning Representations (ICLR)*. 2017. URL: <https://openreview.net/forum?id=Sy2fzU9g1>.
- [6] Samuel R. Bowman et al. “Generating Sentences from a Continuous Space”. En: *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning (CoNLL)*. Berlin, Germany: Association for Computational Linguistics, 2016, págs. 10-21. URL: <https://aclanthology.org/K16-1002/>.
- [7] Jesse Engel et al. “DDSP: Differentiable Digital Signal Processing”. En: *International Conference on Learning Representations (ICLR)*. 2020. arXiv: 2001.04643 [cs.LG]. URL: <https://openreview.net/forum?id=B1x1ma4tDr>.
- [8] Antoine Caillon y Philippe Esling. *RAVE: A variational autoencoder for fast and high-quality neural audio synthesis*. 2021. arXiv: 2111.05011 [cs.LG]. URL: <https://arxiv.org/abs/2111.05011>.
- [9] Lamtharn Hantrakul y Li-Chia Yang. “Neural Wavetable: a playable wavetable synthesizer using neural networks”. En: *NeurIPS Workshop on Machine Learning for Creativity and Design*. 2018. arXiv: 1811.05550 [cs.SD]. URL: <https://arxiv.org/abs/1811.05550>.
- [10] Gordan Kreković. “A Concept of a Wavetable Oscillator Based on a Neural Autoencoder”. En: *Proceedings of the 16th International Audio Mostly Conference (AM '21)*. Association for Computing Machinery, 2021, págs. 240-243. DOI: 10.1145/3478384.3478389. URL: <https://dl.acm.org/doi/10.1145/3478384.3478389>.
- [11] Hazounne Lee et al. *Wavespace: A Highly Explorable Wavetable Generator*. 2024. arXiv: 2407.19862 [cs.SD]. URL: <https://arxiv.org/abs/2407.19862>.

- [12] Kristoffer Ekstrand. *AKWF: Adventure Kid Waveforms*. Colección de formas de onda de un ciclo, licencia Creative Commons Attribution 3.0. Consultado en junio de 2026. URL: <https://www.adventurekid.se/akrt/waveforms/adventure-kid-waveforms/>.
- [13] witnessmenow. *ESP32-Cheap-Yellow-Display*. Repositorio comunitario de documentación de la placa ESP32-2432S028R. Consultado en agosto de 2026. URL: <https://github.com/witnessmenow/ESP32-Cheap-Yellow-Display>.
- [14] Shenzhen XPTEK Technology Co., Ltd. *XPT2046 Touch Screen Controller Data Sheet*. 2007.
- [15] Bodmer. *TFT_eSPI: librería gráfica para pantallas TFT sobre procesadores embebidos*. Ver. 2.5.43. Consultado en agosto de 2026. URL: https://github.com/Bodmer/TFT_eSPI.
- [16] Espressif Systems (Shanghai) Co., Ltd. *ESP32 Series Datasheet*. Version 5.3. Consultado en agosto de 2026. 2026. URL: https://documentation.espressif.com/esp32_datasheet_en.pdf.
- [17] Paul Stoffregen. *XPT2046_Touchscreen: librería de Arduino para el controlador táctil XPT2046*. Ver. 1.4. Consultado en agosto de 2026. URL: https://github.com/PaulStoffregen/XPT2046_Touchscreen.
- [18] Espressif Systems (Shanghai) Co., Ltd. *ESP32-S3 Series Datasheet*. Version 2.2. Consultado en agosto de 2026. 2026. URL: https://documentation.espressif.com/esp32-s3_datasheet_en.pdf.
- [19] Electrosmith. *Daisy Seed Datasheet*. Version 1.2.0. Consultado en agosto de 2026. 2026. URL: https://daisy.nyc3.cdn.digitaloceanspaces.com/products/seed/Daisy_Seed_datasheet.pdf.
- [20] STMicroelectronics. *STM32H750VB, STM32H750ZB, STM32H750IB, STM32H750XB Datasheet*. DS12556 Rev 7. Consultado en agosto de 2026. 2023.
- [21] Electrosmith. *libDaisy: capa de abstracción de hardware para la plataforma de audio Daisy*. Ver. 5.4.0. Versión empleada: revisión 85172e2b, posterior a la etiqueta v5.4.0. Consultado en agosto de 2026. URL: <https://github.com/electro-smith/libDaisy>.
- [22] Electrosmith. *DaisySP: librería de procesamiento digital de señal para la plataforma de audio Daisy*. Ver. 1.0.0. Consultado en agosto de 2026. URL: <https://github.com/electro-smith/DaisySP>.
- [23] MIDI Manufacturers Association. *MIDI 1.0 Detailed Specification*. Document Version 4.2.1. Los Angeles, CA, 1996.
- [24] Solomon Systech Limited. *SSD1306: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*. Revision 1.1. 2008.
- [25] Vishay Semiconductors. *6N138, 6N139: High Speed Optocoupler, 100 kBd, Low Input Current, Photodiode Darlington Output*. Rev. 1.6. Document Number 83605. 2015.

Parte II

Anexos

Apéndice A

Presupuesto de materiales

La Tabla A.1 recoge el coste de los componentes del instrumento descrito en el capítulo 7. Las cantidades son las que hay montadas en el aparato terminado, no las compradas, que en algunos casos son mayores porque se adquirieron unidades de repuesto.

Tabla A.1: Presupuesto de materiales del instrumento.

Referencia	Componente	Cant.	€/ud	Importe
Daisy Seed 1.2	Placa de desarrollo de audio	1	53,25	53,25
ESP32-2432S028	Placa ESP32 con pantalla táctil	1	12,51	12,51
ESP32-S3-N16R8	Placa de desarrollo ESP32-S3	1	4,74	4,74
MIDI Shield	Shield MIDI para Arduino	1	4,73	4,73
Veroboard	Placa de tiras 100×160 mm	1	3,36	3,36
POT10k-BLin	Potenciómetro lineal 10 kΩ	6	0,29	1,71
OLED 0,96in	Pantalla OLED de 0,96 pulgadas	1	1,31	1,31
Switch ON-OFF-ON	Conmutador de palanca	2	0,27	0,54
Jack 3,5 mm	Jack de audio hembra mono	1	0,33	0,33
Tira 22 pines	Tira de pines hembra de 2,54 mm	2	0,15	0,30
Tira 20 pines	Tira de pines hembra de 2,54 mm	2	0,15	0,30
R3,3	Resistencia fusible 3,3 Ω 1 W	1	0,28	0,28
C470u-ELEC	Condensador electrolítico 470 μF	1	0,24	0,24
9V Socket	Conector de alimentación de 2,1 mm	1	0,16	0,16
C100n-CER	Condensador cerámico 100 nF	8	0,005	0,04
1N5817	Diodo Schottky	1	0,013	0,01
R10k	Resistencia de 10 kΩ 1/4 W	2	0,004	0,01
Coste de materiales				83,82

Tres precisiones sobre cómo está construida la tabla. La primera es que los importes no incluyen mermas, es decir, el material desechado durante el montaje; por eso la pantalla OLED figura con una unidad y no con las dos que se compraron, ya que la primera se rompió al montarla. La segunda es que se han omitido los componentes cuyo importe, una vez descontada la merma, redondea a cero céntimos. La tercera es que el presupuesto recoge los componentes electrónicos y la placa que los soporta, y quedan fuera el alimentador externo de 230 V a 5 V, el filamento con el que se

imprimieron las tres piezas de la carcasa, el cable del mazo de panel y la tornillería, de modo que el coste real del conjunto es algo mayor que el que aquí figura.

Conviene señalar que dos de las diecisiete líneas concentran el 78 % del importe, y son las dos placas de desarrollo con procesador de audio y pantalla táctil. Todo lo demás —los seis potenciómetros, las dos pantallas, los conectores, la placa de tiras y la electrónica pasiva completa— suma menos de veinte euros. Es un resultado coherente con el condicionante de partida del apartado 3.1, que exigía construir el instrumento sobre hardware comercial de bajo coste sin fabricar ningún circuito a medida.

Apéndice B

Relación con los Objetivos de Desarrollo Sostenible

Grado de relación del trabajo con los Objetivos de Desarrollo Sostenible de la Agenda 2030.

Tabla B.1: Grado de relación del trabajo con los Objetivos de Desarrollo Sostenible.

Objetivos de Desarrollo Sostenibles	Alto	Medio	Bajo	No procede
ODS 1. Fin de la pobreza.				X
ODS 2. Hambre cero.				X
ODS 3. Salud y bienestar.				X
ODS 4. Educación de calidad.		X		
ODS 5. Igualdad de género.				X
ODS 6. Agua limpia y saneamiento.				X
ODS 7. Energía asequible y no contaminante.				X
ODS 8. Trabajo decente y crecimiento económico.				X
ODS 9. Industria, innovación e infraestructuras.	X			
ODS 10. Reducción de las desigualdades.				X
ODS 11. Ciudades y comunidades sostenibles.				X
ODS 12. Producción y consumo responsables.		X		
ODS 13. Acción por el clima.				X
ODS 14. Vida submarina.				X
ODS 15. Vida de ecosistemas terrestres.				X
ODS 16. Paz, justicia e instituciones sólidas.				X
ODS 17. Alianzas para lograr objetivos.				X

Descripción de la alineación del trabajo con los ODS con un grado de relación más alto

ODS 9. Industria, innovación e infraestructuras. Es el objetivo con el que este trabajo se relaciona de forma directa, y en concreto con su meta de apoyar la investigación científica y mejorar

la capacidad tecnológica. El trabajo no aplica una técnica ya resuelta, sino que lleva un modelo generativo entrenado en un ordenador hasta un instrumento embebido que funciona en tiempo real y sin ordenador conectado, sobre placas de desarrollo de bajo coste. La aportación no está en la interpolación neuronal de tablas de onda, que ya existía en la literatura, sino en resolver los problemas que aparecen al llevarla a un aparato: repartir el trabajo entre tres procesadores según la escala de tiempo de cada tarea, precalcular en el ordenador lo que no cabe ejecutar a bordo y garantizar que la información llegue íntegra entre placas. Ese tipo de trabajo —adaptar una técnica de investigación a hardware asequible— es justamente lo que el objetivo entiende por mejorar la capacidad tecnológica, porque baja la barrera de entrada de una técnica que hasta ahora exigía un ordenador dedicado.

ODS 4. Educación de calidad. El desarrollo completo está publicado en un repositorio de acceso libre, y no sólo el resultado. Incluye los guiones de preprocesado del conjunto de datos, el entrenamiento del modelo, el horneado de la rejilla y las validaciones numéricas, además del firmware de las tres placas y los ficheros paramétricos de la carcasa. Eso permite reproducir el trabajo entero o reutilizar cualquiera de sus partes por separado. A ello se suma que la memoria documenta también los caminos que no funcionaron y por qué, que suele ser la información que más cuesta encontrar y la más útil para quien intente algo parecido.

ODS 12. Producción y consumo responsables. Varias decisiones de diseño del capítulo 7 apuntan en esta dirección, y ninguna se tomó pensando en la sostenibilidad, sino por razones prácticas que resultan coincidir con ella. Las dos placas de desarrollo caras van sobre zócalos y no soldadas, así que se recuperan intactas si la placa que las sostiene falla o si el proyecto se desmonta. Todas las uniones de la carcasa aprietan contra una tuerca metálica embutida en lugar de roscar en el plástico, precisamente porque el plástico aguanta pocos montajes y la pieza dejaría de ser desmontable. El instrumento se construye además con componentes de catálogo, sin ningún circuito integrado a medida, de modo que cualquier pieza averiada se sustituye por otra equivalente. Y el diseño de la carcasa es paramétrico y está publicado, así que una pieza rota se vuelve a fabricar sin rehacer el modelo.